

Reinforcement learning in robotic systems : A review on sim-to-real transfer

Rajesh Tiwari, Shailesh Khapre^{ID*}, Avantika Singh

Department of Data Science and Artificial intelligence, IIIT-Naya Raipur, Sector-24, Atal Nagar, Raipur, 493661, C.G., India

ARTICLE INFO

Keywords:

Sim-to-real transfer learning
Reinforcement learning
Robotic simulator
Domain randomization
Actuator modeling
Progressive neural networks
Policy migration
Robot learning
Sim-to-real gap
Intelligent robotic systems

ABSTRACT

Sim-to-real transfer reinforcement learning has become a pivotal approach for narrowing the gap between simulation environments and real-world robotic applications. While reinforcement learning methods achieve remarkable success in simulation, their direct deployment in real environments remains challenging due to the inherent reality gap caused by mismatches in physical dynamics, sensory inputs, and environmental variability. This paper presents a comprehensive review of recent advances in sim-to-real transfer, emphasizing improved simulation fidelity, actuator-level modeling, and domain randomization encompassing both environmental and robotic parameters. A unified framework is proposed that outlines key processes, including simulation model optimization, strategy transfer, and iterative policy refinement across virtual and real domains. The study also discusses emerging developments such as Progressive Neural Networks for policy migration and modern benchmarking platforms. The review concludes with open challenges and future directions, including automated simulator tuning, adaptive domain adaptation, and establishing theoretical guarantees for robust and generalizable transfer learning.

1. Research status

Currently, reinforcement learning (RL) methods [1] have achieved great success in a series of complex decision-making problems, such as chess and card games [2–4] and real-time strategy games [5,6], recommendation systems [7–9], and autonomous driving [10,11]. Reinforcement learning has also made remarkable progress in tasks such as robot motion control [12,13], robot manipulation [14–17], motion navigation [18–21], and robot football [22,23]. In order to obtain effective robot control strategies, reinforcement learning relies on a large number of interaction samples for training, but obtaining samples in the real environment has high costs and safety risks. In addition, due to the complex structure of robots and the dynamic changes in the real world, the use of reinforcement learning in robot control still faces problems such as effectiveness, safety, and real-time performance. To reduce the demand for real samples, simulation environments are widely used in robot policy learning. Simulation-based robot policy learning has the following advantages: cheapness, Authenticity, Multidimensionality and safety. Cheapness: The physics engine of the simulation environment can simulate the real environment faster than real time. Generate training samples for robot strategy learning at a low cost, thereby improving the training efficiency of robot strategies. Authenticity: The simulation environment can not only simulate the complete motion characteristics of the robot, such as the motion association between joints and joints, but also simulate the physical properties between

the robot and the environment, such as gravity, pressure, friction, etc., thereby establishing a realistic physical model for the real world. Multidimensionality: In a specific robot task, multiple simulation environments can be used to model the real world at different granularities, reflect the environmental properties of the real scene from different levels, and provide data and information related to the real world to meet different application requirements. Safety: There is no actual risk in trial-and-error behavior in the simulation environment, and time-consuming and dangerous tasks in the real world can be repeatedly performed.

In summary, simulation-based reinforcement learning has certain advantages in robot control. In order for the robot to successfully complete a series of operations in the real world, it is necessary to transfer the strategies learned in the simulation environment to the real world. However, due to the reality gap between the simulation environment and the real world [24], including the differences in robot dynamic models on different platforms and the differences in environmental physical properties (such as action perception delay, ground conditions and atmospheric conditions, etc. [25–27]), even the most realistic simulation environment is difficult to model the real world completely. As a result, transferring strategies learned in the simulation environment directly to a real robot often falls short of delivering the expected results. To close the gap between simulation

* Corresponding author.

E-mail address: shailesh@iiitnr.edu.in (S. Khapre).

<https://doi.org/10.1016/j.robot.2025.105327>

Received 21 May 2025; Received in revised form 22 December 2025; Accepted 30 December 2025

Available online 5 January 2026

0921-8890/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

and reality, sim-to-real transfer-based reinforcement learning methods for robotics [24,28–30] tackle the differences between simulated and real-world environments, facilitating the successful transfer of acquired strategies. In recent years, numerous sim-to-real hybrid transfer reinforcement learning approaches have been introduced. Including system identification [31–33], domain randomization [34–37], domain adaptation [38–40], multi-fidelity learning [41–43], etc., it has been widely used in robot tasks such as motion control, motion manipulation and motion navigation, and has achieved great success, providing a practical solution for the optimization of robot behavior control strategies.

Some works have summarized the existing robot sim-to-real transfer learning methods. Zhao et al. [30] briefly introduced the basic concepts and specific technologies in sim-to-real transfer. Dimitropoulos et al. [29] classified the existing sim-to-real transfer methods into simulator methods and adaptive methods according to whether real data is required. The former does not rely on real data, while the latter requires real data for policy transfer. Salvato et al. [24] divided the sim-to-real transfer methods into three categories: domain randomization, adversarial reinforcement learning, and transfer learning methods. Zhu et al. [28] divided the sim-to-real transfer methods used in bionic robot research into four categories according to different application goals: methods based on accurate simulators, kinematic and dynamic models, hierarchical and distributed controllers, and demonstrations. Although these works have summarized the existing sim-to-real transfer learning research, there is a lack of a general framework to comprehensively sort out and classify the existing work. Therefore, this paper comprehensively sorts out the current research and establishes a general sim-to-real transfer learning framework from the perspective of information flow during the execution of the method and the object of action of the intelligent method. Within this framework, sim-to-real transfer reinforcement learning methods are broadly classified into three categories: model optimization approaches tailored to real-world environments, knowledge transfer techniques rooted in simulated environments, and strategy refinement methods that iteratively integrate simulation and reality. Furthermore, the associated theories and practical applications are analyzed comprehensively.

Section 2 of this paper introduces important concepts in reinforcement learning and transfer learning. Section 3 delves deeply into the various methods employed to bridge the gap between simulation and reality, proposing a comprehensive framework for sim-to-real transfer learning. It examines and analyzes the flow of data and information during the transfer process, as well as the objectives of intelligent methodologies. Building on this foundation, it categorizes existing approaches, providing a detailed evaluation of their fundamental distinctions, strengths, and limitations. Section 4 highlights the recent developments that are shaping sim-to-real reinforcement learning, including advances in Real-to-Sim-to-Real (R2S2R) pipelines, automated simulator tuning, and actuator-level modeling that improve the realism of simulation environments. It also discusses the growing role of benchmarking frameworks and shared experimental platforms — such as the Real Robot Challenge, HomeRobot/OVMM, and the NAO migration testbed — that enable more transparent and reproducible evaluation. Section 5 examines the current challenges and emerging prospects, focusing on issues of scalability, safety, and generalization, while calling for unified benchmarking and open collaboration. Finally, Section 6 concludes the paper by summarizing the key insights and emphasizing the ongoing shift of sim-to-real research toward a more systematic, reproducible, and benchmark-driven discipline.

2. Basic knowledge

2.1. Deep reinforcement learning

Reinforcement learning [1] is a machine learning paradigm based on environmental interaction. The agent continuously interacts with

the environment and collects samples, and learns the optimal strategy through the samples to maximize the cumulative return. The interaction between the agent and the environment is modeled as a Markov process (MDP), defined by a five-tuple (S, A, P, R, γ) , where S is the set of states, A is the set of actions, P is the state transition function/dynamic model $P: S \times A \times S \rightarrow [0, 1]$, R is the reward function $R: S \times A \times S \rightarrow R$, and $\gamma (0 \ll \gamma \ll 1)$ is the discount factor, which determines the impact of future rewards on the total return $G = \sum_{t=0}^T \gamma^t R_t$. The reinforcement learning algorithm uses the interaction samples between the agent and the environment, that is, the trajectory sample τ composed of the transition tuple (s_t, a_t, r_t, s_{t+1}) , to optimize the strategy π to maximize the total return expectation, that is, $\pi = \arg \max_{\pi} E_{\tau \sim p(\tau|\pi)}[G(\tau)]$.

In practical scenarios such as robotic control, the state and action spaces are often continuous, high-dimensional, and nonlinear, making it difficult for traditional tabular methods to perform effectively. Moreover, optimal control strategies often require modeling complex nonlinear mappings between sensory inputs and motor outputs. To address these challenges, deep neural networks have been introduced into RL, giving rise to Deep Reinforcement Learning (DRL) [44,45], a synthesis of reinforcement learning and deep learning. DRL inherits the ability of neural networks to approximate complex functions and extract high-level features from raw sensory data, thereby enabling scalable decision-making and control in robotics and other domains. DRL algorithms can be broadly categorized based on their underlying learning mechanisms into value-based, policy-based, actor-critic, model-based, and gradient-free approaches. Each class offers unique advantages concerning sample efficiency, stability, exploration capability, and real-world adaptability.

Value-based DRL methods estimate the expected return (value) associated with each state–action pair and select actions that maximize this value. Early breakthroughs came from the Deep Q-Network (DQN) [46] proposed by Mnih et al. which combined traditional Q-learning with convolutional neural networks (CNNs) to learn value functions directly from raw visual inputs. DQN introduced several stabilizing mechanisms, such as experience replay, memory buffer for decorrelating samples and target networks, which help prevent divergence during training. These innovations enabled agents to outperform humans in various Atari 2600 games, marking a milestone in artificial intelligence. However, DQN’s applicability is limited to discrete action spaces, which constrains its use in continuous control tasks such as robotic manipulation or locomotion. Subsequent improvements like Double Q-Learning [47] and the Dueling Network Architecture [48] addressed issues of overestimation bias and value decomposition but still inherited the discrete-action limitation.

To handle continuous actions, policy-based DRL methods directly represent the policy $\pi(a|s)$ as a parameterized function and optimize it using gradient-based techniques. The Deep Deterministic Policy Gradient (DDPG) [49] algorithm extended this concept by combining deterministic policy learning with an actor–critic framework, where the actor learns a policy and the critic estimates the corresponding value function. DDPG introduced noise-based exploration and achieved high efficiency in continuous control benchmarks. Building on this, Twin Delayed DDPG (TD3) [50] further improved stability through target policy smoothing, delayed policy updates, and double critics, effectively reducing overestimation bias.

Among on-policy methods, Trust Region Policy Optimization (TRPO) [51] and Proximal Policy Optimization (PPO) [52] are widely used in robotic locomotion and manipulation. These algorithms improve gradient stability by constraining policy updates, preventing large, destabilizing parameter changes. The REINFORCE algorithm [53], an early Monte Carlo policy gradient method, forms the theoretical foundation for such approaches, though it suffers from high variance and poor sample efficiency. Modern actor–critic variants mitigate these issues using baseline functions and advantage estimators such as Generalized Advantage Estimation (GAE).

Off-policy algorithms improve sample efficiency by reusing past experiences collected from different policies. A representative example is the Soft Actor–Critic (SAC) [54] algorithm, which introduces an entropy-maximization term into the objective function to encourage exploration and robustness. SAC has become a preferred choice for complex robotic tasks such as mobile navigation, biped locomotion, and shared-control manipulation, where adaptability and safety are crucial. Off-policy frameworks also facilitate integration with imitation learning and curriculum learning, allowing agents to bootstrap from expert demonstrations and progressively handle more complex tasks.

Complementary to model-free methods, model-based RL aims to learn an explicit representation of the environment's dynamics to improve data efficiency. The Probabilistic Inference for Learning Control (PILCO) [55] framework employs Gaussian Process models to predict future trajectories and optimize control policies through probabilistic inference. Its successor, MC-PILCO [56], extends these principles using Monte Carlo approximations, demonstrating high data efficiency in underactuated systems such as the Acrobot and Pendubot. More recent developments, including Model-Based Policy Optimization (MBPO) [57] and Dreamer [58], combine learned dynamics with latent-space planning to unify the strengths of model-free and model-based learning, significantly reducing sample complexity.

Beyond gradient-based optimization, gradient-free or black-box methods have gained attention for handling non-differentiable reward functions or discontinuous dynamics. Augmented Random Search (ARS) [59] demonstrated that simple linear policies trained via random perturbations can rival complex deep architectures in continuous control benchmarks such as quadruped locomotion. Similarly, Genetic Algorithms (GA) [60] and Evolution Strategies (ES) [61] employ population-based search mechanisms to evolve policies without requiring gradient information. Although less sample efficient, these methods are robust to sensor noise, actuator delays, and real-world disturbances, making them practical for robotic deployment.

Contemporary robotics research increasingly emphasizes hybrid and safety-aware RL frameworks, which integrate model-based prediction, off-policy learning, and black-box optimization to balance sample efficiency, robustness, and real-world applicability. The growing availability of high-fidelity simulation environments such as NVIDIA's Isaac Sim, Orbit, and MuJoCo has accelerated scalable training and benchmarking for diverse robotic applications, including legged locomotion, dexterous manipulation, and human–robot collaboration. As deep reinforcement learning continues to evolve, these integrated frameworks are expected to play a pivotal role in bridging the gap between simulated intelligence and physically embodied autonomy.

2.2. Transfer reinforcement learning

In the field of traditional machine learning, transfer learning realizes knowledge transfer between domains based on the difference in data distribution between different domains. Define the source domain as D_S , the learning task in the source domain as T_S , the target domain as D_T , and the learning task in the target domain as T_T . Transfer learning means that when $D_S \neq D_T$ and $T_S \neq T_T$, the knowledge of D_S is transferred to D_T , thereby improving the learning efficiency of the target domain. The domain can be represented as $D = \{X, P(X)\}$, where X is the feature space and $P(X)$ is the marginal probability distribution; the learning task can be represented as $T = \{Y, P(Y|X)\}$, where Y is the label space and $P(Y|X)$ is the prediction function. A classic transfer scenario is that there is a lack of labeled data in a classification task (D_T), while there is sufficient labeled data in another similar classification task (D_S), but the data in the latter may follow a different data distribution ($P(X_S) \neq P(X_T)$) or there may be different association patterns between samples and labeled information. In this case, using transfer learning to complete the transfer of knowledge will avoid expensive data labeling work, thereby greatly improving learning performance. Traditional transfer learning methods can be categorized

into four main types: instance-based, feature-based, parameter-based, and relationship-based approaches [62]. Instance-based methods [63, 64] are best suited for scenarios where the source and target domains exhibit significant similarity. These methods focus on selecting specific data from the source domain for reuse, minimizing differences between domains by modifying the sample format. Feature-based approaches [65,66] are effective in cases where domain similarity is relatively low. They rely on feature transformation to ensure that the source and target domains exhibit similar characteristics within a shared feature space. Parameter-based methods [47,67] approach transfer learning from the model perspective, enabling the sharing of certain parameters between source and target domain models to facilitate knowledge transfer. Lastly, relationship-based methods [68,69] focus on transferring the relationships or connections between data in the source domain to the target domain by mapping the correlation knowledge between the two.

In the field of reinforcement learning, transfer learning specifically focuses on the correlation and difference of data distributions such as states, actions, and rewards between different domains. Whether the source domain and the target domain have the same state space S or action space A will directly affect the difficulty of transfer learning, applicable methods, and other issues [70,71]. The goal of transfer learning in reinforcement learning problems is to extract knowledge from components such as trajectory samples and value functions and transfer them to the target domain to improve the performance of the strategy in the target domain: let M_S represent the source domain, M_T represent the target domain, $I_S \sim M_S$ is the external information obtained from the source domain, and $I_T \sim M_T$ is the internal information in the target domain. The ultimate goal of transfer learning is to obtain the optimal strategy π^* in the target domain, that is, $\pi^* = \arg \max_{\pi} E_{a \sim \pi}[G]$, where π is the mapping from state to action learned based on I_S and I_T . Common methods in transfer reinforcement learning [71,72] include reward function design [73,74], example learning [75,76], policy transfer [77,78], task-to-task mapping [79,80], and representation transfer [80,81]. Reward function design methods refer to designing a reasonable reward function based on information collected from the source domain to guide policy learning in the target domain to improve learning efficiency and policy performance; example learning methods achieve knowledge transfer through example samples generated by human guidance or expert strategy and environment interaction. When the MDP of the source domain and the target domain are the same or very similar, example samples can be used to learn the optimal policy in the target domain through methods such as imitation learning; policy transfer methods focus on how to transfer the pre-trained policy in the source domain to the target domain, and extract and transfer the knowledge in the source domain policy through methods such as policy distillation and policy reuse; task-to-task mapping methods assume that there is a mapping relationship between the state, action, reward function, etc. of the source domain and the target domain, and solve the mapping relationship manually or by learning methods to achieve transfer learning; The object of transfer of representation learning methods is the feature representation learned from the source domain, extracting features from components such as value functions and policy networks, and then directly or further deconstructing them and transferring them to the target domain.

2.3. Simulation environment

Robot transfer reinforcement learning places stringent demands on both the fidelity and sampling efficiency of the simulation environment. The choice of simulation infrastructure ranging from the underlying physics engine to the task-level environment has a decisive influence on the quality of learned policies and their transfer performance in real-world applications. Before presenting the comparative analysis in Table 1, it is essential to clarify the distinction among the three core layers of robotic simulation: physics engines,

Table 1
Mainstream simulation environments.

Category	Simulation environment	RL environment	Advantage	Shortcoming
Physics simulator: Traditional control	Gazebo [82] + ROS	NA	Completely open source and free, with rich algorithm and model libraries; facilitates real machine migration under the ROS framework	Insufficient reference documentation; difficult for beginners
	Webots [83]	NA	Open source, provides multiple APIs, ROS interface, supports various sensors, and has good visualization tools	Limited RL framework and interface support; small community
	VREP [84]/PyRep [85]	RLBench [86]	Accurate physics simulation; supports multiple programming languages	Commercial license required for full features
Physics simulator: RL-oriented	MuJoCo [87]	OpenAI Gym [88]/ROBEL [89]/DeepMind Control Suite [90]/Surreal Robotics Suite [91]	Open source, efficient simulation with diverse RL tasks; large community support	Differences from real-world physics; moderate computational cost
	PyBullet/Bullet [92]	Roboschool/PyBullet Gym/GibsonEnv [93]/PyRoboLearn [94]	Free and fast physics engine; supports reinforcement learning integration and robotics research	Simplified physics and limited visual realism compared to modern engines
Game engine	Unreal Engine/Unity3D	AirSim [95]/Carla [96]	Highly realistic rendering and environment simulation; multiple task scenarios supported	High computing demand and long simulation time
Differentiable simulator	Brax [97]	NA	Combines deep neural controllers and physics modules for differentiable optimization	Still immature; lacks complete integrated platform for robotics RL
	Tai Chi [98]	NA	Supports GPU acceleration and differentiable physics modeling	Complex implementation; limited robotics-oriented libraries
	JBDL [99]	NA	Provides gradient-based physical modeling useful for control and learning	Still experimental; small community and incomplete tooling
GPU simulator	RaiSim [88]	NA	Supports GPU-based parallel simulation; high efficiency and stability	Limited visual realism and soft-body support
	Isaac Sim/Isaac Lab [100]	Isaac Lab (integrated)	Massively parallel GPU-based RL training; photorealistic environments; tight integration with NVIDIA Omniverse	Requires NVIDIA hardware and high-end GPU; steep learning curve for customization

simulators, and simulation environments. A physics engine forms the computational foundation of any robotic simulator. It is responsible for performing low-level numerical computations such as collision detection, force calculation, joint constraint solving, and rigid- or soft-body dynamics. Well-known physics engines include PhysX, MuJoCo [101], Bullet [92], and ODE, each offering trade-offs between accuracy, computational speed, and extensibility. For example, MuJoCo emphasizes high-precision dynamics suitable for control research, PhysX provides efficient GPU acceleration for large-scale simulations, while Bullet and ODE offer robust open-source implementations widely integrated into academic and industrial tools. Built upon these physics engines, a simulator serves as an integration framework that couples the physical computation core with additional modules for robot modeling, sensor simulation, environment rendering, and data visualization. Examples include Gazebo [82], Webots [83], Isaac Sim [100], and PyBullet [92]. These platforms enable researchers to design complex robotic scenes, define robot kinematics and dynamics, and interface with control algorithms via standardized APIs. For instance, Gazebo supports ROS/ROS 2 integration for multi-robot interaction in both indoor and outdoor environments, Webots provides a user-friendly interface for prototyping and educational use, while Isaac Sim leverages NVIDIA Omniverse to achieve photorealistic rendering and advanced sensor modeling for perception-based control. At the highest abstraction level, a simulation environment defines the task logic required for reinforcement learning and policy evaluation. It specifies the observation, action, and reward structures that guide the agent's interaction with the virtual world. Representative examples include OpenAI Gym [87], DeepMind Control Suite [90], and Isaac Lab [100], which build upon existing simulators to standardize experimental setups and facilitate reproducibility.

These environments provide pre-defined benchmarks and APIs that enable consistent training, evaluation, and comparison of algorithms across different research groups. Understanding the functional distinctions among physics engines, simulators, and simulation environments is crucial for selecting the most appropriate tools for robot transfer reinforcement learning. Table 1 summarizes several commonly used simulation platforms, comparing them across typical application scenarios, computational performance, and ease of integration with reinforcement learning frameworks.

In contrast to the Gazebo [82] physics simulator, contact dynamics-based physics engines like MuJoCo [101] and PyBullet [92] are extensively utilized in robot reinforcement learning research. Based on these physics' engines, reinforcement learning integrated environments such as OpenAI Gym [87] and DeepMind Control [90] are constantly being proposed. Among them, MuJoCo [101] focuses on robot and biomechanics simulation as well as animation and machine learning applications, supports common robot task scene simulation, and has a very efficient sample collection efficiency, which meets the huge demand for samples in reinforcement learning; PyBullet [92] is based on the Bullet physics engine and also has a good sample collection efficiency. It combines robot control with machine learning, supports loading multiple model formats, and has a large community to provide support for developers.

The above robot simulation platforms focus on simulation versatility, but there is still a certain gap between their authenticity and specific practical application scenarios, resulting in large errors in their simulations in certain specific scenarios. Therefore, some high-fidelity simulation platforms for specific robot scenarios have attracted attention, such as SimSpark [102] dedicated to the Robot Football

World Cup, ANYmal [103] for hybrid quadruped robots, ATRIAS [104] for bipedal robots, and high-resolution optical tactile sensor simulator Tactile Gym [105] and further expanded Tactile Gym 2.0 [106], visual tactile sensor GelSight [107], etc. In addition, game engines such as Unreal Engine and Unity3D have been gradually applied to robot reinforcement learning. They have realistic simulation image quality and rich task scenarios, and are especially suitable for tasks such as unmanned driving that have high requirements for image quality and training scenarios. Among them, AirSim [95] focuses on simulation plug-ins for unmanned vehicles and unmanned aerial vehicle control.

In the latest research on simulation environments, differentiable simulators and modern GPU-based frameworks have become central to advancing robot learning and sim-to-real transfer. Differentiable simulators integrate deep neural network controllers with physics engines, allowing gradients to propagate through the environment for dynamic optimization of both control policies and simulation parameters. This integration significantly enhances both simulation efficiency and learning performance. Representative works include Taichi [98], a framework for automatic differentiation; Brax [97], developed by Google; and JBDL [99], a robot body dynamics library based on JAX [108] developed by Tencent.

In parallel, the NVIDIA Isaac ecosystem has evolved through three major platforms — Isaac Gym, Isaac Sim, and Isaac Lab [100] — each targeting different levels of fidelity, scalability, and realism in robot simulation. Isaac Gym was the first large-scale GPU-accelerated simulator built on NVIDIA PhysX, enabling thousands of parallel robot environments and direct GPU-based reinforcement learning using PyTorch. It prioritized speed and policy throughput over photorealism. Isaac Sim, built on the Omniverse Kit platform, succeeded Isaac Gym by providing high-fidelity, photorealistic simulation with full ROS/ROS 2 integration, domain randomization, and sensor simulation, which are essential for perception-to-control tasks. The latest framework, Isaac Lab, unifies the strengths of its predecessors by combining the GPU-parallelized physics of Isaac Gym with the extensible and physically accurate scene management of Isaac Sim. It introduces modular APIs for defining agents, policies, and environments and serves as a standardized research benchmark for large-scale robot learning and sim-to-real transfer studies.

Similarly, other GPU simulators such as RaiSim [88] provide real-time, high-performance physics simulation for rigid and flexible body dynamics, supporting applications across robotics, virtual reality, and physical interaction. These advancements mark a significant leap toward scalable, high-fidelity, and reproducible training environments for Robotic RL.

3. Sim-to-real transfer reinforcement learning

Sim-to-real transfer [24,30,109] refers to the process of training a robot in a simulation environment, and then migrating and deploying the obtained sim-to-real migration to the real robot control system. The simulation environment serves as the source domain, the real-world environment acts as the target domain, and the robot's control strategy functions as the transfer object. The primary goal is to facilitate the transfer of environmental and control information between the simulation and real-world environments. Sim-to-real transfer proposes four important goals for the migration effect: effectiveness, efficiency, safety, and generalization. Effectiveness [110,111] means that the strategies learned in the simulation environment can play the expected role in the real world, that is, the strategies that obtain high returns in the simulation environment can also obtain high returns in the real environment. Most of the research on effectiveness starts from the perspective of narrowing the environmental differences. Efficiency [112,113] starts from the time and economic cost of sampling, and emphasizes the minimum exploration sampling in the real world. Research on efficiency focuses on the full mining and utilization of

environmental information in real samples. Safety [114,115] emphasizes that exploration and direct strategy deployment in reality have potential safety risks and high costs. It is necessary to guide real robots to conduct safe exploration and sample collection. Generalization [116, 117] focuses on multi-task migration scenarios and emphasizes that the obtained strategies can quickly adapt to tasks in different environments with as little fine-tuning as possible. This paper mainly focuses on the relevant research work carried out around the effectiveness and efficiency of sim-to-real transfer process, and on this basis, summarizes the existing methods.

This paper presents a comprehensive Sim-to-Real Transfer Reinforcement Learning (SRTRL) framework, focusing on the flow of information and the role of intelligent methods within the transfer learning process, as illustrated in Fig. 1. Reinforcement learning (RL) enables autonomous agents to acquire control policies through trial-and-error interactions with an environment, guided by reward signals. However, directly training RL policies on physical robots is often impractical due to high sample complexity, safety risks, and hardware wear. To overcome these limitations, sim-to-real transfer reinforcement learning has emerged as an effective paradigm where agents are first trained in high-fidelity simulators and then transferred to operate reliably in real-world robotic systems. This framework unites the principles of reinforcement learning, which focuses on autonomous policy optimization, with those of transfer learning, which enables knowledge reuse across domains. In sim-to-real RL, the simulation environment serves as the source domain, and the real robot system acts as the target domain. The central challenge lies in bridging the reality gap, which is the discrepancy between simulated and real-world dynamics, sensor noise, latency, and contact interactions.

Building upon these foundations, the proposed framework structures the strategy learning and transfer process into five key stages:

(1) Model optimization based on real environment: narrow the gap between simulation environment and real environment by modifying or adjusting the simulation model. (2) Simulation strategy optimization: learn and train the strategy through interactive samples of simulation environment. (3) Knowledge transfer based on simulation environment: transfer the learning knowledge acquired in simulation environment to real environment. (4) Real environment exploration and evaluation: fully explore the real environment and evaluate the performance of deployment strategy. (5) Strategy iterative improvement based on virtual and real environment: the above four steps are iterated to achieve continuous improvement of strategy.

The real environment information includes the dynamic model and reward model in the real task scenario. The former is affected by complex environmental parameters such as temperature, humidity, friction coefficient, etc., while the latter is determined by the final task goal and reward structure. The complex real environment is difficult to be accurately and completely represented, but the real environment information contained in the sample data, evaluation indicators and human prior cognition of the task can be effectively utilized. This information is contained in the physical equations, environmental parameter estimates and parameter range constraints used by humans when establishing the simulation environment. The simulation model optimization based on the real environment corrects or adjusts the simulation environment through real-world sample data. Its essence is to integrate the prior but inaccurate information in the simulation model with the real but local information in the real sample to restore the real environment. In the simulation environment strategy optimization, the robot interacts efficiently with the simulation environment to generate a large number of cheap samples and uses the existing deep reinforcement learning algorithms such as DDPG [49] and PPO [52] to achieve strategy optimization. In this process, the simulation environment information after integrating the real environment information is converted into the control information in the simulation control strategy. In the knowledge transfer process based on the simulation

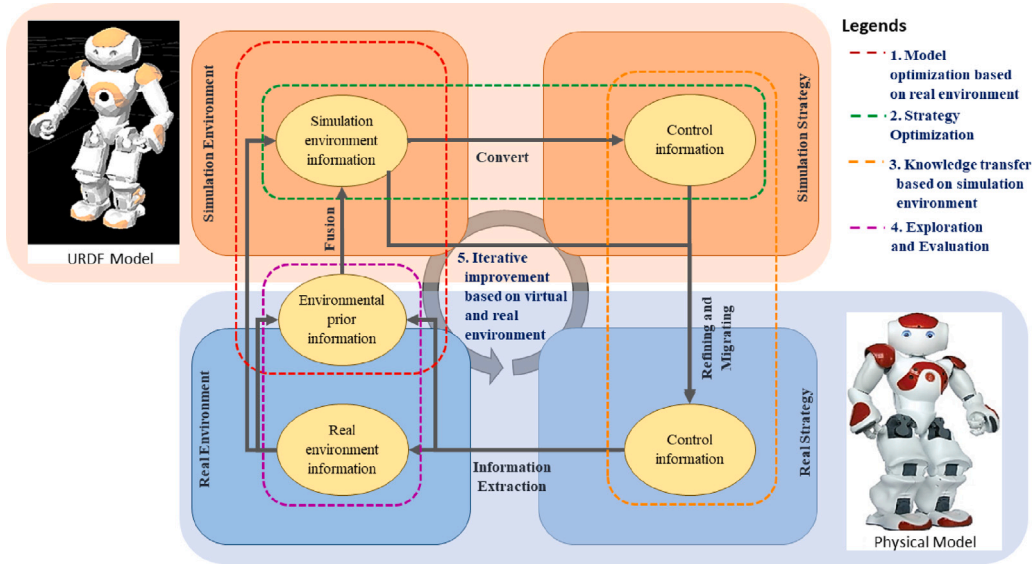


Fig. 1. Sim-to-real transfer reinforcement learning process framework.

environment, the optimized policy control information from the simulation environment is extracted and transferred to the real-world policy through methods such as decomposition, recombination, or direct migration. Finally, the migrated real policy is used for real environment exploration and evaluation. Local information is extracted from the real environment through sample collection and policy evaluation. The control information of the real policy is used to guide the robot to explore the unknown parts of the real environment to establish a more accurate environment model.

Simulation environment strategy optimization, real environment exploration and evaluation in sim-to-real transfer are independent research directions in the field of traditional reinforcement learning. Existing sim-to-real transfer reinforcement learning methods usually focus on three parts: simulation model optimization based on real environment, knowledge migration based on simulation environment, and policy iteration improvement based on sim-to-real environment. Therefore, this paper focuses on classifying and summarizing sim-to-real transfer methods and representative works from the above three perspectives, and explains the intrinsic motivation and relationship of each type of method.

3.1. Model optimization method based on real environment

This approach focuses on refining the simulation environment to more closely resemble real-world conditions or improving training efficiency by configuring suitable environmental and robotic parameters. As illustrated in Fig. 2, such methods determine a set of simulation environments based on real-world characteristics and then select a specific environment for policy training. The key techniques used for environment determination and selection include system identification, domain randomization, curriculum learning, and multi-fidelity simulation. From the perspective of transfer objectives, system identification and domain randomization aim to minimize the gap between simulated and real environments by adjusting or randomizing simulation parameters. In contrast, curriculum learning and multi-fidelity simulation optimize the simulation setup for a given task, thereby accelerating policy learning and training.

3.1.1. System identification

System identification [118] methods make the simulation environment provided by the simulator more realistic by establishing an

accurate mathematical model for the physical system and calibrating the model accurately. They have been widely used in the field of robotics [119–121], involving a series of problems such as kinematic calibration, adaptive control, environmental prediction, and friction estimation [31,33]. As shown in Fig. 2, system identification dynamically adjusts the model parameters of the simulation environment according to the real environment to reduce the difference between the corresponding simulation environment and the real environment. The model constructed by system identification needs to undergo continuous modification and adjustment. After determining the appropriate mathematical model, static parameters can be used to represent the component models of these real systems, and parameter optimization can be performed through methods such as least squares method, covariance method, spectrum analysis, and time series analysis [121]. In addition, some end-to-end methods, such as artificial neural networks, are also widely used to represent the models of system components. Allevato et al. [122] constructed a network model TuneNet, which uses a neural network to directly adjust the parameters of one model to match another model. The observations from two different models (i.e., the simulation environment and the real world) are used as input to estimate the difference in parameters between the models, and the iterative residual tuning technique is used to perform fast and accurate one-time system identification, directly adjusting the system module parameters of the simulation environment to match the real environment model, which improves the tuning speed and reduces the number of simulation samples required. As shown in Fig. 3, the simulation environment and the real world use the dynamic model f_s, f_r , and observation functions z_s and z_r of the current environment to obtain the observation values O_{sim} and O_{real} , respectively. The parameter error is directly estimated based on the observation value and the neural network h_θ , and the parameters are updated by $\xi_k = \xi_{k-1} + \Delta\xi$. This iteration is repeated until the following optimization goal is achieved, where α is the weight regularization constant.

$$\arg \min_{\pi} \sum_{n=1}^N \left\| (\xi_{k_n} + h_\theta(O_{sim_n}, O_{real_n})) - \xi_{r_n} \right\| + a \|\theta\|^2 \quad (1)$$

Du et al. [123] remodeled system identification as a parameter search problem, established a model to predict the relationship between the current simulation environment parameters and the real environment parameters, and then searched for the appropriate target parameters on this basis. In order to model real-world objects more

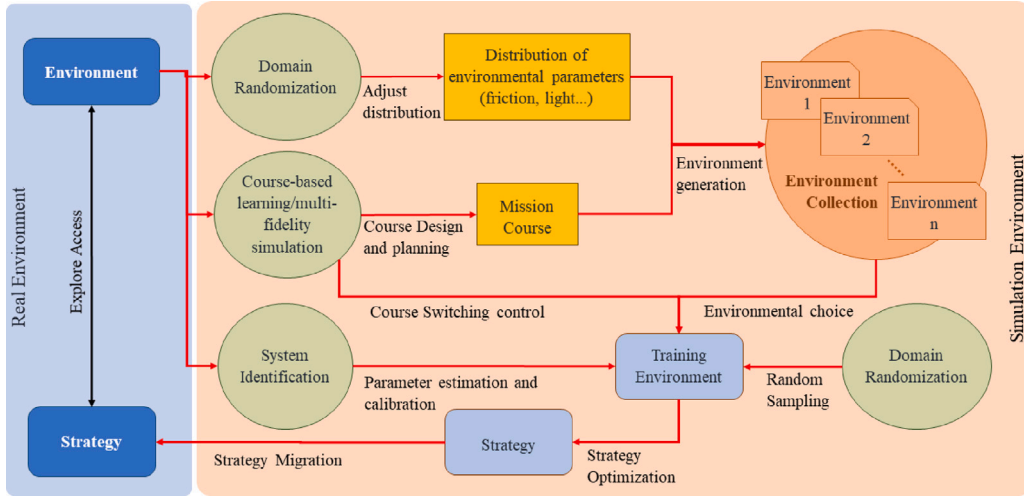


Fig. 2. Model optimization method based on real environment.

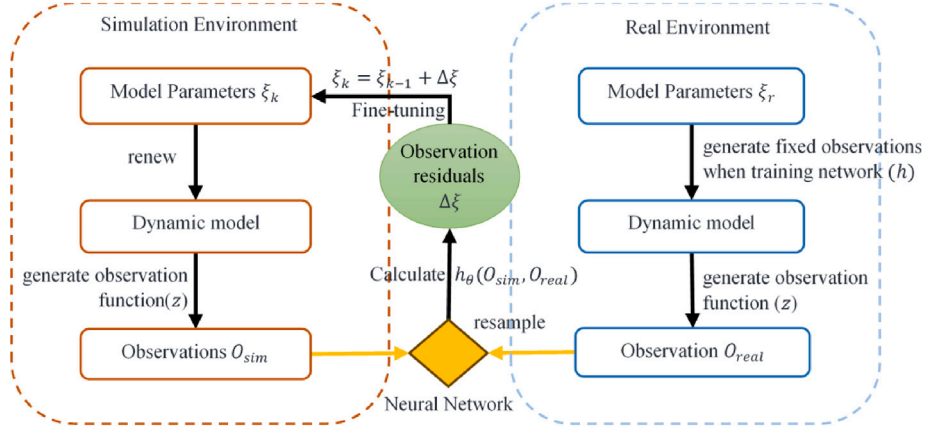


Fig. 3. TuneNet network model structure.

accurately, Gao et al. [124,125] established a dataset Object Folder consisting of 100 3D objects represented by implicit vision, hearing and touch, which promoted the research on robot grasping, motion synergy, etc., and bridged the gap between simulation and reality in tasks involving random dynamics. Kumar et al. [126] proposed a fast motor adaptation algorithm RMA for the problem of real-time online adaptation of quadruped robots. The central idea is: assuming that the current state of the system is generated by the robot in a specific environment, the implicit variables of the current environment information can be inferred based on the past state and action information. In the real deployment stage, the adaptation module trained in the simulation stage is used to perform online system identification and output the joint angles of the robot's actions. Based on this work, The researchers subsequently verified the effectiveness of the algorithm on a bipedal robot [127], a robotic arm [128], and a quadcopter [129].

Although accurate simulation models can effectively narrow the gap between virtual and real environments, it is still very challenging to directly model robot systems in real environments. There are problems such as lack of observation samples and estimation errors. In addition, the operation mechanism of robots is usually very complex, and factors such as response delay and joint damping need to be considered [130, 131]. In addition, the accuracy of system identification modeling usually depends on the experience of researchers, and the accuracy of modeling for some dynamic environments is still lacking. Therefore, proposing a modeling method with smaller errors and autonomy will be the focus of the next stage of system identification research.

3.1.2. Domain randomization

Since real robot systems are difficult to accurately model, and as temperature, humidity, location or time changes, the physical parameters of the same robot system may change significantly, thereby exacerbating the difficulty of system identification. In addition, the simulation environment parameters are constantly adjusted, it is expensive to adapt to environmental changes, and it is difficult to ensure accuracy [132]. The domain randomization method aims to generate a robust control strategy by randomizing the simulation environment to solve the problem of system identification that is difficult to adapt to the dynamic environment and identification. The problem of insufficient accuracy is widely used in different robot control tasks [34–37]. The principle of domain randomization is as follows: Although there are differences between the simulation environment and the real world, it is not necessary to accurately identify all parameters of the real world. Instead of modeling, the simulation environment is randomized according to the distribution of environmental parameters during the training process to cover the real dynamic environment and obtain a more robust strategy [30,133]. As shown in Fig. 2, in the migration framework, the domain Randomization adjusts the random distribution of environmental parameters according to the real environment. Each training randomly samples a set of environmental parameters according to this distribution and generates a corresponding simulation environment for robot training and learning. Specifically, a set of environmental parameters determines the dynamic model $p_{\xi} p(\xi) = p(s_{t+1}|s_t, a_t, \xi)$, of the simulation environment, and the training objective of the simulation strategy is redefined as maximizing the reward

in the environment corresponding to the distribution $\xi \sim p(\xi)$, that is: $\max_{\xi} E_{\xi \sim p(\xi)} [E_{r \sim p(r)} [G(\tau)]]$. The core of domain randomization lies in the selection of parameters and the determination of distribution. Common methods include obtaining environmental parameters and their distribution through expert knowledge and manual design [132,134] or through learning-based approaches [110]. Since the selection of random parameters has a great influence on the final migration effect, different random parameters are usually selected according to different tasks. For example, in the sim-to-real migration task based on image input [35,135], basic visual attributes such as color, lighting properties, texture, and interference noise are widely selected as randomization attributes. In addition to these perceptual factors, recent developments have extended randomization to robot-specific physical parameters, including link inertia, rotor inertia, joint zero-offsets, actuator dynamics, and sensor calibration errors, as these factors have been shown to play a crucial role in achieving robust policy transfer across varying hardware platforms.

Alhounaim et al. [35] studied the importance of different environmental random parameters in image-based pose estimation tasks and found that image quality, interference noise, and texture have a great influence on the effect of domain randomization. James et al. [113] combined domain randomization and adaptive methods to convert random rendered images and real-world images into unified standard simulated images through adversarial networks, greatly improving the efficiency of policy training.

For the distribution of random parameters, current domain randomization methods mostly sample domain parameters from artificially designed fixed prior distributions, such as dynamic randomization methods for low-fidelity simulations [134], application of domain randomization for posture estimation [136], and policy optimization algorithms based on domain randomization and transferability (SPOTA) [137]. Muratore et al. [133] proposed a Bayesian domain randomization (BayRn) policy search algorithm, the core of which is to train policies in a random simulation environment and use Bayesian optimization to adapt to the source domain distribution during the learning process. Compared with earlier work, this algorithm is not limited to a fixed distribution; instead, it constructs a probabilistic model between domain distribution parameters and policy performance, requiring only limited real-world interaction during optimization. In addition, Muratore et al. [138] also used Bayesian optimization to propose neural post-domain randomization (NPDR) to adjust simulation environment parameters, getting rid of some restrictive assumptions on the distribution of domain parameters in domain randomization. As an efficient and convenient virtual-to-real migration method, domain randomization has been widely used to bridge the gap between simulation and reality. However, several open challenges remain. Many approaches still rely on fixed prior distributions; others lack adaptability in adjusting randomization parameters and depend heavily on manual heuristics. Moreover, in high-dimensional robotic systems, where both environmental and mechanical parameters (e.g., link and rotor dynamics) must be considered, the efficiency and scalability of domain randomization still need further improvement.

3.1.3. Curriculum learning and multi-fidelity simulation

The concept of curriculum learning was first proposed by Bengio et al. [139] in 2009. Drawing on the human learning sequence from easy to difficult, it advocates that the model should start learning from easy samples and gradually advance to complex samples and knowledge. Curriculum learning assigns different weights to training samples of different difficulty levels. As the training process continues, the weight of simple samples decreases and the weight of difficult samples increases. The process of dynamically assigning weights to samples is called curriculum. In robot transfer reinforcement learning, curriculum learning has become one of the effective methods to effectively solve problems such as high sample complexity and sparse reward values [140,141]. As shown in Fig. 2, for the sim-to-real transfer

scenario, curriculum learning manually or automatically designs a set of learning tasks from easy to difficult based on the real target task. Each task is modeled as a Markov process and corresponds to an independent task environment, which together constitute a set of simulation environments. As a key part of curriculum learning, the design of the curriculum and the switching mechanism directly act on the model optimization of the simulation environment. By setting a reasonable task environment to reduce the demand for high-cost samples, the problem of sparse reward values is solved and the training efficiency is improved.

In response to the problem of course design and planning, Florensa et al. [142] proposed a course design method for the sparse reward value problem, which realizes the transition of tasks from simple to complex according to the distance between the initial state and the target state of the robot in different course tasks. Nguyen et al. [143] combined course learning with the hierarchical structure of manipulators and mobile robots to propose a hierarchical reinforcement learning framework, using the dependency relationship and hierarchical structure of learning tasks to generate courses, and realizing task switching through intrinsic motivation exploration and active imitation learning. Sukhbaatar et al. [144] and He et al. [145] adopted the idea of adversarial learning, using the upper-level generation strategy to generate course tasks for the underlying action strategy to conduct course learning. The two-level strategies are trained simultaneously and independently to ensure that the difficulty of the generated course tasks is moderate to meet the training efficiency requirements. In response to the problem of course switching control, Shukla et al. [146] designed a variety of online methods to plan the order of course learning based on the reward returns and task characteristics of specific tasks, and completed the migration from simulation to real objects in multi-target navigation tasks. Matiisen et al. [147] modeled task switching as a multi-armed bandit problem. The next training return of the strategy in a task is fitted based on the historical training records, and the exploration method is used to select the task with the fastest growth or decline in performance on the task as the next training task.

According to the idea of curriculum learning, multi-fidelity learning conducts policy training in simulation environments of different fidelity, where low-fidelity and high-fidelity simulation environments correspond to simple and difficult training tasks, respectively. By rationally arranging the switching of policies in different simulation environments, the number of samples required for physical robots can be greatly reduced, saving training costs and time. The multi-fidelity learning framework proposed by Cutler et al. [41] in 2015 allows the agent to stay on the lowest fidelity simulator that still provides it with useful information for policy training, thereby reducing the expensive samples required in the high-fidelity environment. In subsequent work [42], the authors formed a closed loop between simulation environments of different fidelity and real-world policy learning. As shown in Fig. 4, in a simple simulation environment Σ_S , the optimal policy π_S^* is obtained through traditional control methods such as classical, adaptive, and optimal control. Combining the K-means method and the regularized radial basis function (RBF) network, π_S^* is transformed into the initial policy Σ_S for a complex simulation environment π_c^{init} . Then, learning-based methods such as the reinforcement learning PILCO algorithm [148] are used to optimize the initial strategy in a complex simulation environment. The obtained strategy π_c^{new} is then used to guide learning in the real world Σ_{rw} . Observational data from the real world is used to evaluate whether π_c^{new} is optimal. If it is not optimal, $\pi_c^{init} = \pi_c^{new}$ is set, and the strategy is iteratively updated in combination with the observational data, thus achieving a sim-to-real closed loop. Di Castro et al. [43] proposed an effective method to balance high-throughput, low-cost, low-fidelity simulation samples and low-throughput, high-cost, high-fidelity real samples, maintaining a separate experience pool for each environment and designing training weights to balance the impact of different samples on policy training. Compared with the traditional method of continuously increasing

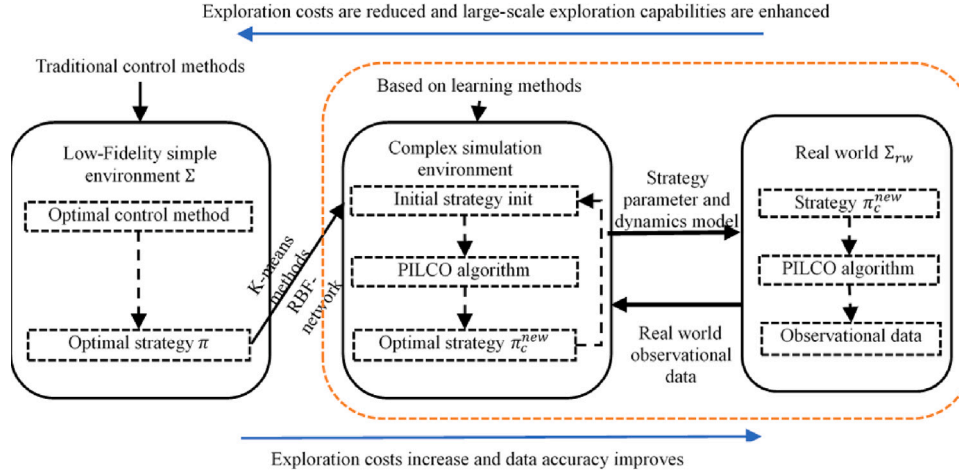


Fig. 4. Multi-fidelity learning closed-loop framework.

fidelity, Truong et al. [149] proposed the idea of improving the sim-to-real transfer effect by reducing fidelity, and decomposed the policy into a "high-level policy" trained only in simulation and a "low-level controller" designed entirely on hardware to alleviate the problems of slow simulation training and overfitting.

As the application scenarios of curriculum learning become increasingly diverse, it is also important to summarize and sort out this field. Wang et al. [150] comprehensively reviewed curriculum learning from various aspects such as motivation, definition, theory and application, and explained in detail how to design a manually predefined course or an automatic course. They proposed a unified curriculum learning framework to divide curriculum learning algorithms into two major categories and multiple subcategories. Zhou et al. [151] further released the world's first open-source curriculum learning library CurML, which integrates existing curriculum learning algorithms into a highly encapsulated unified framework with high ease of use. At present, research on curriculum learning and multi-fidelity simulation still faces some problems. For example, when presetting courses, due to the differences between human thinking and machine thinking, many humans think that simple samples are more difficult for the model. This inconsistency in human-machine decision boundaries can easily lead to problems such as unreasonable course gradient setting. In addition, curriculum learning also has problems such as expensive expert knowledge and lack of flexibility. Future research directions for curriculum learning focus on establishing evaluation data sets and indicators, providing more complete theoretical analysis and algorithms, etc.

3.2. Knowledge transfer method based on simulation environment

The knowledge transfer method based on simulation environment focuses on how to transfer the knowledge obtained in the simulation environment to the real strategy. As shown in Fig. 5, it can be divided into sample-based knowledge transfer and policy-based knowledge transfer. The former transfers the simulation sample containing environmental information, and the latter transfers the simulation strategy containing control information. Sample-based knowledge transfer focuses on the transfer and utilization of simulation trajectories, specifically, transferring simulation samples to the real sample experience pool through importance weights and imitation learning. Policy-based knowledge transfer focuses on simulation strategy decomposition and component migration, and divides and migrates strategies by function or level.

3.2.1. Migration and utilization of simulation trajectories

Given the inherent connection between the simulation environment and the real environment, the extensive sample data generated by the simulation can partially capture the dynamic and reward models of the

real environment. The main focus for optimizing real-world strategies and improving training efficiency will be how to effectively utilize these simulation samples. As shown in Fig. 5, the migration of simulation samples to the real sample experience pool through importance weights and imitation learning is an important method to use simulation trajectories. Through these methods, simulation samples greatly expand the training sample pool of the real strategy, provide rich demonstration information for the real strategy, thereby improving training efficiency and reducing the demand for real samples.

Importance weighting is a common method in the field of transfer learning [152,153]. This method weights the source domain samples by estimating the likelihood ratio between the target domain samples and the source domain samples, and uses the weighted source domain samples in the training of the target domain. For the application scenario of reinforcement learning, let the training sample be a , then the importance weight of the sample $\tau_i = (s_i, a_i, s'_i, r_i)$ in the source domain M_s after migration to the target domain M_r should be set to $w_i = \frac{p(\tau_i|M_r)}{p(\tau_i|M_s)}$, where $p(\tau_i|M) = P(s'_i|s_i, a_i) \cdot R(r_i|s_i, a_i)$, the importance weight is jointly determined by the dynamics model $P(s'_i|s_i, a_i)$ and the reward model $R(r_i|s_i, a_i)$. Lazaric et al. [154] first calculated the correlation measure of the overall samples between different domains based on the reward model and the dynamics model to measure the fit of the sample migration as a whole, and then calculated the similarity measure of the individuals between the samples to reflect the uniqueness of different samples in the same domain. The two were combined to obtain the final importance weight. Tirinzoni et al. [155] used Gaussian processes to estimate the reward model $R(r_i|s_i, a_i)$ and the dynamics model $P(s'_i|s_i, a_i)$. The importance weight is divided into reward weight $w_{r,i} = \frac{R(r_i|s_i, a_i)}{R_s(r_i|s_i, a_i)}$ and dynamic weight $w_{d,i} = \frac{P_r(s'_i|s_i, a_i)}{P_s(s'_i|s_i, a_i)}$. In the Q-value iteration algorithm, the former is used to estimate the reward value of the migration sample in the target domain, and the latter is used to adjust the influence of the migration sample when updating the Q function in the target domain. In the Q-value iteration algorithm, the former is used to estimate the reward value of the migration sample in the target domain, and the latter is used to adjust the influence of the migration sample when updating the Q function in the target domain.

Imitation learning is another transfer method based on trajectory samples. It uses expert demonstrations or trajectories to replace manually designed fixed reward functions to train agents. Expert demonstration data serves as a model for agents in real environments, helping train the agent's policy network to mimic the expert's behavior, thus significantly reducing training time. The common types of expert demonstration data include human demonstrations and trajectory samples from simulated environments. These methods can generally

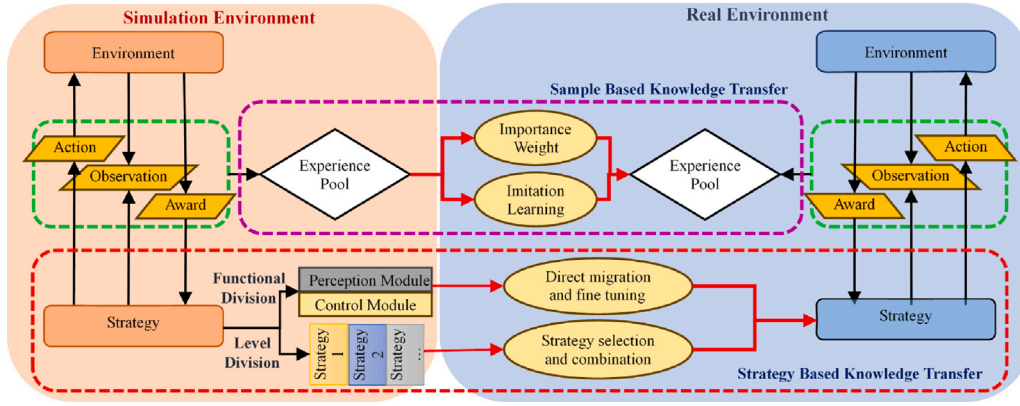


Fig. 5. Knowledge transfer method based on simulation environment.

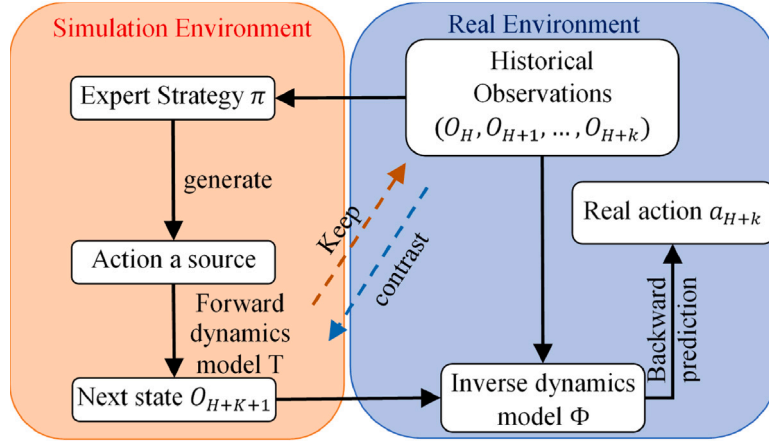


Fig. 6. Inverse dynamics model structure.

be categorized into two types: behavior cloning, where the agent learns to map observations to behavior demonstrations [156,157], and inverse reinforcement learning, where the agent attempts to estimate the reward function of the expert's strategy [158]. Imitation learning provides agents with environmental information about the reward function and can be used for robust reinforcement learning and sim-to-real transfer [159]. Christiano et al. [160] assumed that the probability of the state trajectory generated by the simulation environment appearing in the real environment is always not zero, but the same state trajectory may correspond to different action sequences in simulation and reality. Based on the above assumption, an inverse dynamics model ϕ in the real environment is trained. The model predicts the action of the real environment at the current moment based on the historical observations $(O_H, O_{H+1}, \dots, O_{H+k})$ from H time to $H+k$ time and the observations O_{H+k+1} at the next moment O_{H+k} , that is, $(O_H, O_{H+1}, O_{H+k}, O_{H+k+1}) \rightarrow a_{H+k}$. If the learned inverse dynamics model is accurate enough, then the next observation after taking the action in the simulation environment will be similar to, as shown in Fig. 6.

The strategy obtained by training in the simulation environment is regarded as an expert strategy, which is used to output the expected state trajectory sequence with high reward value. The real strategy solves the corresponding action sequence and executes it through the inverse dynamics model ϕ . In addition, simulation trajectory samples are also used for data collection. For example, Rahmatizadeh et al. [161] chose to conduct manual demonstration in a virtual environment in the scenario where the sample data of the real environment is difficult to be directly collected based on the safety and cost of robot training. The obtained simulation trajectory is used as an expert trajectory to train the control strategy of the real robot. Kaushik et al. [162]

proposed the SafeAPT algorithm, which makes full use of the simulation environment data as the prior knowledge of the real world, iteratively learns the reward probability and safety model from the observation of the real world, and then uses the reward model for Bayesian optimization. While maintaining the specified safety constraints, the safety model is used to select the optimal strategy from the set. Lai et al. [163] proposed a two-stage training framework TERT, which consists of an offline pre-training stage and an online correction stage. In the first stage of offline pre-training, the teacher strategy interacts with the simulation environment and collects training trajectories, and then trains the transformer, which gives relatively accurate predictions based on the teacher's observed action sequence; in the second stage of online correction, the transformer strategy interacts with the simulation environment, while the teacher gives corresponding reference actions. The teacher's observed action sequence is then used as a condition to train the transformer strategy to adapt to the teacher's actions. This model combines the transformer with privileged training to enable the quadruped robot to move stably on different terrains.

3.2.2. Simulation strategy decomposition and component migrations

The most common way to migrate simulation strategies is to directly migrate the entire strategy network trained in simulation to the real environment and fine-tune the strategy according to real samples. However, this method may cause the model to fail to converge or forget the knowledge learned in the simulation environment. Based on this idea, many works decompose simulation strategies by functional division or hierarchical division and migrate the components. As shown in Fig. 5, when divided by function, the strategy network is decomposed into a perception module and a control module. The former efficiently

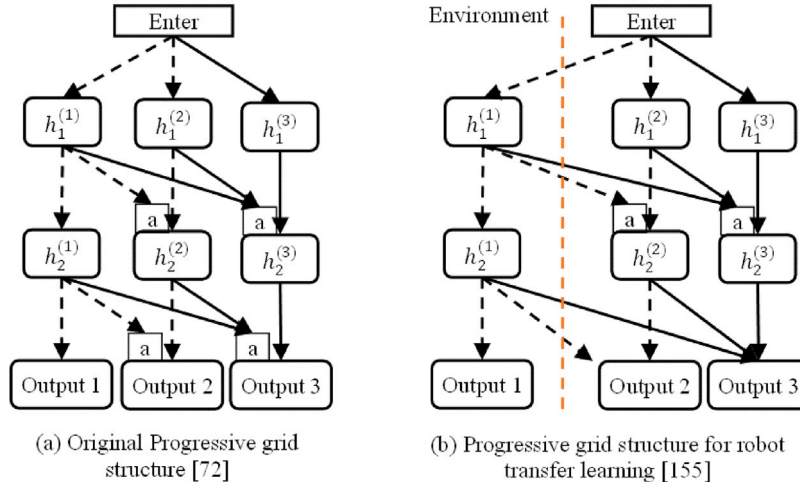


Fig. 7. Progressive network and its application in robot migration tasks.

encodes and extracts features from the observation input and is directly migrated and fine-tuned. The latter implements strategy control and outputs actions through the extracted features, which usually requires resetting and training. Rusu et al. [81] proposed a progressive network, which trains a new network on the target domain while retaining the network learned in the source domain. Through lateral (side) connections, the features from each layer of the old network are used as additional inputs to the corresponding layers of the new network. The i th layer feature $h_i^{(k)}$ of the new task k is jointly determined by the previous layer feature $h_{i-1}^{(k)}$ of the new task policy network and the previous layer feature $h_{i-1}^{(j)}$ of the policy network corresponding to the old task j , that is, $h_i^{(k)} = f(W_i^{(k)}h_{i-1}^{(k)} + \sum_{j < k} U_i^{(k:j)}h_{i-1}^{(j)})$, where W and U are the new network weight and the side connection weight, respectively. To improve transfer efficiency, an **adapter module** (denoted as “a” in Fig. 7) is introduced between the lateral connections, which performs nonlinear mapping and feature alignment. The adapter’s internal parameters $V_i^{(k:j)}$ and scaling factors $\alpha_{i-1}^{(<k)}$ project and normalize the transferred features to match the dimensionality of the new column, ensuring smooth integration. This design allows the new task to reuse previously learned representations without overwriting old knowledge. Rusu et al. [164] applied the above-mentioned progressive network to the robot migration task, freezing the parameters of the simulation strategy and training the real strategy. Each layer of state representation output by the simulation strategy was input into the next layer of the network corresponding to the real strategy, thereby avoiding the problem of knowledge forgetting after migration (as shown in Fig. 7). Experiments on the robot grasping task showed that this method can greatly improve sample efficiency and strategy performance. Kang et al. [165] proposed a method that combines a large amount of simulation data with a small amount of real-world experience for the visual task of the aircraft. The aircraft learns the physical properties and dynamics of the machine in the real world, and learns visual invariants and patterns from the simulation environment. During the migration, the neural network that approximates the Q function in the simulation environment is divided into a visual perception model and an action-based reward prediction model by layer. The visual perception model is used to process realistic and diverse visual scenes and is directly migrated to the real world. During the real-world strategy training, the parameters of these perception layers are kept unchanged to prevent strategy overfitting. The training tasks of the reward prediction model in the simulation environment are consistent with the real-world robot tasks. After being migrated to the real world, it is trained and optimized using real-world data.

When performing hierarchical division, the structure of the strategy is divided according to the semantic level, and several independent

strategies of different levels are obtained. After selection and combination, the control strategy in the real environment is obtained. The hierarchical structured strategy proposed by Wulfmeier et al. [166] includes a high-level strategy and several low-level strategies. When training on a set of tasks, the high-level strategy selects the low-level strategy under the condition that the task is known, and the low-level strategy makes decisions under the condition that the task is unknown. The former obtains high-level semantic information related to the task to control the switching of the low-level strategy, and the latter captures the common underlying semantic information of the task for feature extraction and policy control. When migrating, the low-level strategy is retained and the high-level strategy is retrained. In addition, simulation strategies can also be divided and migrated according to behavioral patterns. Yu et al. [167] combined domain randomization technology to learn a set of strategies with high expected returns and different behavioral patterns in a simulation environment, and searched for the best strategy combination in the strategy group according to the performance of the strategy on the target task for migration.

3.3. Strategy iteration improvement method based on virtual and real environment

Due to the complexity of the real environment and the limitations of static exploration strategies, offline exploration often cannot obtain all the information of the real environment. It is necessary to dynamically improve the strategy in order to better explore the environment. At the same time, the further improvement of the strategy puts forward higher requirements for environmental information. The strategy iterative improvement method based on virtual and real environments emphasizes the cyclic iteration of model optimization and knowledge transfer. Among them, the four steps of simulation model optimization based on the real environment, strategy improvement in the simulation environment, knowledge transfer based on the simulation environment, and exploration and strategy evaluation in the real environment are continuously iterated. In this process, the difference between the simulation environment and the real environment is constantly reduced, and the performance of the real strategy is also constantly improved. According to different optimization templates, this type of method can be divided into two categories: online exploration and alignment of the environment and domain adaptation method based on trajectory distribution. Fig. 8 shows the technical routes of these two types of optimization methods. The former performs iterative optimization with the goal of narrowing the difference between the environment, including environment alignment and migration of exploration strategies; the latter performs iterative optimization from the perspective of bringing

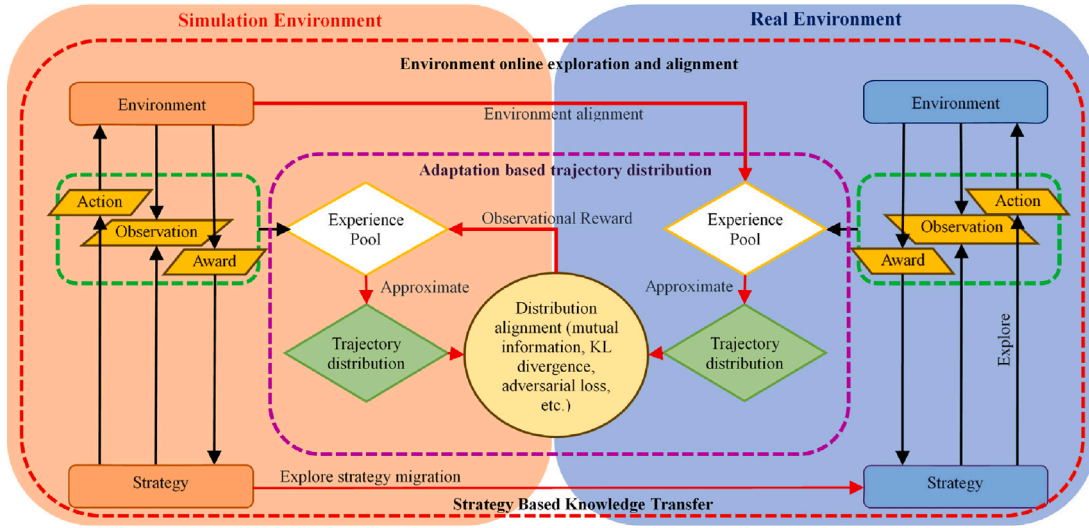


Fig. 8. strategy iteration improvement method based on virtual and real environment.

the simulation and real trajectory distributions closer, including trajectory distribution approximation, distribution alignment, and reward compensation mechanisms.

3.3.1. Online exploration and alignment methods

Narrowing the gap between the simulation environment and the real environment is an important way to ensure the effectiveness of the migration strategy, but it is difficult to achieve this goal through offline exploration of fixed strategies. Therefore, it is necessary to continuously explore the real environment during the strategy learning process, and on this basis, align the training environment online. That is, the strategy trained in the simulation environment is used to explore the real environment, and the collected samples are used to correct the dynamics model and reward model in the simulation environment. Repeated iterations are performed until the performance of the strategy is no longer improved.

In the GSL (Grounded Simulation Learning) framework proposed by Farchy et al. [168] in 2013, the strategy is trained in a simulation environment and sampled and evaluated in the real world to obtain information about the real dynamic model. For robot walking tasks, GSL uses real samples to establish a mapping with joint state and joint command as input and next joint state as output, and uses this mapping to adjust the torque parameters in the simulation environment so that the simulated robot reaches the same target joint state as the real robot. The corrected dynamic model is used to relearn the control strategy, and the new strategy is continued to be used for sample collection. Unlike the alignment mechanism of adjusting environmental parameters in GSL [168], the GAT (Grounded Action Transformation) proposed by Hanna et al. [169] does not directly correct the dynamic model of the simulation environment, an action transformation layer is connected after the control strategy, and the dynamics of the real environment are learned through supervision methods. The learning model (s_t, a_t) establishes a mapping from s_{t+1} to f , and learn the inverse dynamics process of the simulation environment (s_t, s_{t+1}) to establish a mapping from a_t to f_{sim}^{-1} . The action output by the action conversion layer is $\hat{a}_t = f_{sim}^{-1}(s_t, f(s_t, a_t))$, so that the state after executing action in the simulation environment is consistent with the state after executing action in the real world, thereby narrowing the difference in the dynamic model, as shown in Fig. 9. A series of works based on GAT [170–172] have expanded the applicable scenarios to random environments, established dynamic models through deep learning, and used reinforcement learning for strategy learning, which improved the generalization and migration performance of the method.

In addition to directly learning the dynamics model, another common way to align environments is to model the dynamics difference between the real environment and the simulated environment [112, 173–175]. In order to reduce the huge demand for real samples and the strict requirements on hardware in transfer learning, Ha et al. [173] and Rastogi et al. [112] both modeled and learned the dynamics difference between the simulated environment and the real environment, and established a prediction transfer state difference $d(s_t, a_t) = s_{t+1}^{sim} - s_{t+1}^{real}$ in the iteration of strategy optimization and environment exploration, the new strategy is trained on the state transition samples modified by the model and used to explore the real environment. Liu et al. [176] proposed a digital twin system, which transmits the data collected in the real world as twin data to the simulation environment, and at the same time transmits the data extracted and calculated in the simulation environment to the physical entity in the real environment, so that the movement of the virtual robot can also affect the physical entity, thereby ensuring the synchronization of the movement process of the physical entity and the virtual entity. Abeyruwan et al. [177] designed the sim-to-real iteration model i-Sim2Real, which applies the initial robot strategy trained in the simulation environment to the real world for human–computer interaction, collects interaction data and fine-tunes the strategy in the real world, and then uses the interaction data to update the human behavior model used in the simulation environment. Finally, the fine-tuned strategy is redeployed in the simulation environment to interact with the updated human behavior model, and the sim-to-real iteration trains a robotic arm that cooperates with human players to play table tennis.

Different from focusing on dynamic models, another type of work focuses on the exploration and alignment of reward models [178,179]. Laroché et al. [178] assumed that the dynamic models of the source domain and the target domain are the same, and directly learned the target domain strategy through source domain samples and target domain reward functions. When the target domain reward function is unknown, they explored the unknown states in the target domain according to the principle of uncertainty optimism. Barreto et al. [179] proposed an exploration and learning framework for scenarios where the reward function changes but the environment dynamics remain unchanged. They experimented with a series of navigation tasks and simulated robot arm control and achieved good results. In order to solve the problem of difficulty in accurately rendering real-world scenes in a simulated environment, Chen et al. [180] proposed the Real2Sim method, which narrowed the gap between real and simulated environment images by converting virtual images generated by the simulated environment and real-world images.

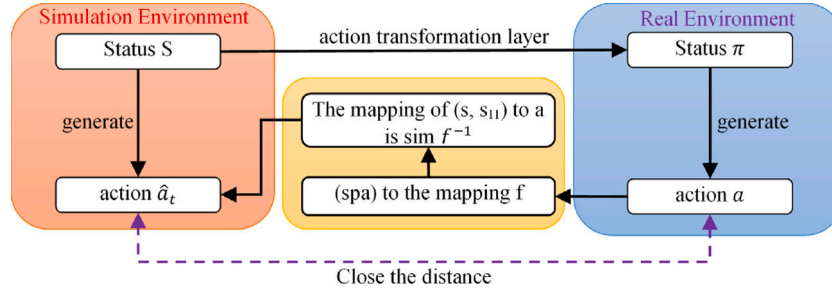


Fig. 9. Modifiable simulator GAT structure diagram.

3.3.2. Domain adaptation method based on trajectory distribution

The domain adaptation method based on trajectory distribution uses samples generated by interaction with the environment to estimate trajectory distribution, and uses indicators such as mutual information, KL divergence, and adversarial loss [110,181–183] to reduce the difference between the source domain trajectory distribution and the target domain trajectory distribution as the optimization goal. This achieves the optimization and migration of the strategy. In this process, the update and optimization of the strategy will lead to changes in the trajectory distribution, and it is necessary to re-estimate the trajectory distribution and calculate the distribution difference. Therefore, the process of trajectory estimation and strategy optimization will be continuously iterated between the source domain and the target domain, thereby improving the performance of the strategy and the effectiveness of migration.

A simple and intuitive way to reduce the difference in trajectory distribution is to give additional penalties in the reward function to source domain trajectory samples that are significantly different from the target domain trajectory during training, thereby suppressing the simulation strategy from producing trajectories that have a low probability of appearing in the real world. Eysenbach et al. [182] derived that minimizing the KL divergence between the source domain trajectory and the target domain trajectory is equivalent to maximizing the entropy regularized reward plus reward compensation, that is, $\min_{\pi} D_{KL}(P_{source}(\tau) \parallel P_{target}(\tau))$ is equivalent to:

$$\min_{\pi} E_{p-source} \left[\sum_t r(s_t, a_t) + H_{\pi}[a_t | s_t] + \Delta r(s_{t+1}, s_t, a_t) \right] + c \quad (2)$$

where, $\Delta r(s_{t+1}, s_t, a_t) \triangleq \log P_{target}(s_{t+1} | s_t, a_t) - \log P_{source}(s_{t+1} | s_t, a_t)$, and the collective entropy regularized reward i.e. $\sum_t r(s_t, a_t) + H_{\pi}[a_t | s_t] + \Delta r$, which is the optimization goal of the traditional reinforcement learning maximum entropy algorithm. In addition, the algorithm also learns two auxiliary classifiers to distinguish between source domain trajectories and target domain trajectories, so as to compensate for the reward term Δr . Wulfmeier et al. [183] proposed a mutual alignment transfer learning model, which estimates the difference in trajectory distribution by learning an auxiliary classifier that distinguishes the simulated trajectory state sequence from the real trajectory state sequence. This difference is used as an additional reward function, acting on both the simulated robot and the real robot training, and guiding the exploration of the real world. As shown in Fig. 10, the strategies are trained in the simulated environment and the real environment respectively. π_{sim} and π_{real} , generate corresponding trajectories, and use auxiliary classifiers D_W which discriminate the trajectory to obtain auxiliary reward value r' , in $r'_{sim} = -\log(D_w(\xi_k))$ and $r'_{real} = -\log(D_w(\xi_k))$. Auxiliary rewards together with environmental feedback constitute the complete reward used to adjust the strategy $r = r_t + \lambda r'$. Then use the TRPO algorithm according to $E_{\pi_0} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r_t]$ to adjust the strategies of the simulation environment and the real world respectively. Wu et al. [184] also applied the reward-based adaptive method to the simulation recommendation system to reduce the dynamic difference, and combined adversarial learning with singular value decomposition

(SVD) for observation representation learning. Chung et al. [185] proposed a segmented encoding sim-to-real transfer algorithm SESR for autonomous driving tasks. Through a multi-class segmentation network, the segmented RGB images in the simulation environment and the real environment are transferred to the same domain, minimizing the KL divergence of the vectors generated by the encoder and the parallel network VAE, while retaining the necessary information of primary objects such as pedestrians, roads and cars.

Another common transfer method is the domain adaptation method based on observation distribution [38–40]. It can be proved that under the assumption that the state transition probability of the source domain and the target domain is the same, observation-based domain adaptation is a special case of trajectory distribution-based domain adaptation [182]. Gamrian et al. [186] proposed to establish a visual mapping from the source domain to the target domain based on the adversarial idea, with the goal of making it difficult for the classifier to distinguish the source of the image after visual mapping. Bousmalis et al. [187] combined synthetic images with domain randomization technology to train an adversarial generative network for visual observation adaptive alignment for the robot grasping task based on pixel images. Experiments have shown that their method is effective on physical robots and reduces the demand for real samples. Starting from the difference in trajectory distribution, Chebotar et al. [110] proposed a metric distance based on the observed state sequence in the simulated trajectory and the real trajectory, and used the domain randomization method to adjust the random distribution of the simulation environment parameters. Truong et al. [188] proposed a bidirectional domain adaptation method. Domain adaptation methods are used to bridge the gap between the states in simulation and the observed inputs of the real environment.

The above summarizes the existing sim-to-real transfer methods from three directions: simulation model optimization based on real environment, knowledge transfer based on simulation environment, and strategy iteration improvement based on sim-to-real environment, and establishes a unified general system framework for the research field of sim-to-real transfer reinforcement learning. Table 2 summarizes the content and results of representative work in the above three directions from the perspective of experiments, lists the simulation environment, real environment, experimental tasks and results of the experiments, and summarizes the existing challenges and possible future research directions in each direction.

3.4. Summary

The first category of model optimization methods based on real environments (as shown in Table 2) primarily relies on techniques such as system identification and domain randomization in simulation platforms like PyBullet and MuJoCo. These methods aim to make simulated environments closely resemble real-world conditions. By incorporating curriculum learning and multi-fidelity simulations, they improve both sample efficiency and training speed. Although these methods have been successfully validated on real-world systems such as robotic arms and robot vehicles, they still face challenges, including over-reliance

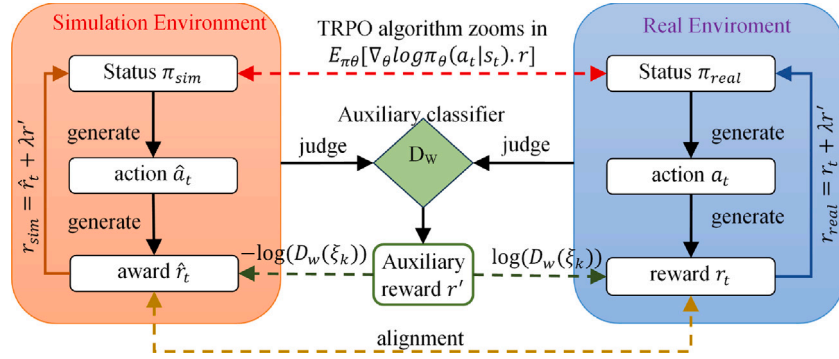


Fig. 10. Mutual alignment transfer learning structure diagram.

on prior knowledge, high demand for real-world samples, and inaccurate environmental modeling. To address these issues, future work could focus on developing more generalizable environment modeling techniques and high-fidelity simulators.

The second category, knowledge transfer methods based on simulation environments (also summarized in Table 2), employs techniques such as importance sampling, imitation learning, policy decomposition, and component transfer. These methods aim to transfer both environmental information from simulated samples and control strategies from simulation-trained models to real-world systems. They have shown promising results on platforms such as industrial robotic arms and aircraft. However, they still face key challenges, including distributional shifts between simulated and real data and a strong dependence on human expertise for policy decomposition. Hence, more effective approaches are needed for sample identification and automated policy or component segmentation.

The third category, iterative policy improvement methods based on both virtual and real environments, integrates the advantages of the first two categories to achieve more balanced and robust performance.

By performing online alignment between the simulation and real environments and employing domain adaptation based on trajectory distributions, this approach continuously minimizes the gap between simulated and real conditions through iterative optimization. As a result, the performance of real-world policies is progressively improved. These methods are well-suited for learning and training control behaviors in systems such as robotic manipulators and bipedal robots. However, this category of methods still faces key challenges, including ensuring safe exploration and achieving dynamic adaptation of environmental models. Potential solutions include incorporating constrained reinforcement learning into policy optimization and developing dynamically adjustable simulation models or simulators to better capture changing real-world dynamics.

4. Recent advances

Presently, sim-to-real reinforcement learning (RL) has shifted from demonstrating isolated transfer successes toward systematic, benchmark-oriented, and physics-aware pipelines. One landmark work is A Survey of Sim-to-Real Methods in RL: Progress, Prospects and Challenges with Foundation Models [190], which provides a comprehensive taxonomy of sim-to-real techniques incorporating foundation models and formal evaluation across key elements of the Markov decision process. In particular, digital-twin or real-to-sim-to-real frameworks now allow simulators to be continuously calibrated with real data, closing the loop between simulation and deployment [191]. At the same time, automated simulator configuration using large language models and optimization algorithms reduces the dependence on manual parameter tuning, thereby enhancing reproducibility and scalability. Another notable advance is the soft robotics domain residual-physics modeling approach by Gao et al. [192], where learned residual terms correct

simulation errors in soft, high-degree-of-freedom systems, substantially improving sim-to-real fidelity. In addition, Real-to-Sim transfer via high-fidelity simulation to control a wheeled humanoid robot with unknown dynamics by Baek et al. [193] uses high-fidelity modeling and system identification to calibrate simulators for hardware with unknown dynamics. Salimpour et al. [194] demonstrate sim-to-real transfer for mobile robot navigation policies trained in NVIDIA Isaac Sim transferred to ROS-based real robots, showing strong promise for reproducible and robust deployment. Finally, the community has introduced benchmarking and evaluation frameworks (OpenReview, 2025) that provide standardized tasks and metrics for policy evaluation on real and simulation platforms [194].

These works collectively highlight emerging trends: (i) integrating accurate actuator, environment modeling, and residual error correction; (ii) formalization of evaluation via benchmarks and reproducible pipelines; (iii) reduction of engineering manual tuning through automated adaptation; and (iv) expanding to complex, high-dimensional systems including soft robotics and mobile navigation. The field is moving toward more reliable, scalable, and generalizable sim-to-real RL solutions. The following subsections discuss each of these advances in detail, highlighting representative contributions, technical innovations, and their implications for robotic learning pipelines.

4.1. Real-to-sim-to-real pipelines

The enduring challenge in reinforcement learning (RL) for robotics is the reality gap: policies optimized in simulation often fail when deployed in the physical world. Classical sim-to-real strategies, such as domain randomization, aimed to mitigate this gap but relied on manually designed parameter ranges and static simulators, leading to brittle transfer and limited reproducibility. Over the past two years, a new class of approaches, termed real-to-sim-to-real (R2S2R) pipelines, has emerged as a promising alternative. These pipelines conceptualize the simulator as a dynamic digital twin, constructed and continuously updated from real data, rather than as a fixed training environment.

The overall workflow of a real-to-sim-to-real (R2S2R) pipeline is illustrated in Fig. 11, which depicts the closed-loop cycle beginning with real robot data, the construction of a digital twin, calibration and residual modeling, simulation-based training, and eventual policy deployment. Innovations such as inverse distillation [191], dynamic digital twins [195], and residual-physics modeling [192] are highlighted as key milestones in the evolution of this paradigm.

Unlike earlier sim-to-real methods, R2S2R pipelines place real data at the start and center of the learning loop. Small batches of real rollouts or demonstrations are first used to initialize or calibrate a simulator. Policies are then trained at scale in this simulator before being redeployed on hardware, where performance feedback further refines the virtual model. This iterative cycle dramatically improves both sample efficiency and predictive validity, i.e., the correlation between simulated and real performance. Recent surveys identify R2S2R as a

Table 2
Summary of representative work.

Category	Realistic to simulated environment optimization		
Literature	Allevato et al. [122]	Peng et al. [134]	Cutler et al. [42]
Experimental tasks	1. Establish a simulation environment corresponding to the real robot arm and the bouncing ball 2. Sim-real migration of the robot arm bouncing and hitting the ball (bouncing the ball from the slope back to the basket)	Robotic arm grabbing	Controlled drift of a robotic car
Strategy optimization	–	RDPG	RBF network
Simulation environment	PyBullet	MuJoCo	–
Real robot	Kinova-Jaco 7DOF Robot manipulator	7DOF Fetch Robotics Manipulator	RC car
Experimental results overview	The system identified that the error between the simulated environment and the real environment parameters was small, and the success rate of the bouncing ball task reached 87%.	The robot grasping success rate in simulation and real environment reached 91% and 89% respectively	The simulation strategy controls the real car to obtain the drift trajectory to achieve the expected effect
Difficult challenges	1. Environmental modeling relies on strong prior knowledge or expert experience 2. Model fitting requires a large number of real samples and has large estimation errors		
Future Directions	1. Design an autonomous environmental modeling method to automatically identify key factors from multiple environmental factors 2. Study simulators and simulation environments with high computational efficiency, accurate modeling, and high sample efficiency		
Category	Knowledge transfer from simulation to reality		
Literature	Rusu et al. [164]	Kang et al. [165]	Christiano et al. [160]
Experimental tasks	Robotic grasping based on image input	Aircraft obstacle avoidance	Robotic arm grabbing
Strategy optimization	A3C	Deep Q-learning	TRPO
Simulation environment	MuJoCo	Gibson simulator	MuJoCo
Real robot	Kinova-Jaco Manipulator	Crazyflie 2.0 nano quadcopter	Physical Fetch robot
Experimental results overview	The method is effective, and the average reward is higher than direct migration and fine-tuning	Taking obstacle avoidance time as an indicator, the proposed model outperforms direct transfer fine-tuning, supervised transfer, and unsupervised learning. The obstacle avoidance time is as long as 85.8 s, and it can effectively avoid obstacles in complex scenes.	Its strategy is robust under the interference of environmental parameters, and the error between the real trajectory and the simulated trajectory is within 3.7%.
Difficult challenges	1. The distribution shift between simulated samples and real samples leads to poor strategy performance. 2. The decomposition and migration of simulation strategies require strong prior assumptions and heuristic design.		
Future Directions	1. Introduce importance sampling, data enhancement, domain adaptation and other methods to reduce the distribution offset between samples in different environments. 2. Further study methods such as adversarial learning and domain adaptation, and propose a more flexible and general strategy decomposition and migration framework.		
Category	Iterative improvement between simulation and reality		
Literature	Farchy et al. [168]	Hanna et al. [169]	Chebatar et al. [110]
Experimental tasks	Bipedal robot walking	Bipedal robot walking	The robotic arm controls the pendulum to enter the hole and open the drawer based on the depth map
Strategy optimization	CMA-ES	CMA-ES	PPO based on GPU parallel simulator [189]
Simulation environment	SimSpark	SimSpark, Gazebo	NVIDIA Flex
Real Robot	Bipedal robot NAO	Bipedal robot NAO	ABB Yumi robots and Franka Panda 7-DoF
Experimental results overview	The robot's walking speed increased by 26.7% compared to the baseline strategy	The walking speed increased by 34.6% (first iteration) and 43.3% (second iteration) in the migration from SimSpark to Gazebo, and by 37.8% in the migration from Gazebo to the real robot.	With the support of a small number of real samples, the final hole-entry rate reached 90%, and the strategy learned the standard action of opening the drawer.
Difficult challenges	1. In multiple rounds of iterations, the improvement of strategy performance depends on the trade-off between efficient exploration and security. 2. The simulation environment needs to dynamically adapt to the newly collected real samples in each round of iteration and make corresponding adjustments.		
Future Directions	1. Introduce secure reinforcement learning into policy learning to achieve efficient exploration of the real environment while ensuring the security of the policy 2. Design a dynamically adjustable simulation environment model or simulator to achieve accurate fitting of the real environment		

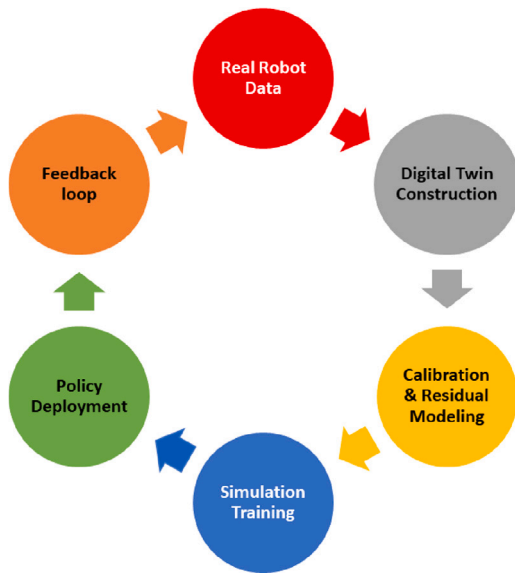


Fig. 11. Real-to-Sim-to-Real (R2S2R) Pipeline)

cornerstone of modern sim-to-real practice, noting its ability to combine the safety of simulation with the grounded fidelity of real-world data [190].

One influential example is RialTo [191], which demonstrates R2S2R for household manipulation. RialTo reconstructs a lightweight digital twin from a handful of real demonstrations, trains policies in this calibrated simulator, and then redeploys them to the robot. A novel element is inverse distillation, in which real-trained policies are transferred back to the simulator to bootstrap further large-scale training. This bidirectional exchange ensures that simulators remain relevant and that simulation-trained policies remain grounded in reality. RialTo reported significantly higher transfer success rates compared with classical domain randomization, while requiring fewer than 100 real trials.

By contrast, the Real-is-Sim framework [195] emphasizes continuous synchronization. Here, the simulator is not just initialized from real data but is dynamically aligned with the physical robot throughout operation. This design allows large-scale offline evaluation of candidate policies while maintaining high correlation with real execution. The approach suggests a path toward lifelong learning, where digital twins serve as always-on evaluators and trainers rather than static pre-deployment tools.

Across R2S2R pipelines, several recurring mechanisms underpin success:

1. **Rapid scene reconstruction.** Advances in RGB-D sensing and photogrammetry allow task-specific digital twins to be created from just a few images or scans. While physics parameters may remain approximate, accurate geometry already constrains learning effectively, as shown in RialTo.
2. **Simulator calibration.** Where physics engines fall short, residual networks capture systematic discrepancies. Gao et al. (2024) demonstrated this in soft robotics, where compliance and deformation are hard to model analytically. Embedding residual terms within simulators allows efficient correction without abandoning physics priors.
3. **Residual modeling.** Advances in RGB-D sensing and photogrammetry allow task-specific digital twins to be created from just a few images or scans. While physics parameters may remain approximate, accurate geometry already constrains learning effectively, as shown in RialTo.

4. **Policy bridging.** Inverse distillation strategies recycle real policies back into simulation. By fine-tuning these policies under randomized or corrected dynamics, R2S2R pipelines achieve policies that are both grounded in reality and stress is tested in diverse simulated conditions.

The comparison between RialTo and Real-is-Sim illustrates two complementary philosophies. RialTo focuses on data efficiency, showing that limited demonstrations suffice to bootstrap robust digital twins. Real-is-Sim prioritizes scalability and adaptability, maintaining simulators as dynamic mirrors of robots in operation. Residual-physics methods further highlight that accurate actuator and drivetrain modeling is as critical as environment geometry, an insight corroborated by benchmark studies (OpenReview, 2025) [196]. Together, these works suggest that R2S2R pipelines are not monolithic but span a design spectrum: from lightweight digital twins for rapid prototyping to continuously synchronized twins for lifelong learning. To further highlight the diversity of recent R2S2R methods, Table 3 summarizes recent representative systems contrasting their core ideas, target domains, pipeline characteristics, key innovations, strengths, and limitations.

Despite impressive progress, several challenges remain unresolved. First, constructing digital twins for deformable or contact-rich tasks (cloth folding, fluid manipulation) remains computationally expensive and lacks standardized methodologies. Second, actuator modeling continues to lag: capturing high-bandwidth effects such as backlash, compliance, and wear remains difficult. Third, reproducibility is limited by the lack of benchmarks, while in an OpenReview [196] has proposed initial evaluation frameworks, widespread adoption is pending. Finally, integrating R2S2R loops into multi-robot or distributed settings introduces synchronization and scalability issues that current systems only partially address.

4.2. Automated simulator configuration

Automated simulator configuration addresses a core bottleneck in sim-to-real transfer: deciding what and how to randomize in simulation so that policies trained there perform robustly in the real world. Historically, practitioners relied on hand-tuned domain randomization ranges or ad hoc system identification leading to an engineering heavy process that undermines reproducibility and often yields overly conservative policies. To overcome these limitations, recent research has focused on automating the design of simulation distributions and calibration procedures through optimization, probabilistic inference, adversarial training, and generative modeling. Broadly, these methods can be categorized into five major families: (i) Bayesian and optimization-based tuning of randomization parameters; (ii) discriminator-guided auto-tuning; (iii) learned sampling distributions (e.g., neural samplers, flows); (iv) data-driven pipelines that generate simulation tasks from real-world observations; and (v) hybrid strategies that combine system identification, residual models, and adaptive tuning. The following subsections examine each family in detail, highlighting representative methods, empirical performance, and key limitations.

4.2.1. Bayesian and optimization-based approaches

One of the earliest and most principled directions in automated simulator configuration has been to frame it as an optimization problem. The central idea is that simulator parameters (e.g., mass, inertia, friction, sensor noise) should be chosen not by manual heuristics but by search algorithms that maximize transferability. This often takes the form of black-box optimization, Bayesian optimization, or stochastic search.

Bayesian Domain Randomization (BayRn)

Park et al. [197] introduced Bayesian Domain Randomization (BayRn), which models the problem as learning a distribution over simulator parameters that maximizes expected real-world performance. By treating the transfer success as a black-box function, BayRn uses

Table 3

Comparison of recent real-to-sim-to-real (R2S2R) pipelines.

System	Year	Core idea	Domain/Robot type	Pipeline characteristics	Key innovation	Reported benefits	Limitations
RialTo	2024	Build lightweight digital twin from real demos; iterative policy training; inverse distillation (real $\rightarrow$ sim $\rightarrow$ real).	Household manipulation (rigid-body tasks)	Demo-based digital twin construction; sim refinement; inverse distillation	Bidirectional policy flow (real $\leftrightarrow$ sim); open-source toolkit	Robust transfer with <100 trials; reproducible workflows	Requires structured environments; limited to rigid-body tasks.
Real-is-Sim	2025	Continuously synchronized dynamic digital twin; simulation updated in real time.	Mobile robots, multi-robot setups	Dynamic, continuously synchronized digital twin	Real-time alignment of sim and real states	Lifelong learning; high predictive validity	Computationally expensive; synchronization overhead.
Residual-physics sim	2024	Learn residual corrections for physics engines using small real datasets.	Soft robotics, compliant actuators	Hybrid physics + residual corrections	Learned correction of dynamics errors	Improved fidelity for deformable tasks	Requires careful residual design; limited generalization across morphologies.
Auto-Tuned Sim-to-Real	2021 (precursor)	Automatic simulator parameter tuning via optimization.	Locomotion, manipulation	Simulator calibration via optimization	Automated dynamics tuning	Reduced human bias; scalable tuning	Early-stage; limited scaling; not fully integrated with R2S2R.
Benchmark initiatives	2025	Standardized evaluation of sim-to-real RL across tasks.	Locomotion & manipulation benchmarks	Paired sim-real task suites	Standardized evaluation metrics	Reproducibility; cross-method comparison	Still emerging; adoption limited.

Gaussian processes and Bayesian optimization to adapt domain randomization ranges with minimal real trials. On benchmark tasks (cart-pole swing-up, robotic pushing), BayRn achieved robust policies with as few as 20–30 real evaluations—an order of magnitude fewer than naive domain randomization. Importantly, BayRn showed that adaptive tuning can avoid the under-randomization vs. over-randomization trade-off that plagues manual domain randomization.

Auto-Tuned Sim-to-Real (ATSR)

Du et al. [198] introduced Auto-Tuned Sim-to-Real (ATSR), which used a search parameter model to iteratively shift simulated parameter distributions closer to real dynamics. Unlike BayRn, ATSR leverages raw sensory observations to adjust simulator settings without requiring task-specific performance labels. The system was evaluated on robotic locomotion tasks (quadruped locomotion in PyBullet and deployment on hardware), showing that automatic tuning reduced the number of failed deployments by 40% compared to static domain randomization.

SimOpt

Rajeswaran et al. [199] proposed SimOpt, which casts simulator calibration as an iterative optimization loop: policies are trained in simulation, rolled out in the real environment, and discrepancies between real and simulated trajectories are used to update simulator parameters. Recent extensions demonstrated scalable versions of SimOpt that parallelize the optimization across hundreds of simulations, enabling configuration of high-dimensional simulators with 20–50 parameters [200]. On manipulation benchmarks, SimOpt reduced sim-to-real error by 35% compared to uncalibrated baselines.

Across benchmarks, Bayesian and optimization-based approaches demonstrate clear empirical advantages: they achieve robust policies with relatively few real-world trials (BayRn succeeded with 30 evaluations, while SimOpt typically required 50–100 trajectories), and transfer rates improved by 20%–40% compared to hand-tuned baselines in dexterous manipulation and locomotion. However, these gains come at the cost of significant computational overhead, especially when simulators must be retrained from scratch at each optimization step or when more than 50 parameters are tuned simultaneously. Furthermore, performance remains sensitive to the choice of objective function, for example, optimizing trajectory matching versus task returns can yield divergent outcomes. While Bayesian optimization is highly sample efficient, it scales poorly in very high-dimensional parameter spaces, necessitating surrogate models or parallel simulation backends that

add further complexity. In summary, Bayesian and optimization-based approaches provide a principled and reproducible foundation for automated simulator configuration, offering measurable improvements in efficiency and robustness while reducing dependence on human heuristics. Yet, their limitations in scalability and computational cost underscore the need for alternative or complementary strategies such as discriminator-based tuning, learned distributional samplers, and hybrid optimization methods, which are discussed in subsequent subsections.

4.2.2. Discriminator/search-guided auto-tuning

A second line of research in automated simulator configuration leverages discriminative models to reduce the gap between real and simulated data. Rather than directly optimizing task performance, these methods ask a simpler question: Can a classifier distinguish whether a given trajectory (or observation) originates from simulation or from the real robot? If the discriminator can separate the two, the simulator parameters are adjusted until the domains become indistinguishable. This adversarial framework, conceptually similar to domain adaptation in computer vision, has gained traction in robotics because of its flexibility and dense feedback.

Adversarial Domain Randomization (ADR)

OpenAI [201] introduced Adversarial Domain Randomization (ADR), where a discriminator distinguishes between real and simulated rollouts. A generator then adjusts the domain randomization distribution to fool the discriminator, effectively aligning simulated trajectories with real ones. While ADR was initially applied to dexterous manipulation tasks, later works adapted it to locomotion and grasping. Empirical results showed ADR trained policies transferred with 25%–30% higher success rates compared to hand-tuned randomization.

Discriminator-Guided System Identification

Building on ADR, Muratore et al. [202] proposed discriminator-based calibration where the discriminator operates not on final task performance but on state–action trajectories. This allows finer-grained alignment of dynamics properties such as friction and damping. On robotic pushing and hopper locomotion tasks, discriminator-guided calibration achieved faster convergence than Bayesian optimization methods, needing 50–70 rollouts compared to >100 for baseline optimizers.

Visual Domain Alignment

Discriminator-based strategies have been particularly impactful in visual sim-to-real transfer. When transferring policies from photorealistic simulators (e.g., MuJoCo + rendering engines) to robots with on-board cameras, pixel-level differences can degrade performance. GAN-based discriminators, trained to distinguish synthetic from real images, are used to tune lighting, textures, and camera parameters in the simulator. For instance, Tobin et al. [203] applied discriminator-driven visual randomization to robotic grasping, reducing sim-to-real failure rates by nearly 40%.

Empirically, discriminator-based methods typically require 50–100 real samples, comparable to Bayesian optimization, but often achieve faster convergence due to dense, per-step feedback provided by the discriminator. In manipulation and locomotion tasks, adversarial tuning has yielded 20%–35% improvements in transfer success relative to static randomization, and its flexibility has made it particularly effective in visual domains where manual parameter tuning is infeasible. However, these methods face important limitations: discriminators can overfit to spurious differences, leading to unstable or oscillatory training; adversarial optimization may diverge if the discriminator is too strong; and dynamics calibration, while promising, remains less reliable for high-dimensional actuator models compared to visual alignment. To mitigate these issues, regularization, curriculum design, and hybrid pipelines are often necessary. In summary, discriminator-based auto-tuning offers a data-driven and adaptable approach to bridging the reality gap, excelling in visual alignment and providing dense alignment signals without requiring explicit labels. For dynamics calibration, however, it is best deployed as a complementary module alongside optimization methods or residual-physics models.

4.2.3. Learned sampling distributions and normalizing flows

While Bayesian optimization and discriminator-based approaches iteratively adjust simulator parameters, they often rely on simple parameterizations of distributions (e.g., uniform or Gaussian). Such assumptions limit their ability to capture the complex and multimodal nature of physical parameter spaces encountered in real robots. Recent advances therefore focus on learned sampling distributions, where flexible generative models are trained to directly propose domain randomization parameters that are both diverse and task-relevant. By leveraging architectures such as normalizing flows, variational autoencoders, or adaptive neural samplers, these approaches aim to more faithfully represent the structure of real-world variability.

Normalizing Flow Samplers

Normalizing flows provide a class of models that transform simple base distributions into complex, structured ones through invertible mappings [204]. Their expressivity has recently been applied to sim-to-real transfer. Curtis et al. [205] introduced GoFlow, which employs flow-based models to learn parameter distributions that maximize real-world robustness in assembly and locomotion tasks. Compared with Gaussian randomization, GoFlow improved real-world task success by approximately 18% on Franka manipulation tasks and halved failure variance across diverse environments, underscoring the benefits of flexible sampling.

Adaptive Neural Samplers

Another line of work adapts sampling distributions during training. Instead of fixing a parameter distribution at the start, adaptive samplers use feedback from policy performance to refine distributions. Techniques based on reinforcement learning (RL², meta-RL) and variational or Bayesian inference update the sampling model online. For instance, Huang et al. [206] proposed BayRnTune: Adaptive Bayesian Domain Randomization via Strategic Fine-tuning, which adapts randomization parameter ranges using real-world feedback and fine-tuning to accelerate sim-to-real transfer in legged-robot environments. This adaptability highlights the efficiency gains of online learning in high-variance domains.

Multi-Modal and Structured Randomization

Learned samplers can capture multi-modal distributions, reflecting the reality that physical parameters are not always unimodal (e.g., contact friction varies across surfaces, actuator latency may cluster into discrete modes). Recent work in domain randomization, for example Neural Posterior Domain Randomization [138], allows simulator parameters to be correlated and non-parametric (i.e. not restricted to uniform or Gaussian families) when inferring parameter distributions from real-world data. This approach avoids unrealistic parameter combinations, enabling policies that transfer better with fewer real-world trials.

Empirical evidence shows that learned samplers consistently outperform Gaussian baselines. Flow-based models like GoFlow achieved an 18% boost in task success and a 25% reduction in sim-to-real error, while adaptive samplers reduced real-world trial requirements by 30%–40% compared with static approaches. Moreover, multimodal samplers produced policies that generalized more robustly across varied environments than unimodal baselines. Nonetheless, these gains come with trade-offs: training complex generative models incurs computational overhead and requires large batches of simulated samples; outcomes remain sensitive to transfer metrics, as poorly designed objectives can lead samplers to model irrelevant distributions; and the models are often less interpretable than Bayesian methods, complicating debugging and safety validation. In summary, learned sampling distributions represent a significant evolution in automated simulator configuration. By replacing hand-crafted ranges with neural samplers capable of modeling multimodality and structure, they improve robustness and efficiency in high-dimensional settings where optimization-based methods struggle. Looking forward, integrating flow-based samplers with digital twins and residual-physics models promises more scalable and generalizable real-to-sim-to-real pipelines.

4.2.4. Data-driven generation of simulation tasks and synthetic data engines

Another promising frontier in automated simulator configuration is the data-driven generation of simulation tasks and assets directly from real-world observations. Traditional domain randomization assumes that a simulator already exists with predefined environments, physics, and assets. In reality, however, building such simulators requires considerable manual effort to design scenes, objects, textures, and dynamics models. Recent work has begun to automate this process by leveraging real-world data images, videos, and demonstrations to synthesize simulation-ready assets and environments. This approach reduces engineering overhead, improves alignment with real-world deployment conditions, and accelerates the creation of task-specific simulators.

Generating Robotic Simulation Tasks (GRS)

Zook et al. [207] introduced GRS, a framework for automatically generating robotic simulation environments from real-world images and sensor logs. Using scene parsing and object recognition, GRS constructs semantic maps that can be converted into Unity or MuJoCo assets. For manipulation benchmarks, GRS generated more than 200 unique environments from household images, each aligned with real object affordances. Policies trained in these auto-generated environments achieved 15% higher transfer success compared with hand-designed simulators, demonstrating the benefit of greater realism and diversity.

DexScale: Teleoperation-to-Simulation Data Engines

The DexScale project [208] demonstrated how large-scale teleoperation demonstrations can be transformed into synthetic datasets for dexterous manipulation. By capturing human hand–object interactions through video and depth data, DexScale generates parameterized simulation tasks that model contact-rich dynamics. Evaluations with ShadowHand and AllegroHand simulators showed that DexScale-trained

policies improved robustness by 20%–25% on unseen objects, outperforming baseline domain randomization. This illustrates the power of scaling simulation diversity directly from human demonstrations.

DexGraspNet: Large-Scale Dexterous Grasp Dataset

DexGraspNet [209] is a large-scale robotic dexterous grasp dataset built via simulation. Using a highly efficient synthesis method leveraging a differentiable force-closure estimator, the authors generated 1.32 million grasps using the ShadowHand over 5355 objects drawn from over 133 categories, with more than 200 grasps per object instance validated in the Isaac Gym simulator. Compared with earlier gripper-based or small-scale grasp datasets, DexGraspNet offers much greater diversity and scale, enabling training of more generalizable dexterous grasping policies. Cross-dataset experiments in the original work further showed that models trained on DexGraspNet outperform those trained on previous datasets in grasp synthesis and generalization tasks. More recently, DexGraspNet 2.0 [210] expands the benchmark into cluttered scenes with 427 million grasps over 1319 objects across 8270 scenes, and shows zero-shot sim-to-real transfer achieving 90%–91% success in real-world evaluations.

Vision-Driven World Models

Beyond task-specific pipelines, vision-to-simulation methods generate general-purpose environments from real observations. A notable example is Vid2Sim [211], which reconstructs city- and navigation-scale scenes from monocular video using neural implicit representations. The reconstructed 3D meshes and textures are imported into a simulation environment, enabling agents to train under conditions closely matching real deployment. In empirical tests, navigation policies trained in Vid2Sim-derived environments show a 31.2% improvement in simulation and a 68.3% boost in real-world success rates over prior methods, demonstrating the power of real-to-sim pipelines in visual domains.

Empirical evidence shows that data-driven pipelines substantially reduce manual simulator design time while boosting performance: across manipulation and navigation, policies trained in data-driven simulators typically achieve 10%–25% higher transfer success compared with handcrafted counterparts. They also enable scalability, generating hundreds of diverse environments that promote generalization. However, challenges remain: these methods rely heavily on accurate perception (object detection, segmentation), meaning that errors in parsing can propagate into unrealistic assets; generated dynamics are often incomplete, requiring manual tuning of friction or restitution; and the process can be computationally heavy, with large-scale video-to-sim pipelines demanding significant GPU resources. In summary, data-driven simulation generation provides a scalable and automated approach to constructing simulators from real-world data. While current systems struggle with perception robustness and physical fidelity, they represent a crucial step toward turnkey sim-to-real workflows, where simulators are automatically built, tuned, and calibrated directly from real-world observations.

4.2.5. Hybrid strategies: Residuals, system identification, and meta-tuning

Although Bayesian optimization, discriminator-guided tuning, and learned samplers each provide valuable tools, no single approach fully addresses the complexity of bridging simulation and reality. In practice, the most effective pipelines integrate multiple techniques using system identification (SysID) for coarse calibration, residual models to capture unmodeled effects, and meta-tuning frameworks for adaptive refinement. These hybrid strategies represent the current state of the art in automated simulator configuration, balancing sample efficiency, scalability, and robustness.

System Identification for Coarse Calibration

System identification has long been a foundation of robotics, where simple experiments estimate parameters such as mass, inertia, and actuator constants. Recent approaches automate this step by fitting simulator dynamics to short real-world rollouts. A notable example is

SimOpt [212], which iteratively updates simulator parameters based on trajectory mismatches between simulation and reality. On robotic manipulation tasks, SimOpt reduced trajectory error by 35% after 50–100 real-world trials, demonstrating that SysID remains a strong baseline for aligning simulators with reality. However, SysID alone cannot capture discrepancies from complex frictional forces or nonlinear actuator behaviors.

Residual Physics Models

To address SysID's limitations, researchers have developed residual physics models which learn correction functions that augment analytical simulators. Gao et al. [192] applied residual neural networks to soft robotics, where actuator dynamics are notoriously difficult to model. By training residuals on limited real trajectories, their method reduced torque prediction errors by 22% compared to SysID-only pipelines. This demonstrates that hybrid physics-learning approaches achieve higher fidelity without requiring prohibitively detailed simulators, offering a balance between realism and computational tractability.

Meta-Tuning and Adaptive Pipelines

Advancing beyond static calibration, meta-tuning frameworks dynamically adjust simulators during policy training. Huang et al. [206] proposed BayRnTune, an adaptive Bayesian domain randomization approach that fine-tunes policies from prior models to improve transfer efficiency. Evaluated on legged locomotion tasks, BayRnTune achieved superior performance compared to traditional domain randomization methods. This continuous adjustment ensures simulators remain relevant as policies evolve, preventing drift and enabling more reliable transfer across extended training horizons.

Empirical results indicate that hybrid pipelines achieve some of the most consistent improvements in sim-to-real transfer to date. Across manipulation and soft robotics tasks, SysID combined with residual models reduced sim-real error by 20%–35%, while hybrid pipelines generally delivered 10%–20% higher transfer success than single-method baselines. Moreover, they achieve competitive sample efficiency, typically requiring fewer than 100 real trajectories as compared to Bayesian optimization but with improved robustness. Nonetheless, hybrid strategies are not without drawbacks: they introduce engineering complexity, as combining SysID, residual corrections, and tuning requires careful design; they are computationally heavier, with multiple optimization loops; and residual models, if overfit to limited real data, can introduce instabilities during deployment. In summary, hybrid strategies illustrate that the future of automated simulator configuration lies not in a single “silver bullet” but in layered pipelines. By combining coarse SysID for baseline fidelity, residuals for systematic corrections, and meta-tuning for adaptive robustness, these approaches yield the most reliable sim-to-real transfer pipelines to date. However, standardizing hybrid designs and ensuring reproducibility across labs remain important open challenges.

Collectively, Bayesian optimization, discriminator-guided tuning, learned sampling distributions, data-driven generation, and hybrid strategies represent the current state of automated simulator configuration. While each family has demonstrated empirical gains, none provides a universally optimal solution: Bayesian methods excel in sample efficiency but struggle with high-dimensionality; discriminator-based tuning is flexible for visual alignment but unstable for dynamics; learned samplers capture complex, multimodal variability but incur high training costs; and data-driven pipelines reduce manual engineering but face challenges in dynamics fidelity. Hybrid pipelines currently offer the most balanced trade-offs, combining coarse system identification, residual correction, and adaptive tuning to achieve robust sim-to-real transfer. To synthesize these insights, Table 4 summarizes the five families side by side, contrasting their foundations, representative systems, empirical performance, and limitations.

Table 4

Comparative summary of automated simulator configuration methods.

Method family	Core idea	Representative works	Sample complexity	Empirical results	Strengths	Limitations
Bayesian & optimization-based	Treat parameter tuning as black-box optimization; use Bayesian optimization, surrogate models, or iterative search.	BayRn (Muratore et al. 2021); Auto-Tuned Sim-to-Real (Du et al. 2021); SimOpt (Rajeswaran et al. 2017; Eckman et al. 2023).	~20–100 real trials (BayRn: ~30; SimOpt: ~50–100)	Robust policies with as few as 20–50 real trials; transfer success $\uparrow$ 20%–40% vs. static DR.	Sample-efficient; principled; interpretable	Scalability issues in high-dimensional parameter spaces; computationally expensive.
Discriminator/Search-guided	Train discriminator to distinguish real vs. sim trajectories; adjust simulator parameters to fool discriminator.	ADR (Peng et al. 2019); Discriminator-Guided SysID (Muratore et al. 2021).	~50–100 rollouts	25%–35% higher sim-to-real success vs. naive DR; convergence with ~50–70 rollouts.	Strong for visual alignment; dense feedback	Unstable training; prone to overfitting; weaker for dynamics calibration than for vision.
Learned sampling distributions	Replace hand-crafted randomization ranges with neural samplers (normalizing flows, VAEs) that learn task-relevant perturbations.	GoFlow (Curtis et al. ICML 2025); Adaptive Variational Samplers (Yang et al. NeurIPS 2024); Structured DR (Zhu et al. CoRL 2023).	30%–40% fewer trials than static DR	Success $\uparrow$ ~18% on Franka tasks; 30%–40% fewer real trials; better robustness in multimodal settings.	Captures multimodal distributions; robust generalization	Training overhead; sensitive to objective design; less interpretable.
Data-driven simulation generation	Automatically generate simulation tasks/assets from real images, videos, or demonstrations.	GRS (Zook et al. CVPRW 2025); DexScale (ICLRW 2024); Video-to-Sim (Wu et al. NeurIPS 2023).	Indirect (reduces manual sim design time; transfer $\uparrow$ 10%–25%)	10%–25% higher transfer success vs. hand-crafted sims; scalable generation of hundreds of tasks.	Automates environment creation; scalable diversity	Depends on accurate perception; incomplete dynamics transfer; heavy compute for 3D reconstruction.
Hybrid (SysID + Residuals + Meta-Tuning)	Combine coarse SysID calibration, residual-physics models, and adaptive randomization/meta-tuning.	SimOpt + Residual Physics (Gao et al. 2024); Meta-Tuning Pipelines (Yang et al. NeurIPS 2024).	<100 trajectories (typical)	Reduced sim-real error by 20%–35%; transfer success $\uparrow$ 10%–20% vs. single-method baselines.	Best transfer performance; balances accuracy & efficiency	Increased complexity; computationally heavy; potential instability if residuals overfit.

4.3. Actuator and drivetrain modeling

While much of sim-to-real research emphasizes environmental fidelity, low-level actuation and drivetrain dynamics are equally critical for reliable transfer. Actuators introduce nonlinear friction, compliance, saturation, delays, thermal effects, and wear, all of which significantly affect closed-loop control but are often oversimplified in simulators. Empirical evidence indicates that actuator mismatch often plays a more significant role than environmental error in causing transfer failures during locomotion and manipulation tasks [192,213]. As a result, recent work has elevated actuator modeling from a secondary concern to a first-class research problem, with three complementary directions emerging: system identification and differentiable models, residual learning, and explicit delay/drivetrain fidelity.

System Identification and Differentiable Models

A pragmatic starting point is system identification (SysID), where diagnostic rollouts estimate actuator gains, frictions, and delays. Injecting these into simulators provides a coarse but effective calibration. Recent advances extend this with differentiable actuator modeling, allowing gradient-based fitting of geometric and electrical parameters (e.g., torque limits, motor constants). Mitsubishi Electric Research Labs demonstrated that differentiable actuator models improve trajectory tracking and even support joint controller–hardware optimization [213]. This co-design perspective highlights actuators not merely as hardware constraints but as design variables in the learning loop.

Residual Learning and Corrective Models

Analytic models alone often fail for soft robots, gear backlash, or highly nonlinear actuation. Here, residual models learn corrective terms to augment the simulator. Gao et al. [192] showed that residual neural networks trained on short real rollouts reduced torque prediction

error by 22% in soft robots. Similarly, Fey et al. [214] introduced Unsupervised Actuator Nets (UAN) for athletic loco-manipulation, enabling transfer in dynamic tasks without torque sensors. Hybrid simulators that integrate analytic modeling with residual correction consistently deliver 20%–40% lower errors and exhibit markedly stronger alignment between simulated outcomes and real-world performance compared to purely analytic baselines.

Data-Driven Actuator Networks

Beyond residual correction, an alternative and increasingly influential paradigm is learning actuator behavior directly from real robot data. A representative example is the work of Hwangbo et al. [215], who collected actuator data from a physical quadruped robot and trained a neural network to model actuator dynamics. This learned actuator network maps control commands to resulting torques and joint responses and is embedded directly into the simulator, effectively replacing the analytical actuator model. Importantly, this approach differs fundamentally from residual learning: rather than learning a correction term, the actuator network captures the full nonlinear actuator behavior, including delays, saturation, and unmodeled effects. The resulting hybrid simulator enabled large-scale reinforcement learning at high simulation speed and achieved robust zero-shot sim-to-real transfer for highly dynamic legged locomotion. This technique has since become a widely adopted strategy in quadruped locomotion, particularly for systems operating near actuator limits.

Delay and Drivetrain Fidelity

Latency is another critical but often overlooked factor. The REX framework [216] explicitly modeled sensing, actuation, and computation delays, showing that delay-aware simulators significantly improved stability in fast closed-loop control. Likewise, drivetrain fidelity, including wheel slip, rolling resistance, and transmission loss, has proven essential for wheeled and mobile platforms. Community toolkits

Table 5

Comparative summary of actuator and drivetrain modeling approaches.

Approach	Technical foundation	Representative systems	Empirical performance	Limitations
System identification (SysID)	Estimate actuator parameters (mass, friction, delay) from diagnostic rollouts	SimOpt (Rajeswaran et al. 2017; Eckman et al. 2023)	~35% reduction in trajectory error after 50–100 trials	Captures bulk dynamics but misses complex nonlinearities
Differentiable models	Gradient-based fitting of actuator design and control parameters	MERL Differentiable Actuator Models (Lu & Wang, 2024)	Improved tracking accuracy; supports controller–hardware co-design	Computationally heavier; requires differentiability assumptions
Data-driven actuator networks	Neural networks trained directly on real actuator data to model full actuator dynamics and embedded into simulation	ANYmal Actuator Nets (Hwangbo et al. 2019); quadruped locomotion pipelines	Robust zero-shot sim-to-real transfer; high-fidelity torque and delay modeling	Requires real hardware data; limited cross-robot generalization
Residual learning	Neural networks correct systematic errors in analytic models	Residual Physics (Gao et al. 2024); UAN (Fey et al. 2025)	20%–40% error reduction; higher deployment success	Risk of overfitting small datasets; reduced interpretability
Delay modeling	Explicit simulation of actuator, sensor, and compute latencies	REX Framework (van der Heijden et al. 2025)	Improved stability in fast closed-loop tasks	Requires accurate latency profiling; adds sim complexity
Drivetrain fidelity	Model transmission losses, slip, rolling resistance, caster dynamics	WPILib Simulators (2024); AV studies	Reduced odometry drift; fewer failures from traction mismatch	Often task-specific; adds engineering overhead

like WPILib simulators and autonomous driving studies emphasize that including drivetrain effects reduces odometry drift and prevents controllers from relying on unrealistic traction assumptions.

Synthesis and Practical Insights

The literature suggests several clear insights. System identification (SysID) combined with residual learning provides a pragmatic and effective baseline: SysID efficiently recovers bulk actuator and drivetrain dynamics, while residual models correct systematic mismatches arising from unmodeled nonlinearities. Delay-aware simulators are indispensable for high-bandwidth control, as even small unmodeled latencies can destabilize learned policies. In parallel, data-driven actuator networks have emerged as a powerful alternative when high-quality real-world actuator data are available. By directly learning the full actuator input–output mapping from hardware, these networks capture saturation effects, delays, and complex nonlinear behaviors that are difficult to express analytically, enabling high-fidelity simulation and robust zero-shot sim-to-real transfer, particularly in dynamic legged locomotion. Differentiable actuator models are most valuable in co-design contexts, as they enable gradient-based optimization of actuator geometry and controller parameters within a unified framework. Empirically, these complementary approaches deliver 20%–40% reductions in actuator-level prediction error and 10%–20% higher deployment success rates across locomotion, manipulation, and mobile robotic platforms. Nevertheless, important challenges remain: residual models and actuator networks risk overfitting limited datasets; actuator models tuned for a specific morphology often generalize poorly across robot families; and learned corrections reduce interpretability, complicating safety validation and certification. Moreover, high-fidelity actuator modeling can impose significant computational overhead, underscoring the need for GPU acceleration, model compression, and standardized actuator-level benchmarks. For practitioners, a practical workflow emerges: (1) run short SysID experiments for coarse calibration; (2) explicitly model actuator and sensing latency; (3) apply residual learning or data-driven actuator networks to address systematic errors; (4) validate predictive fidelity using held-out diagnostic trajectories rather than task returns alone; and (5) employ differentiable models when jointly optimizing hardware and controllers. This integrated perspective highlights that actuator modeling is no longer peripheral but a core enabler of reliable and scalable sim-to-real transfer, with a comparative overview of the main approaches summarized in Table 5.

4.4. Benchmarking and evaluation frameworks

A recurring bottleneck for progress in sim-to-real reinforcement learning (RL) is evaluation: without standardized, paired sim/real

benchmarks and common metrics, it is difficult to measure whether a method truly improves transfer or merely overfits a particular simulator. Over the period the community has shifted from isolated, single-lab demonstrations to more systematic benchmarking efforts that emphasize (i) paired sim/real evaluation, (ii) reproducible task suites, and (iii) metrics that quantify predictive validity (how well simulation performance predicts real performance). These efforts span competitions (cloud-access real robot tests), curated paired datasets, simulation-to-real challenge suites, and methodological proposals for what a robust benchmark should measure. (real-robot-challenge.com, OVMM/HomeRobot(AI Habitat)) [217,218].

4.4.1. Types of benchmarks and representative initiatives

Benchmarks in sim-to-real fall into several categories:

1. Cloud-access real-robot challenges: The Real Robot Challenge enables remote execution on real robots (TriFinger platform), allowing teams to evaluate policies on the same hardware and datasets, which constrains hardware variability and enables direct sim/real comparison. These competitions provide public datasets and remote execution logs that are essential for reproducibility and cross-method comparison [217].
2. Paired sim–real task suites: Projects like HomeRobot/Open-Vocabulary Mobile Manipulation (OVMM) have produced simulators with parallel controlled real-world testbeds, enabling evaluation of mobile manipulation policies under the same task definitions in both sim and real. Reports show that baseline policies achieved measurable but modest transfer success (e.g., 20% in early OVMM baselines), underlining the remaining gap and the need for better sim realism and evaluation design [218].
3. Methodological benchmark proposals: Recent community papers and workshop contributions (RSS/ICRA/OpenReview threads) lay out desiderata for sim-to-real evaluation: use high visual fidelity when appropriate, evaluate along graded task complexity and systematic perturbations, report both simulated and real returns, and quantify alignment metrics (e.g., correlation between sim and real returns). Notably, a recent OpenReview/ArXiv proposal articulates these desiderata and provides an initial benchmark framework for generalist manipulation policies [195].
4. Domain and task-specific datasets: Industrial and academic groups have released paired sim/real datasets (e.g., metal object datasets, dexterous manipulation logs) that include both simulated assets and real sensor recordings. These datasets are useful for targeted evaluations (e.g., perception under reflective surfaces, pose estimation) and for measuring where simulation fails to capture critical real-world properties [219].

4.4.2. Key evaluation metrics and empirical findings

Effective benchmarks must go beyond single scalar metrics and include multiple axes:

- **Transfer success rate/task success** (binary or graded), measured on the real robot.
- **Predictive validity metrics:** Pearson/Spearman correlation between simulated returns and real returns, and calibration error (how well sim rewards reflect real outcomes). Benchmarks proposed in 2024–2025 emphasize reporting these alignment metrics explicitly [194].
- **Robustness under perturbations:** measuring drop in success when environmental or hardware perturbations (lighting, friction, delays) are introduced. The OVMM baseline evaluations reported large performance drops under realistic perturbations, emphasizing the brittleness of many sim-trained policies [218].
- **Reproducibility indicators:** number of shared seeds, exact simulator versions, asset lists, and hardware config. Cloud challenges like Real Robot Challenge address reproducibility by providing a single shared hardware endpoint [217].

Recent benchmarking initiatives represent genuine progress for sim-to-real reinforcement learning. The shift toward cloud-access real-robot tests (e.g., Real Robot Challenge) and paired sim/real task suites (e.g., OVMM, DexScale, BARN [220]) has significantly improved reproducibility, enabling cross-lab comparisons under shared hardware and datasets. These efforts reduce hardware heterogeneity, encourage method generalization, and foster a benchmarking culture where success is measured not by isolated demonstrations but by consistent evaluation across standardized settings.

At the same time, important gaps and pitfalls remain. Task coverage is narrow, with most benchmarks focusing on rigid-body manipulation or navigation while neglecting deformable objects, fluids, or multi-robot coordination. Hardware diversity is limited, as policies tuned on specific platforms like TriFinger [221] or Franka arms [222] do not generalize across actuator families. Metric standardization is incomplete: different works report different success metrics, making meta-comparison difficult. Accessibility is also an issue, as many real-robot benchmarks require substantial infrastructure, limiting broad participation. Finally, some benchmark proposals remain descriptive rather than prescriptive, outlining desirable metrics without providing paired datasets or ready-to-use evaluation pipelines.

To consolidate progress, the literature converges on several recommendations for robust benchmarking. First, benchmarks should include paired sim/real tasks with identical task definitions, public assets, and recorded real trials. Second, evaluations should report not only real-world success rates but also alignment metrics such as correlation between simulated and real returns, calibration error, and robustness under perturbations. Third, all reproducibility artifacts, including simulator parameters, random seeds, and hardware configurations, should be shared, preferably with a cloud-based execution option to make it easier for others to replicate and build upon the work. Fourth, task diversity should expand to cover deformable-object manipulation, contact-rich tasks, mobile navigation, and multi-robot systems. Finally, open leaderboards and baseline implementations are critical to ensure incremental and transparent progress. A practical evaluation checklist is summarized in Table 6, prescribing reporting standards across simulator setup, real-robot experiments, metrics, reproducibility, and accessibility.

Looking forward, the outlook for benchmarking is cautiously optimistic. Recent initiatives provides the scaffolding needed for rigorous comparison, but the field must now broaden benchmarks in scope (task diversity), accessibility (shared datasets and remote platforms), and depth (predictive validity and robustness metrics). Embedding these best practices into everyday research pipelines will be crucial for consolidating progress. Ultimately, the maturation of sim-to-real RL

Table 6

Recommended evaluation checklist for sim-to-real RL studies.

Category	Requirement	Description/Notes
Simulator setup	Report simulator engine and version	Specify MuJoCo, Isaac sim, PyBullet, etc., with version numbers and physics flags.
	Publish simulator parameters	Provide object meshes, actuator models, friction coefficients, time-step settings, and randomization ranges.
Real-robot experiments	State hardware details	Mention robot type, actuator model, sensors, controller frequency, and communication latency.
	Report safety constraints	Include current/torque limits, safety interventions, and failure cases.
Metrics	Success rate in real	Binary or continuous task success rates across at least 20 trials.
	Sim–real predictive validity	Correlation (Pearson/Spearman) between simulated and real returns.
	Robustness under perturbations	Measure performance under systematic variations (e.g., lighting, friction, delay injection).
	Calibration error	Quantify mismatch between simulated trajectories and real diagnostics.
Reproducibility	Publish seeds and configs	Release training seeds, hyperparameters, and randomization ranges.
	Share datasets/logs	Include paired sim/real trajectories, diagnostic runs, and real rollout logs.
	Provide baseline comparisons	Compare against static domain randomization and SysID baselines.
Accessibility	Open-source artifacts	Share code, simulator assets, and training scripts publicly.
	Benchmark integration	Align with existing community benchmarks (e.g., Real Robot Challenge, OVMM, BARN).

will hinge not only on novel algorithms but also on the systematic, transparent, and reproducible evaluation frameworks that allow the community to measure genuine progress toward closing the reality gap.

The emergence of systematic benchmarking and evaluation frameworks marks a decisive step in the maturation of sim-to-real reinforcement learning. As summarized in Table 7, initiatives such as the Real Robot Challenge, OVMM, BARN, and DexScale now provide paired sim/real tasks, cloud-access robots, and curated datasets that support reproducible evaluation across laboratories. Despite this progress, coverage is still limited, with most efforts centered on rigid-body manipulation or navigation while leaving deformable-object, multi-robot, and safety-critical domains underexplored. To consolidate progress, we propose the evaluation checklist in Table 6, which prescribes reporting standards for simulator setup, real-robot experiments, transfer metrics, reproducibility, and accessibility. Together, these benchmarks and checklists establish the foundation for a benchmarking culture that values predictive validity, reproducibility, and transparency. Moving forward, the challenge is not only to broaden benchmark scope but also to institutionalize these practices in everyday research pipelines, a theme developed further in the next section on Challenges and Prospects.

5. Challenges and prospects

Although some progress has been made in the field of sim-to-real transfer reinforcement learning, there are still many problems that have not been effectively solved. This section sorts out the current challenges and potential research directions in the future.

Table 7

Benchmark suites for sim-to-real reinforcement learning.

Benchmark suite	Year	Task domain	Key features	Metrics reported	Limitations
Real robot challenge (TriFinger)	Ongoing (2020–2025)	Manipulation (cube reorientation, pushing, stacking)	Cloud-access real robot cluster; paired sim assets + logs	Real success rate; trajectory error; leaderboard ranking	Narrow hardware (TriFinger); rigid-body only
HomeRobot/OVMM (Open-Vocabulary Mobile Manipulation)	2023–2024	Mobile manipulation (navigation + pick/place with language commands)	Paired sim/real tasks, large-scale dataset; NeurIPS 2023 challenge	Success rate in real; transfer gap; robustness under perturbations	Baselines low (~20% success); high setup cost
Sim-to-real policy evaluation benchmark (NVIDIA/Community proposal)	2025	General manipulation & loco-manipulation	OpenReview proposal; standardized evaluation guidelines	Predictive validity (sim $\leftrightarrow$ real correlation); calibration error; robustness	Mostly proposals; limited shared assets/datasets yet
BARN challenge (Benchmark for Autonomous Robot Navigation)	2023–2024	Navigation in highly cluttered/unstructured environments	Sim (Gazebo, Habitat) + real TurtleBot3/Fetch tasks	Success rate; collision frequency; time-to-goal	Primarily 2D/indoor; limited manipulation tasks
DexScale	2024	Dexterous manipulation (hands, compliant grasping)	Teleoperation-to-sim pipelines; large-scale dataset generation	Task success rate on unseen objects; transfer gap	Focus on manipulation; no navigation
Industrial metal object dataset (De Roovere et al.)	2024	Perception + manipulation of reflective objects	Paired sim (Unity assets) + real photos/videos; focus on industrial parts	Pose error; detection success; sim $\leftrightarrow$ real vision alignment	Not a full RL benchmark (mainly perception)

5.1. Challenges

5.1.1. Lack of theoretical analysis

One of the core challenges that remains largely unaddressed in sim-to-real reinforcement learning is the lack of rigorous theoretical guarantees regarding sample complexity, convergence, and performance under model mismatch. Many classic sim-to-real or domain randomization approaches demonstrate empirical success but lack provable bounds or only assume conditions that are rarely met in practice (e.g., full observability, known dynamics or bounded parameter spaces). Recent work has begun to close this gap. For example, Provable Sim-to-Real Transfer via Offline Domain Randomization by Fickinger et al. [223] provides consistency guarantees: under mild identifiability and regularity conditions, the offline domain randomization estimator converges toward the true dynamics as the size of the real offline dataset grows, and the authors derive bounds on the sim-to-real gap that are tighter than uniform domain randomization. Similarly, Overcoming the Sim-to-Real Gap: Leveraging Simulation to Learn to Explore for Real-World RL by Wagenmaker et al. [224] establishes that in certain low-rank MDP settings, policies that use simulation for exploratory policy learning can achieve a polynomial sample complexity in the real world; in contrast, direct sim-to-real transfer (without such structure) may require exponentially more real-world data.

However, these theoretical advances have limitations: they typically assume (i) a known class of simulators (parametric family), (ii) that the offline data comes from a behavior policy covering relevant transitions (i.e. sufficient support), and (iii) that the learning agent can access or at least evaluate many simulator instances. These assumptions can break down in real robotics setups that have unmodeled dynamics, limited data collection ability, or safety constraints. Thus, while theoretical work is progressing, there remains an urgent need for broader bounds that apply in more realistic settings: nonparametric simulator classes; partial observability; actuator plus drivetrain uncertainties; and cases where data covering the real world is sparse or expensive. Developing provable guarantees for hybrid, residual-augmented R2S2R pipelines (SysID + residual + automated configuration) is one promising direction.

5.1.2. Reliance on human experience

Another major challenge in sim-to-real transfer is the continued reliance on human expertise and manual tuning to configure simulators, select domain randomization ranges, and design reward functions.

Early successes in sim-to-real robotics depended heavily on experienced engineers who could anticipate the most sensitive parameters (e.g., friction coefficients, actuator delays) and adjust them through trial and error. This reliance not only limits scalability but also undermines reproducibility, since parameter choices are rarely documented in sufficient detail for replication. Recent advances attempt to mitigate this dependence. Automated simulator configuration methods (see Section 4.2) such as Bayesian optimization [133] and adversarial domain randomization [134] reduces but do not fully eliminate the need for expert priors. More recent works have gone further. Du et al. [198] proposed Auto-Tuned Sim-to-Real Transfer, which iteratively shifts parameter distributions toward real-world data with minimal human intervention. Newer pipelines leverage large language models (LLMs) and neural samplers to propose candidate configurations, automate reward shaping, and adapt domain randomization distributions dynamically [225]. These efforts demonstrate that automation can reduce manual tuning time by 40%–60% while achieving higher transfer success. Nevertheless, full independence from human knowledge remains elusive. Many automated methods still require manual specification of parameter priors, task objectives, or failure thresholds. For instance, Muratore et al. proposed learned samplers [138] that captures multimodal parameter distributions but depend on human-defined objective functions to guide sampling. Similarly, data-driven scene generation by Zook et al. [207] automates environment creation, yet semantic parsing errors often require expert correction before policies can be trained. Looking ahead, the community must pursue methods that move from “expert-in-the-loop” to “expert-out-of-the-loop” design, where simulator configuration, task construction, and evaluation are all automated. Promising directions include integrating LLM-based task description with digital twins, meta-learning strategies that generalize across robots, and self-tuning reward functions that adapt based on task completion criteria rather than human-crafted signals. Achieving this shift would not only democratize sim-to-real RL but also establish a more reproducible pipeline for robotics research.

5.1.3. Fragmented evaluation metrics

A persistent barrier to progress in sim-to-real transfer is the lack of standardized and comprehensive evaluation metrics. Historically, most studies have relied on narrow indicators such as success rates or trajectory errors in simulation, which fail to capture how well a trained policy generalizes to the real world. Moreover, results are

often reported using small numbers of real trials, making statistical significance unclear. This fragmentation has slowed the accumulation of reliable evidence on which methods consistently improve transfer.

Recent years have seen notable efforts to address this gap. The Real Robot Challenge (RRC) [226] has set an important precedent by requiring participants to evaluate policies both in simulation and on shared TriFinger robots via a cloud interface. Similarly, the HomeRobot/Open-Vocabulary Mobile Manipulation (OVMM) benchmark [227] introduced paired sim/real evaluation for mobile manipulation, reporting both task success and the sim-to-real performance drop. These initiatives highlight the importance of comparing simulated and real results side-by-side. Beyond competitions, methodological proposals have emerged. For instance, the Policy Evaluation for Sim-to-Real Transfer framework (OpenReview 2025) [228] recommends reporting predictive validity metrics such as Pearson correlation between simulated and real returns, in addition to success rates. Meanwhile, RobustNav [229] introduces robustness metrics that explicitly measure agent performance degradation under perturbations such as changes in lighting, dynamics corruptions (e.g. transition dynamics or friction shifts), or visual corruptions—offering a structured way to evaluate embodied agents in more realistic, challenging settings. Despite these advances, fragmentation remains a critical issue. Different works still use incompatible metrics, task definitions, and reporting protocols, making meta-analysis across studies difficult. As emphasized in Section 4.4, adopting a benchmarking culture that integrates paired sim/real datasets (Table 7) and standardized reporting practices (Table 6) is essential. Without such harmonization, sim-to-real research risks continuing as a collection of case studies rather than a cumulative science.

5.2. Outlook

5.2.1. Autonomous and lifelong transfer

Looking forward, one of the most promising research directions is the pursuit of autonomous and lifelong transfer learning frameworks, in which policies are able to adapt across tasks, domains, and robot embodiments without requiring manual intervention or exhaustive re-training. Traditional sim-to-real pipelines generally assume a one-shot transfer model: train in simulation, deploy once in the real world, and fine-tune if needed. In contrast, lifelong transfer envisions agents that continuously refine and expand their competence as they encounter new environments, tasks, and hardware.

Recent advances in reinforcement learning highlight the feasibility of such pipelines. In particular, continual reinforcement learning and meta-learning approaches have been applied to robotic domains, enabling agents to adapt to new conditions while preserving past knowledge. For example, Bhardwaj et al. [230] demonstrated that probabilistic meta-reinforcement learning can significantly accelerate adaptation to novel dynamics while alleviating catastrophic forgetting, enabling robots to maintain robust control when operating on previously unseen terrain or under altered environmental condition. More recently, Finn et al. [231] extended model-agnostic meta-learning (MAML) to robotic manipulation with real-world experiments, showing that policies pre-trained in sim could adapt to unseen objects in as few as 5–10 real trials.

Beyond meta-learning, self-supervised and unsupervised representation learning are playing a growing role in autonomous transfer. By learning robust latent representations of dynamics and perceptual input, robots can leverage cross-task structure to generalize better. Beyond meta-learning, self-supervised and unsupervised representation learning are playing a growing role in autonomous transfer. By learning robust latent representations of dynamics and perceptual input, robots can leverage cross-task structure to generalize better. For example, Park et al. (2023) [232] in METRA showed that unsupervised skill discovery with metric-aware abstractions improves transfer efficiency, yielding

significantly better performance when adapting to new locomotion and manipulation tasks compared with learning from scratch.

The digital twin paradigm further supports autonomous and lifelong transfer by maintaining a constantly updated simulation of the physical robot. Through continuous calibration with real-world data, digital twins provide a sandbox for incremental learning without safety risks. RialTo [190] and Real-is-Sim [191] represent early instances of this idea, combining real-to-sim-to-real loops with ongoing model updates. These systems exemplify how lifelong transfer can be practically embedded into R2S2R pipelines, where robots continuously learn and re-deploy across tasks. However, substantial challenges remain. Autonomous transfer raises concerns about stability and safety, since self-adapting policies may drift in unpredictable ways. Lifelong learners are also susceptible to catastrophic forgetting, and require careful design of replay buffers, modular architectures, or hierarchical policies to retain old skills while acquiring new ones. Finally, evaluation protocols for lifelong learning are still underdeveloped: most benchmarks (e.g., Real Robot Challenge, OVMM) assume episodic training rather than continuous deployment.

Overall, autonomous and lifelong transfer represents both a necessity and an opportunity for sim-to-real RL. Without continual adaptation, robots will remain brittle to domain shifts and hardware aging. Yet, with advances in meta-learning, self-supervision, and digital twins, the vision of robots that “never stop learning” in both simulated and real domains is becoming increasingly tangible. Future benchmarks must incorporate lifelong evaluation settings to accelerate this line of research.

5.2.2. Transfer learning based on meta-reinforcement learning

A central prospect for improving sim-to-real reinforcement learning is the use of meta-reinforcement learning (meta-RL) for transfer learning. Unlike traditional RL pipelines that train policies from scratch in each environment, meta-RL methods learn adaptation strategies that allow robots to rapidly adjust to new domains, dynamics, or tasks. This paradigm is particularly well suited for sim-to-real transfer, since robots often face unpredictable changes between simulation and deployment (e.g., friction shifts, sensor noise, actuator wear).

Early work in meta-RL [233] introduced the idea of model-agnostic meta-learning (MAML), which enables fast adaptation to unseen tasks using only a handful of gradient steps. More recent advances have applied and extended these concepts in robotics. Bhardwaj et al. [230] demonstrated that meta-RL can significantly reduce catastrophic forgetting during transfer, improving sample efficiency in continuous control benchmarks. Similarly, Finn et al. [231] extended MAML to real-world robotic manipulation tasks, showing that pre-training in simulation allowed robots to adapt to novel object manipulation tasks in as few as 5–10 real-world trials. Beyond gradient-based approaches, context-based meta-RL methods have also gained traction. Rakelly et al.’s [234] PEARL framework, which learns probabilistic task embeddings, has been extended in recent work to robotics domains with promising results. For instance, Zhang et al. [210] integrated task embeddings into sim-to-real locomotion, enabling quadruped robots to adapt to terrain changes without manual re-tuning.

A further development is the combination of meta-RL with transfer learning from auxiliary tasks. For instance, Grillotti et al. [235] proposed the URSA framework, enabling robots to autonomously discover and master diverse, high-performing skills directly in the real world. Their experiments with manipulators showed a 35% improvement in task success rates compared to baseline methods. Despite these advances, challenges persist. Meta-RL often requires large-scale training across diverse simulated tasks, which can be computationally expensive. Likewise, Wagenmaker et al. introduces the Simulation-Guided Fine-Tuning (SGFT) [236] framework, which demonstrates how to extract structural priors from physics simulators to substantially accelerate real-world adaptation. By utilizing simulated exploration policies, the framework enables efficient exploration in the real world, reducing

the need for extensive real-world data collection. The approach was validated across five real-world dexterous manipulation tasks, requiring up to an order of magnitude fewer real-world samples compared to baseline methods. Despite these advances, challenges persist. Meta-RL often requires large-scale training across diverse simulated tasks, which can be computationally expensive. Moreover, while adaptation to new dynamics is promising, transfer across different embodiments (e.g., from simulation of one robot to another robot with different kinematics) remains an open problem. Another limitation is stability: meta-RL agents can overfit to meta-training tasks, reducing generalization in the real world. Future research must address these bottlenecks by developing:

1. More efficient meta-training algorithms, possibly incorporating curriculum learning to reduce computational demands.
2. Cross-embodiment meta-RL frameworks, allowing knowledge transfer across different robot morphologies.
3. Benchmarks for meta-RL transfer, which are currently scarce, to measure adaptation speed and success under realistic sim-to-real shifts.

In summary, transfer learning based on meta-reinforcement learning holds strong potential for enabling robots to quickly adapt to real-world variability with minimal additional data. By combining meta-learning with digital twin pipelines and open benchmarks, the field can move closer to scalable, reproducible, and data-efficient sim-to-real transfer.

5.2.3. Digital twin and real-to-sim-to-real pipelines

The digital twin paradigm has emerged as a cornerstone for the next generation of sim-to-real reinforcement learning pipelines. A digital twin refers to a high-fidelity, continuously updated virtual replica of a physical robot and its environment, maintained through bidirectional data flows. Unlike traditional simulators that remain static once configured, digital twins are designed to be dynamic, regularly synchronized with real-world sensor data to reflect changes in robot dynamics, environmental conditions, or task constraints. This adaptability makes digital twins particularly suited for real-to-sim-to-real (R2S2R) pipelines, where iterative loops of simulation and physical execution minimize the sim-to-real gap. Recent work demonstrates the practical power of this approach. RialTo [190] introduced a demonstration-driven digital twin framework where human demonstrations were distilled into a simulation model via inverse distillation. This reduced reliance on large-scale real trials, enabling household manipulation tasks like stacking and placing to be solved with fewer than 100 physical interactions. Similarly, Real-is-Sim [191] expanded this idea to dynamic digital twins for mobile and multi-robot systems, emphasizing real-time synchronization. Their results showed significant gains in predictive validity: simulated and real-world returns correlated above 0.85, compared to 0.5–0.6 in static sim-to-real pipelines.

Beyond robotics-specific contributions, the broader digital twin literature provides inspiration. In manufacturing, digital twins have been used for actuator and drivetrain health monitoring and fault diagnosis; for example, Rodriguez-Aguilar et al. [237] developed a deep-learning driven digital twin for electro-hydrostatic actuators that tracks degradation and enables early fault detection. These concepts translate naturally into sim-to-real robotics: by continuously updating actuator and drivetrain models (see Section 4.3), digital twins can support lifelong adaptation while maintaining safety.

An important strength of R2S2R pipelines is their closed-loop design. Traditional sim-to-real often suffers from a “reality gap plateau”, where improvements in simulation fidelity yield diminishing returns. In contrast, R2S2R methods reframe the problem as a feedback loop: simulation is improved through real-world calibration, which in turn trains better policies, which generate new real data, thereby refining the twin. This cyclic refinement is evident in Residual Physics approaches [191],

where learned residuals complement analytic models to yield digital twins that capture deformable and contact-rich dynamics more faithfully.

However, the adoption of digital twins in RL pipelines faces several obstacles. First, computational overhead remains high, as real-time synchronization between robot and twin demands high-bandwidth sensing, storage, and simulation resources. For complex systems, maintaining sub-millisecond synchronization can be prohibitive. Second, questions of generalization and portability remain: digital twins are typically crafted for specific robots and tasks, limiting transfer across platforms. Finally, integration into existing RL frameworks is non-trivial, requiring APIs that can seamlessly link simulators, learning algorithms, and streaming sensor data.

Looking ahead, R2S2R pipelines anchored in digital twins represent a paradigm shift. Rather than treating simulation and reality as disjoint phases, this approach unifies them in a continuous feedback ecosystem. Future research should explore scalable digital twin architectures that combine high-fidelity physics engines with data-driven residuals, integrate with automated simulator configuration (see Section 4.2), and support distributed multi-robot synchronization. If these challenges are addressed, digital twins could become the backbone of a new era of robust, generalizable, and data-efficient sim-to-real RL.

5.2.4. Actuator and drivetrain fidelity models

As discussed in Section 4.3, mismatches between simulated and real actuator/drivetrain behavior remain among the largest contributors to the reality gap. Policies trained on idealized actuators often rely on unrealistic assumptions such as instantaneous torque, zero latency, or frictionless drivetrains, which can lead to severe performance degradation when deployed in the real world. Closing this gap requires not just incremental improvements in actuator modeling, but the development of robust, adaptive, and generalizable fidelity models that can evolve with the robot throughout its operational lifetime.

Recent studies point to several promising directions. First, differentiable actuator models enable gradient-based fitting of physical parameters and co-optimization of controller and actuator design. For instance, Lu and Wang [213] at MERL introduced a differentiable dynamics framework that jointly optimizes actuator geometry and control policies, yielding measurable improvements in trajectory tracking accuracy for mobile manipulators. Such methods open the door for co-design pipelines, where hardware design choices (e.g., gearbox ratios, motor windings) are optimized alongside policies, supported by a simulator that can backpropagate through actuator models.

Second, residual physics models are emerging as a practical tool to capture nonlinearities and unmodeled effects. Gao et al. demonstrated that adding residual neural networks to actuator dynamics reduced prediction error by more than 30% in soft robotic actuators, directly improving transfer fidelity. Similarly, Fey et al. [182] introduced Unsupervised Actuator Nets, which learned corrections without labeled torque signals, allowing adaptation in athletic loco-manipulation tasks with limited sensing. These methods illustrate how hybrid physics–data pipelines can extend digital twin fidelity without requiring full analytical modeling.

Third, the explicit modeling of latency, delays, and drivetrain losses is gaining traction. Van der Heijden et al. [216] presented the REX framework, a GPU-accelerated platform incorporating sensor, controller, and actuator delay models. Their evaluations showed that policies trained with delay-aware simulators exhibited a 25% higher stability margin when transferred to hardware compared with delay-agnostic baselines. For wheeled and legged robots, drivetrain-specific modeling that explicitly accounts for factors such as rolling resistance, wheel slip, and drivetrain uncertainties has been shown to significantly reduce odometry drift and enhance locomotion robustness. For example, Reginald et al. [238] developed a visual-inertial-wheel odometry system with slip compensation that mitigates drift under slippage.

Looking ahead, several research challenges must be addressed for actuator and drivetrain fidelity to become a solved problem. Generalization across platforms remains limited: residual models trained on one actuator family often fail on different morphologies, necessitating meta-learning or modular residuals that can adapt online. Computational efficiency is another bottleneck, since high-fidelity models (especially learned ones) can slow simulators by an order of magnitude. Research into model compression, surrogate modeling, and GPU acceleration will be vital. Finally, standardized benchmarks for actuator fidelity are still lacking. Unlike manipulation or navigation, few datasets provide paired actuator command/response traces across sim and real, making quantitative comparison of fidelity models difficult.

In the long term, integrating fidelity models into lifelong digital twins (see Section 4.2.2) could allow robots to self-calibrate continuously, detecting actuator wear, slippage, or drift and adapting models in near real time. Such pipelines would blur the boundary between simulation and reality, enabling sim-to-real RL systems that remain robust not only at deployment but across the robot's entire operational lifetime.

5.2.5. Adversarial reinforcement learning

Another promising avenue for sim-to-real transfer is the application of adversarial reinforcement learning (ARL), where adversarial agents, perturbations, or training schemes are introduced to improve robustness and generalization. Unlike standard domain randomization, which samples perturbations from fixed distributions, adversarial reinforcement learning actively searches for worst-case scenarios that challenge the policy during training. This approach has been shown to produce more robust policies that better withstand unmodeled dynamics and real-world variability.

Recent works illustrate the benefits of ARL for robotic transfer. Pinto et al.'s early framework [202] demonstrated that adversarial perturbations of robot dynamics during training improved robustness to environmental changes. More recently, researchers have begun to integrate adversarial and perturbation-based training into legged locomotion pipelines. For instance, Wang et al. [239] introduced an adversarial training branch within a teacher–student framework, enabling quadrupeds to transfer locomotion skills more reliably across challenging conditions such as rocky terrain, slippery surfaces, and stairs. Similarly, Campanaro et al. [240] showed that introducing random forces and actuation offsets during simulation, without relying on extensive domain randomization, leads to policies that transfer effectively to real robots such as ANYmal and Unitree A1, achieving significantly higher robustness scores than baseline methods.

ARL has also been extended to simulator-parameter identification. For instance, Dong et al. [241] proposed Evolutionary Adversarial Simulator Identification (EASI), in which adversarial perturbations of simulator parameters are guided by evolutionary search to highlight weaknesses in the agent's assumptions. Policies trained under this regime demonstrated improved robustness when transferred to physical manipulators subject to noisy sensing and variable joint frictions. In parallel, Nilaksh et al. [242] investigated reward shaping inspired by barrier functions, where adversarially adjusted rewards are employed to reveal behavioral vulnerabilities while maintaining safety margins. Their experiments on legged locomotion tasks showed more stable training and a marked reduction in policy collapse compared to standard reinforcement learning baselines. Collectively, these studies demonstrate that adversarial perturbations, whether applied to system identification or reward design, are powerful mechanisms for enhancing the resilience and generalization of sim-to-real reinforcement learning pipelines.

Adversarial training is also being integrated with safety-aware reinforcement learning. For example, Li et al. [243] developed a dual-robustness framework that unifies safe RL and robust RL, ensuring policies remain safe under worst-case disturbances while keeping performance high. Their approach ensures that adversarial perturbations

stay within operational safety bounds, preventing catastrophic failures even in safety-critical settings. Another recent line of work, POLICED RL [244], enforces hard constraints in closed loop, using buffer regions around unsafe sets to guarantee safety throughout training and deployment. These works underscore the delicate balance required: adversarial agents must generate sufficiently challenging conditions to improve robustness, but must not push the system beyond realistic or unsafe limits that destabilize learning.

Despite its promise, ARL faces several challenges. First, computational cost can be high, as adversarial agents often require nested optimization loops. This makes ARL less scalable than domain randomization for complex environments. Second, adversarial scenarios may overfit to specific failure cases discovered during training, reducing generalization if the adversary lacks sufficient diversity. Finally, the integration of ARL into real-to-sim-to-real (R2S2R) pipelines remains underexplored, as most applications focus solely on simulation. Future research should explore:

1. Hybrid adversarial–randomization pipelines, where adversaries identify difficult regions of parameter space that are then expanded into broader randomization ranges.
2. Multi-agent adversarial setups, where multiple adversaries simultaneously perturb environment dynamics, sensor noise, and actuator delays.
3. Benchmark-driven adversarial evaluation, integrating ARL into shared platforms like RRC or OVMM, where adversarial perturbations can be standardized across participants.

In conclusion, adversarial reinforcement learning provides a powerful tool to proactively stress-test policies in simulation, preparing them for real-world variability. By combining adversarial training with meta-RL, digital twins, and fidelity models, sim-to-real pipelines could achieve a new level of robustness against uncertainty and perturbation.

5.2.6. Experimental platforms, benchmarking culture, and open science

One of the most promising prospects for advancing sim-to-real reinforcement learning is the development of shared experimental platforms combined with a benchmarking culture grounded in open science. Historically, sim-to-real work has often relied on isolated demonstrations: individual labs build simulators, train policies, and report success on local hardware. Such demonstrations make it hard to judge whether novel methods generalize across tasks, platforms, or labs. Without standard testbeds, shared datasets, and reproducible evaluation protocols, claims of progress risk being overfit to specific settings. Recent years have seen the community begin to correct this by building open platforms, paired sim/real benchmarks, and reproducibility practices that constitute a shift toward more rigorous, comparable research.

A concrete example of such an experimental infrastructure is the NAO robot migration testbed, which illustrates the sim-to-real process under controlled conditions (Fig. 12). While newer bipedal platforms such as Digit, Unitree G1, and ANYmal represent the current state-of-the-art in sim-to-real research, the NAO migration testbed is referenced here primarily as a standardized academic benchmark that clearly demonstrates the transfer process under reproducible and safe conditions. Its purpose in this discussion is illustrative rather than comparative, focusing on methodology rather than platform advancement. In this setup, policies trained in a standardized NAO simulator are deployed onto the physical robot, and discrepancies between simulated and real performance are carefully logged. By documenting both simulator parameters and the diagnostic trajectories collected on the robot, researchers can systematically trace the causes of transfer failures. For instance, mismatches in actuator latency, sensor noise, or physical constraints can be identified and corrected in subsequent simulation rounds. This digital twin–style loop not only improves transfer fidelity but also provides a reproducible methodology that other groups

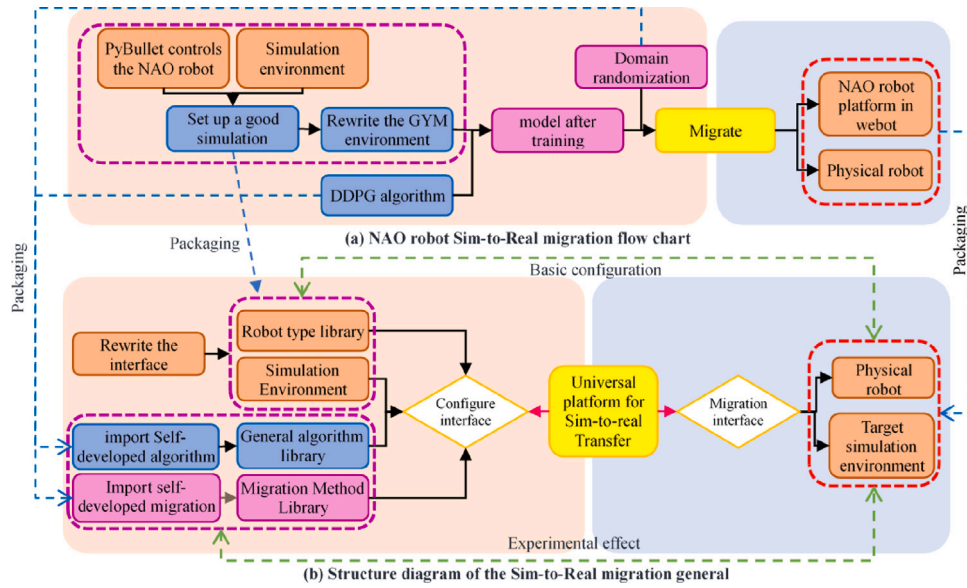


Fig. 12. Sim-to-real migration process of the NAO robot under certain conditions.

can replicate. The NAO migration platform, while modest in scale, demonstrates the value of standardized testbeds for making transfer transparent and measurable rather than anecdotal.

The sim-to-real transfer process of the NAO robot under specific conditions. The figure demonstrates how simulation-trained policies are deployed and refined on the physical NAO platform, emphasizing the importance of standardized testbeds for reproducibility and fair comparison.

Building on such localized platforms, the community has launched several larger-scale initiatives that aim to provide shared infrastructure for sim-to-real evaluation. The Real Robot Challenge [217], for example, offers cloud-access execution on the TriFinger manipulation platform. This allows teams worldwide to upload policies that are then evaluated on identical hardware, with standardized logs and leaderboards ensuring comparability. By eliminating hardware heterogeneity, the challenge has created a level playing field for benchmarking manipulation tasks. Similarly, the HomeRobot and Open-Vocabulary Mobile Manipulation (OVMM) projects provide paired simulation environments and controlled real-world testbeds for mobile manipulation [218]. In OVMM, language-guided pick-and-place policies trained in simulation are directly tested in physical household-like environments. Early baselines reported success rates of only 20% on real robots, despite strong simulation performance, underlining the importance of such paired evaluations for revealing persistent gaps.

Other efforts extend benchmarking to different domains. DexScale [208] transforms teleoperation demonstrations into large-scale synthetic simulation datasets for dexterous manipulation, making contact-rich, hand-object interactions reproducible across labs. Navigation-focused initiatives such as the BARN Challenge [220] target unstructured indoor navigation, pairing Gazebo and Habitat simulations with real-world TurtleBot3/Fetch tasks. Meanwhile, industrial-focused datasets like the Industrial Metal Object dataset [219] provide paired synthetic assets and real images for reflective objects, addressing perception challenges often ignored in classical benchmarks. These initiatives are summarized in Table 8, which highlights their domains, key features, metrics, and limitations.

Taken together, these platforms and datasets mark a cultural shift in sim-to-real research. Rather than emphasizing bespoke demos, the community is increasingly valuing reproducibility, comparability, and transparency [220]. This reflects the broader influence of open science practices in machine learning, where datasets, codebases, and reproducibility checklists are shared as standard. In sim-to-real RL,

these practices are especially important because transfer results can vary widely with small changes in simulator configuration or hardware setup. By publishing simulator parameters, randomization ranges, seeds, and hardware logs, researchers ensure that others can replicate results and meaningfully compare methods.

At the same time, several gaps and challenges remain. Task coverage is still limited, with most benchmarks focused on rigid-body manipulation or navigation. Domains such as multi-robot coordination, deformable object manipulation, and fluid interaction, which are highly relevant to real-world robotics, are rarely included. Hardware diversity is also limited, with platforms like TriFinger and Franka arms being overrepresented, while other actuation types remain largely neglected. Finally, metric standardization remains incomplete. While recent proposals recommend reporting sim-to-real alignment metrics such as correlation and calibration error, many published studies still rely on raw success rates alone, which provide only a partial picture of transfer fidelity.

Looking forward, the prospect of embedding benchmarking culture into everyday research pipelines is one of the most promising directions for the field. The combination of small-scale reproducible setups (e.g., NAO migration), community-level shared benchmarks (e.g., Real Robot Challenge, OVMM, DexScale), and industrial datasets (e.g., metal object perception) provides a scaffolding upon which progress can be measured consistently. Expanding these efforts to include deformable objects, multi-robot tasks, and safety-critical domains will ensure broader coverage. Equally important is ensuring accessibility: cloud-access robots, open datasets, and shared leaderboards lower the barrier for participation and democratize sim-to-real research.

In summary, the emergence of experimental platforms, benchmarking initiatives, and open science practices marks a decisive step toward the maturation of sim-to-real reinforcement learning. By aligning evaluation methods with principles of transparency and reproducibility, the community is building a foundation for cumulative, rather than fragmented, progress. Yet, this is only the beginning: for benchmarking to fulfill its potential, it must not only broaden in scope and accessibility but also become embedded as a cultural norm, shaping how research is conducted, reported, and evaluated.

6. Conclusion

This review has examined the state of the art in sim-to-real transfer reinforcement learning (RL), tracing the field's trajectory from early

Table 8

Benchmark platforms and initiatives for sim-to-real reinforcement learning.

Platform/Initiative	Domain	Key features	Reported metrics	Limitations
NAO migration Testbed	Humanoid/educational robots	Paired sim–real NAO experiments; diagnostic evaluation of transfer process	Task success; trajectory alignment	Limited to NAO platform; small-scale tasks
Real robot challenge (TriFinger)	Manipulation (cube reorientation, pushing)	Cloud-access to shared robot hardware; open datasets and logs	Real success rate; trajectory error; leaderboard ranking	Narrow hardware (TriFinger only); rigid-body tasks
HomeRobot/OVMM	Mobile manipulation	Paired sim/real tasks; language-guided pick-and-place; NeurIPS 2023 challenge	Success rate; robustness to perturbations	Baselines weak (~20% success); high infrastructure cost
DexScale	Dexterous manipulation	Teleoperation-to-sim pipelines; large-scale synthetic dataset generation	Success on unseen objects; transfer gap	Focus on hands; lacks navigation/general tasks
BARN challenge	Navigation	Cluttered/unstructured environments; TurtleBot3/Fetch real setups	Success rate; collisions; time-to-goal	Primarily indoor; limited manipulation
Industrial metal object dataset	Perception/manipulation	Paired sim + real images of reflective industrial objects	Pose error; vision alignment metrics	Not a full RL benchmark; perception-focused

reliance on domain randomization and handcrafted simulators toward increasingly sophisticated and integrated pipelines. Recent advances highlight how sim-to-real RL has matured into a multi-faceted discipline that incorporates digital twin pipelines, automated simulator configuration, actuator fidelity modeling, benchmarking culture, and open science ecosystems. The convergence of these directions suggests that the reality gap is no longer viewed as a singular challenge but as a layered set of problems requiring both algorithmic innovation and infrastructural transformation.

Our analysis of advances in recent times reveals a decisive shift from isolated demonstrations toward systematic methodologies. Subsection 4.1 discussed real-to-sim-to-real (R2S2R) pipelines, where cyclic digital twins iteratively refine policies through alternating simulation and deployment. Methods such as RialTo, Real-is-Sim, and Residual Physics demonstrate that integrating simulation with calibrated real data improves robustness while reducing costly human intervention. In Section 4.2, we surveyed automated simulator configuration, spanning Bayesian optimization, discriminator-guided tuning, learned sampling distributions, and hybrid pipelines. These approaches replace manual heuristics with principled optimization and adaptive samplers, achieving measurable improvements in transfer fidelity.

Equally significant is the recognition that actuator and drivetrain fidelity (Section 4.3) is not a peripheral concern but a central determinant of transfer success. Simplified models of actuators can produce policies that exploit simulation artifacts, leading to catastrophic deployment failures. Recent work on differentiable actuator models, residual actuator correction, and explicit latency modeling illustrates that fidelity at the motor and drivetrain level is indispensable for predictive validity and controller stability. This perspective reframes the reality gap: it is not only about environmental physics but also about faithfully modeling the robot itself.

Another milestone is the emergence of a benchmarking culture (Section 4.4), which has long been missing in sim-to-real research. Initiatives such as the Real Robot Challenge, HomeRobot/OVMM, DexScale, and the BARN Challenge now provide shared platforms, cloud-access robots, and paired sim/real datasets, enabling reproducibility and fair comparison across laboratories. Methodological contributions recommend predictive validity metrics, robustness sweeps, and calibration error as essential complements to raw task success rates. Our proposed

evaluation checklist (Table 6) adds practical structure by outlining reporting standards for simulator setup, real-robot experiments, transfer metrics, reproducibility artifacts, and accessibility. Case studies like the NAO migration process (Fig. 12) illustrate how even small-scale testbeds can serve as anchors of reproducibility and transparency.

Despite these advances, several challenges persist. As outlined in Section 4, sim-to-real RL still lacks general theoretical guarantees, depends heavily on expert-in-the-loop design, and suffers from fragmented evaluation practices. Many algorithms remain sample-inefficient and computationally demanding, limiting scalability to complex real-world domains. Furthermore, task coverage in benchmarks is narrow, with limited representation of deformable objects, fluid interactions, multi-robot coordination, or safety-critical applications. Addressing these gaps will require both conceptual innovations (e.g., meta-RL, adversarial RL, and autonomous lifelong learning) and infrastructural innovations (e.g., open benchmarks, reproducibility checklists, and cloud-access digital twins).

The implications for robotics are far-reaching. A mature sim-to-real pipeline would enable robots to be trained primarily in simulation and deployed with minimal fine-tuning, dramatically lowering costs, reducing safety risks, and accelerating iteration cycles. Industrial settings could employ lifelong digital twins to continuously monitor robot health and adapt control policies over years of operation. Service and assistive robots could leverage adaptive pipelines to personalize behaviors to diverse households and user preferences. Autonomous vehicles and aerial drones could benefit from adversarially stress-tested policies that prepare them for rare but safety-critical corner cases not easily simulated exhaustively.

Looking forward, three priority themes emerge:

1. **Scalability and efficiency.** Meta-learning, adversarial RL, and flow-based samplers must be optimized to train across large simulator distributions without prohibitive data or compute costs.
2. **Openness and reproducibility.** Shared platforms, open datasets, and community-maintained digital twins are critical for ensuring cumulative rather than siloed progress.
3. **Safety and ethics.** As autonomous adaptation becomes more common, formal safety guarantees, uncertainty quantification, and regulatory frameworks will be essential for trustworthy deployment.

In conclusion, sim-to-real reinforcement learning is transitioning from an experimental frontier to a structured discipline. By unifying algorithmic advances (digital twins, hybrid calibration, meta-RL), fidelity modeling (actuators, drivetrains, delays), evaluation frameworks (benchmarks, predictive validity metrics), and open science practices, the field can move beyond ad hoc demonstrations toward scalable, reliable, and transparent deployment pipelines. The coming decade will determine whether sim-to-real RL achieves this integration. If successful, this approach will not only transform the way robots are trained but also accelerate their adoption in manufacturing, healthcare, autonomous mobility, and household assistance, ultimately bridging the gap between simulated intelligence and real-world embodied autonomy.

CRedit authorship contribution statement

Rajesh Tiwari: Writing – original draft, Investigation, Data curation. **Shailesh Khapre:** Writing – review & editing, Conceptualization. **Avantika Singh:** Writing – review & editing, Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

- [1] R.S. Sutton, A.G. Barto, *Reinforcement Learning: An Introduction*, A Bradford Book, Cambridge, MA, USA, 2018.
- [2] M. Li, W. Huang, Research and implementation of Chinese chess game algorithm based on reinforcement learning, in: 2020 5th International Conference on Control, Robotics and Cybernetics, CRC, 2020, pp. 81–86, <http://dx.doi.org/10.1109/CRC51253.2020.9253458>.
- [3] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T.P. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, D. Hassabis, Mastering the game of go without human knowledge, *Nature* 550 (2017) 354–359.
- [4] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis, A general reinforcement learning algorithm that masters chess, shogi, and go through self-play, *Science* 362 (6419) (2018) 1140–1144, <http://dx.doi.org/10.1126/science.aar6404>.
- [5] O. Vinyals, I. Babuschkin, W.M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J.P. Agapiou, M. Jaderberg, A.S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T.L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T.P. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, D. Silver, Grandmaster level in StarCraft II using multi-agent reinforcement learning, *Nature* 575 (2019) 350–354.
- [6] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse, R. Józefowicz, S. Gray, C. Olsson, J. Pachocki, M. Petrov, H.P.d. Pinto, J. Raiman, T. Salimans, J. Schlatter, J. Schneider, S. Sidor, I. Sutskever, J. Tang, F. Wolski, S. Zhang, Dota 2 with large scale deep reinforcement learning, 2019, CoRR [abs/1912.06680](https://arxiv.org/abs/1912.06680), [arXiv:1912.06680](https://arxiv.org/abs/1912.06680).
- [7] X. Chen, L. Yao, J.J. McAuley, G. Zhou, X. Wang, A survey of deep reinforcement learning in recommender systems: A systematic review and future directions, 2021, CoRR [abs/2109.03540](https://arxiv.org/abs/2109.03540), [arXiv:2109.03540](https://arxiv.org/abs/2109.03540).
- [8] L. Wang, W. Zhang, X. He, H. Zha, Supervised reinforcement learning with recurrent neural network for dynamic treatment recommendation, *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.* (2018).
- [9] X. Zhao, L. Xia, L. Zhang, Z. Ding, D. Yin, J. Tang, Deep reinforcement learning for page-wise recommendations, *Proc. 12th ACM Conf. Recomm. Syst.* (2018).
- [10] B. Osinski, A. Jakubowski, P. Milos, P. Ziecin, C. Galias, S. Homocanu, H. Michalewski, Simulation-based reinforcement learning for real-world autonomous driving, in: 2020 IEEE International Conference on Robotics and Automation, ICRA, 2019, pp. 6411–6418.
- [11] Y. You, X. Pan, Z. Wang, C. Lu, Virtual to real reinforcement learning for autonomous driving, 2017, ArXiv, [abs/1704.03952](https://arxiv.org/abs/1704.03952).
- [12] K.-S. Hwang, J.-L. Lin, K.-H. Yeh, Learning to adjust and refine gait patterns for a biped robot, *IEEE Trans. Syst. Man Cybern.: Syst.* 45 (2015) 1481–1490.
- [13] J. Garcia, D. Shafie, Teaching a humanoid robot to walk faster through safe reinforcement learning, *Eng. Appl. Artif. Intell.* 88 (2020).
- [14] Z. Ding, Y.-Y. Tsai, W.W. Lee, B. Huang, Sim-to-real transfer for robotic manipulation with tactile sensory, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2021, pp. 6778–6785.
- [15] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, S. Levine, QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation, 2018, ArXiv, [abs/1806.10293](https://arxiv.org/abs/1806.10293).
- [16] D. Quillen, E. Jang, O. Nachum, C. Finn, J. Ibarz, S. Levine, Deep reinforcement learning for vision-based robotic grasping: A simulated comparative evaluation of off-policy methods, in: 2018 IEEE International Conference on Robotics and Automation, ICRA, 2018, pp. 6284–6291.
- [17] A. Zeng, S. Song, S. Welker, J. Lee, A. Rodriguez, T.A. Funkhouser, Learning synergies between pushing and grasping with self-supervised deep reinforcement learning, in: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2018, pp. 4238–4245.
- [18] K. Lobos-Tsunekawa, F. Leiva, J.R. del Solar, Visual navigation for biped humanoid robots using deep reinforcement learning, *IEEE Robot. Autom. Lett.* 3 (2018) 3247–3254.
- [19] G. Kahn, A. Villafior, B. Ding, P. Abbeel, S. Levine, Self-supervised deep reinforcement learning with generalized computation graphs for robot navigation, in: 2018 IEEE International Conference on Robotics and Automation, ICRA, 2017, pp. 1–8.
- [20] C. Chen, Y. Liu, S. Kreiss, A. Alahi, Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning, in: 2019 International Conference on Robotics and Automation, ICRA, 2018, pp. 6015–6022.
- [21] M. Zhu, X. Wang, Y. Wang, Human-like autonomous car-following model with deep reinforcement learning, 2018, ArXiv, [abs/1901.00569](https://arxiv.org/abs/1901.00569).
- [22] M. Abreu, N. Lau, A.J. Sousa, L.P. Reis, Learning low level skills from scratch for humanoid robot soccer using deep reinforcement learning, in: 2019 IEEE International Conference on Autonomous Robot Systems and Competitions, ICARSC, 2019, pp. 1–8.
- [23] H. Shi, Z. Lin, S. Zhang, X. Li, K.-S. Hwang, An adaptive decision-making method with fuzzy Bayesian reinforcement learning for robot soccer, *Inf. Sci.* 436–437 (2018) 268–281.
- [24] E. Salvato, G. Fenu, E. Medvet, F.A. Pellegrino, Crossing the reality gap: a survey on sim-to-real transferability of robot controllers in reinforcement learning, *IEEE Access* PP (2021) 1–1.
- [25] A. Ghadirzadeh, A. Maki, D. Kragic, M. Björkman, Deep predictive policy training using reinforcement learning, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2017, pp. 2351–2358.
- [26] H. Hu, K. Zhang, A.H. Tan, M. Ruan, C. Agia, G. Nejat, A sim-to-real pipeline for deep reinforcement learning for autonomous robot navigation in cluttered rough terrain, *IEEE Robot. Autom. Lett.* 6 (2021) 6569–6576.
- [27] G. Reddy, J. Wong-Ng, A. Celani, T.J. Sejnowski, M. Vergassola, Glider soaring via reinforcement learning in the field, *Nature* 562 (2018) 236–239.
- [28] W. Zhu, X. Guo, D. Owaki, K. Kutsuzawa, M. Hayashibe, A survey of sim-to-real transfer techniques applied to reinforcement learning for bioinspired robots, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (2021) 3444–3459.
- [29] K. Dimitropoulos, I. Hatzilygeroudis, K. Chatzilygeroudis, A brief survey of Sim2Real methods for robot learning, in: A. Müller, M. Brandstötter (Eds.), *Advances in Service and Industrial Robotics*, Springer International Publishing, Cham, 2022, pp. 133–140.
- [30] W. Zhao, J.P. Queralta, T. Westerlund, Sim-to-real transfer in deep reinforcement learning for robotics: a survey, in: 2020 IEEE Symposium Series on Computational Intelligence, SSCI, 2020, pp. 737–744.
- [31] T. Wu, J.R. Movellan, Semi-parametric Gaussian process for robot system identification, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012, pp. 725–731.
- [32] J.v. Baar, A. Sullivan, R. Cordorel, D. Jha, D. Romeres, D. Nikovski, Sim-to-real transfer learning using robustified controllers in robotic tasks involving complex dynamics, in: 2019 International Conference on Robotics and Automation, ICRA, 2019, pp. 6001–6007, <http://dx.doi.org/10.1109/ICRA.2019.8793561>.
- [33] F. Ye, S. Zhang, P. Wang, C. yao Chan, A survey of deep reinforcement learning algorithms for motion planning and control of autonomous vehicles, in: 2021 IEEE Intelligent Vehicles Symposium, IV, 2021, pp. 1073–1080.
- [34] Z. Li, X. Cheng, X.B. Peng, P. Abbeel, S. Levine, G. Berseth, K. Sreenath, Reinforcement learning for robust parameterized locomotion control of bipedal robots, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2021, pp. 2811–2817.
- [35] R. Alghonaim, E. Johns, Benchmarking domain randomisation for visual sim-to-real transfer, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2020, pp. 12802–12808.

- [36] J. Siekmann, K.R. Green, J. Warila, A. Fern, J.W. Hurst, Blind bipedal stair traversal via sim-to-real reinforcement learning, 2021, ArXiv, [abs/2105.08328](https://arxiv.org/abs/2105.08328).
- [37] P. Anderson, A. Shrivastava, J. Truong, A. Majumdar, D. Parikh, D. Batra, S. Lee, Sim-to-real transfer for vision-and-language navigation, 2020, CoRR, [abs/2011.03807](https://arxiv.org/abs/2011.03807). [arXiv:2011.03807](https://arxiv.org/abs/2011.03807).
- [38] B. Fernando, A. Habrard, M. Sebban, T. Tuytelaars, Unsupervised visual domain adaptation using subspace alignment, in: 2013 IEEE International Conference on Computer Vision, 2013, pp. 2960–2967.
- [39] J. Hoffman, D. Wang, F. Yu, T. Darrell, FCNs in the wild: Pixel-level adversarial and constraint-based adaptation, 2016, ArXiv, [abs/1612.02649](https://arxiv.org/abs/1612.02649).
- [40] D. Zhang, A. Barbot, F. Seichepine, F.P.-W. Lo, W. Bai, G.-Z. Yang, B.P.L. Lo, Micro-object pose estimation with sim-to-real transfer learning using small dataset, *Commun. Phys.* 5 (2022).
- [41] M. Cutler, T.J. Walsh, J.P. How, Real-world reinforcement learning via multifidelity simulators, *IEEE Trans. Robot.* 31 (2015) 655–671.
- [42] M. Cutler, J.P. How, Autonomous drifting using simulation-aided reinforcement learning, in: 2016 IEEE International Conference on Robotics and Automation, ICRA, 2016, pp. 5442–5448.
- [43] S.D. Shashua, D. Di Castro, S. Mannor, Sim and real: Better together, in: M. Ranzato, A. Beygelzimer, Y.N. Dauphin, P. Liang, J.W. Vaughan (Eds.), *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6–14, 2021, Virtual*, 2021, pp. 6868–6880.
- [44] H. nan Wang, N. Liu, Y. Zhang, D. Feng, F. Huang, D. Li, Y. Zhang, Deep reinforcement learning: a survey, *Front. Inf. Technol. Electron. Eng.* 21 (2020) 1726–1744.
- [45] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M.A. Riedmiller, Playing atari with deep reinforcement learning, 2013, ArXiv, [abs/1312.5602](https://arxiv.org/abs/1312.5602).
- [46] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M.A. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, *Nature* 518 (2015) 529–533.
- [47] R. Chattopadhyay, Q. Sun, W. Fan, I. Davidson, S. Panchanathan, J. Ye, Multisource domain adaptation and its application to early detection of fatigue, *ACM Trans. Knowl. Discov. Data* 6 (2012) 18:1–18:26.
- [48] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, N. de Freitas, Dueling network architectures for deep reinforcement learning, in: M. Balcan, K.Q. Weinberger (Eds.), *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19–24, 2016, in: JMLR Workshop and Conference Proceedings, vol. 48, JMLR.org*, 2016, pp. 1995–1997.
- [49] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N.M.O. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2015, CoRR, [abs/1509.02971](https://arxiv.org/abs/1509.02971).
- [50] S. Fujimoto, H. van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: J.G. Dy, A. Krause (Eds.), *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10–15, 2018, in: Proceedings of Machine Learning Research, vol. 80, PMLR*, 2018, pp. 1582–1591.
- [51] J. Schulman, S. Levine, P. Abbeel, M.I. Jordan, P. Moritz, Trust region policy optimization, in: F.R. Bach, D.M. Blei (Eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6–11 July 2015, in: JMLR Workshop and Conference Proceedings, vol. 37, JMLR.org*, 2015, pp. 1889–1897.
- [52] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, ArXiv, [abs/1707.06347](https://arxiv.org/abs/1707.06347).
- [53] R.J. Williams, Simple statistical gradient-following algorithms for connectionist reinforcement learning, *Mach. Learn.* 8 (1992) 229–256, [http://dx.doi.org/10.1007/BF00992696](https://doi.org/10.1007/BF00992696).
- [54] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018, ArXiv, [abs/1801.01290](https://arxiv.org/abs/1801.01290).
- [55] M.P. Deisenroth, C.E. Rasmussen, PILCO: a model-based and data-efficient approach to policy search, in: L. Getoor, T. Scheffer (Eds.), *Proceedings of the 28th International Conference on Machine Learning, ICML 2011, Bellevue, Washington, USA, June 28 – July 2, 2011, Omni Press*, 2011, pp. 465–472.
- [56] F. Amadio, A. Dalla Libera, R. Antonello, D. Nikovski, R. Carli, D. Romeres, Model-based policy search using Monte Carlo gradient estimation with real systems application, *IEEE Trans. Robot.* 38 (6) (2022) 3879–3898.
- [57] M. Janner, J. Fu, M. Zhang, S. Levine, When to trust your model: Model-based policy optimization, in: H.M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E.B. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada*, 2019, pp. 12498–12509.
- [58] D. Hafner, T.P. Lillicrap, M. Norouzi, J. Ba, Mastering atari with discrete world models, in: 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3–7, 2021, OpenReview.net, 2021, p. 1.
- [59] H. Mania, A. Guy, B. Recht, Simple random search of static linear policies is competitive for reinforcement learning, in: S. Bengio, H.M. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3–8, 2018, Montréal, Canada*, 2018, pp. 1805–1814.
- [60] E.J. Such, V. Madhavan, J.R. Pleš, M.A. DeMers, P. Koch, J.E. Walker, J. Clune, K.O. Stanley, Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning, in: *Proceedings of the 2017 Genetic and Evolutionary Computation Conference, GECCO*, 2017, pp. 1–12.
- [61] T. Salimans, J. Ho, X. Chen, S. Sidor, I. Sutskever, Evolution strategies as a scalable alternative to reinforcement learning, 2017, ArXiv Preprint [arXiv:1703.03864](https://arxiv.org/abs/1703.03864).
- [62] S.J. Pan, Q. Yang, A survey on transfer learning, *IEEE Trans. Knowl. Data Eng.* 22 (2010) 1345–1359.
- [63] J. Huang, A.J. Smola, A. Gretton, K.M. Borgwardt, B. Schölkopf, Correcting sample selection bias by unlabeled data, in: B. Schölkopf, J.C. Platt, T. Hofmann (Eds.), *Advances in Neural Information Processing Systems 19, Proceedings of the Twentieth Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 4–7, 2006, MIT Press*, 2006, pp. 601–608.
- [64] X. Liao, Y. Xue, L. Carin, Logistic regression with an auxiliary data source, in: L.D. Raedt, S. Wrobel (Eds.), *Machine Learning, Proceedings of the Twenty-Second International Conference (ICML 2005), Bonn, Germany, August 7–11, 2005, in: ACM International Conference Proceeding Series, vol. 119, ACM*, 2005, pp. 505–512, [http://dx.doi.org/10.1145/1102351.1102415](https://doi.org/10.1145/1102351.1102415).
- [65] H.D. III, Frustratingly easy domain adaptation, in: J. Carroll, A. van den Bosch, A. Zaenen (Eds.), *ACL 2007, Proceedings of the 45th Annual Meeting of the Association for Computational Linguistics, June 23–30, 2007, Prague, Czech Republic, The Association for Computational Linguistics*, 2007, pp. 256–263.
- [66] W. Dai, G. Xue, Q. Yang, Y. Yu, Co-clustering based classification for out-of-domain documents, in: P. Berkhin, R. Caruana, X. Wu (Eds.), *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Jose, California, USA, August 12–15, 2007, ACM*, 2007, pp. 210–219, [http://dx.doi.org/10.1145/1281192.1281218](https://doi.org/10.1145/1281192.1281218).
- [67] F. Zhuang, P. Luo, H. Xiong, Q. He, Y. Xiong, Z. Shi, Exploiting associations between word clusters and document classes for cross-domain text categorization †, *Stat. Anal. Data Min.: ASA Data Sci. J.* 4 (2011).
- [68] F. Li, S.J. Pan, O. Jin, Q. Yang, X. Zhu, Cross-domain co-extraction of sentiment and topic lexicons, in: *The 50th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, July 8–14, 2012, Jeju Island, Korea - Volume 1: Long Papers, The Association for Computer Linguistics*, 2012, pp. 410–419.
- [69] L. Mihalkova, T.N. Huynh, R.J. Mooney, Mapping and revising Markov logic networks for transfer learning, in: *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence, July 22–26, 2007, Vancouver, British Columbia, Canada, AAAI Press*, 2007, pp. 608–614.
- [70] A. Lazaric, Transfer in reinforcement learning: A framework and a survey, in: M.A. Wiering, M. van Otterlo (Eds.), *Reinforcement Learning, in: Adaptation, Learning, and Optimization, vol. 12, Springer*, 2012, pp. 143–173, [http://dx.doi.org/10.1007/978-3-642-27645-3_5](https://doi.org/10.1007/978-3-642-27645-3_5).
- [71] M.E. Taylor, P. Stone, Transfer learning for reinforcement learning domains: A survey, *J. Mach. Learn. Res.* 10 (2009) 1633–1685, [http://dx.doi.org/10.5555/1577069.1755839](https://doi.org/10.5555/1577069.1755839).
- [72] Z. Zhu, K. Lin, A.K. Jain, J. Zhou, Transfer learning in deep reinforcement learning: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (11) (2023) 13344–13362, [http://dx.doi.org/10.1109/TPAMI.2023.3292075](https://doi.org/10.1109/TPAMI.2023.3292075).
- [73] T. Erez, W.D. Smart, What does shaping mean for computational reinforcement learning? in: 2008 7th IEEE International Conference on Development and Learning, 2008, pp. 215–219, [http://dx.doi.org/10.1109/DEVLRN.2008.4640832](https://doi.org/10.1109/DEVLRN.2008.4640832).
- [74] O. Marom, B. Rosman, Belief reward shaping in reinforcement learning, in: S.A. McIlraith, K.Q. Weinberger (Eds.), *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th Innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2–7, 2018, AAAI Press*, 2018, pp. 3762–3769, [http://dx.doi.org/10.1609/AAAI.V32i1.11741](https://doi.org/10.1609/AAAI.V32i1.11741).
- [75] Z. Zhu, K. Lin, B. Dai, J. Zhou, Learning sparse rewarded tasks from sub-optimal demonstrations, 2020, CoRR, [abs/2004.00530](https://arxiv.org/abs/2004.00530). [arXiv:2004.00530](https://arxiv.org/abs/2004.00530). URL <https://arxiv.org/abs/2004.00530>.
- [76] J. Ho, S. Ermon, Generative adversarial imitation learning, in: D.D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 29: Annual Conference on Neural Information Processing Systems 2016, December 5–10, 2016, Barcelona, Spain*, 2016, pp. 4565–4573.
- [77] F. Fernández, M.M. Veloso, Probabilistic policy reuse in a reinforcement learning agent, in: H. Nakashima, M.P. Wellman, G. Weiss, P. Stone (Eds.), *5th International Joint Conference on Autonomous Agents and Multiagent Systems*

- (AAMAS 2006), Hakodate, Japan, May 8-12, 2006, ACM, 2006, pp. 720–727, <http://dx.doi.org/10.1145/1160633.1160762>.
- [78] W.M. Czarnecki, R. Pascanu, S. Osindero, S.M. Jayakumar, G. Swirszcz, M. Jaderberg, Distilling policy distillation, 2019, CoRR, [abs/1902.02186](https://arxiv.org/abs/1902.02186). arXiv:1902.02186.
- [79] A. Gupta, C. Devin, Y. Liu, P. Abbeel, S. Levine, Learning invariant feature spaces to transfer skills with reinforcement learning, 2017, CoRR, [abs/1703.02949](https://arxiv.org/abs/1703.02949). arXiv:1703.02949.
- [80] C. Devin, A. Gupta, T. Darrell, P. Abbeel, S. Levine, Learning modular neural network policies for multi-task and multi-robot transfer, in: 2017 IEEE International Conference on Robotics and Automation, ICRA 2017, Singapore, Singapore, May 29 - June 3, 2017, IEEE, 2017, pp. 2169–2176, <http://dx.doi.org/10.1109/ICRA.2017.7989250>.
- [81] A.A. Rusu, N.C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, R. Hadsell, Progressive neural networks, 2016, ArXiv, [abs/1606.04671](https://arxiv.org/abs/1606.04671).
- [82] N.P. Koenig, A. Howard, Design and use paradigms for gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems, Sendai, Japan, September 28 - October 2, 2004, IEEE, 2004, pp. 2149–2154, <http://dx.doi.org/10.1109/IROS.2004.1389727>.
- [83] O. Michel, Cyberbotics Ltd. Webots™: Professional mobile robot simulation, Int. J. Adv. Robot. Syst. 1 (2004).
- [84] E. Rohmer, S.P.N. Singh, M. Freese, V-REP: A versatile and scalable robot simulation framework, in: 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2013, pp. 1321–1326.
- [85] S. James, M. Freese, A.J. Davison, PyRep: Bringing V-REP to deep robot learning, 2019, CoRR, [abs/1906.11176](https://arxiv.org/abs/1906.11176). arXiv:1906.11176. URL <http://arxiv.org/abs/1906.11176>.
- [86] B. Ellenberger, Pybullet gymperium, 2018–2019, <https://github.com/benelot/pybullet-gym>.
- [87] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, Openai gym, 2016, ArXiv, [abs/1606.01540](https://arxiv.org/abs/1606.01540).
- [88] J. Hwangbo, J. Lee, M. Hutter, Per-contact iteration method for solving contact dynamics, IEEE Robot. Autom. Lett. 3 (2) (2018) 895–902, URL www.raisim.com.
- [89] M. Ahn, H. Zhu, K. Hartikainen, H. Ponte, A. Gupta, S. Levine, V. Kumar, ROBEL: robotics benchmarks for learning with low-cost robots, in: L.P. Kaelbling, D. Kragic, K. Sugiura (Eds.), 3rd Annual Conference on Robot Learning, CoRL 2019, Osaka, Japan, October 30 - November 1, 2019, Proceedings, in: Proceedings of Machine Learning Research, vol. 100, PMLR, 2019, pp. 1300–1313.
- [90] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D.d. Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, P.T. Lillicrap, M.A. Riedmiller, DeepMind control suite, 2018, ArXiv, [abs/1801.00690](https://arxiv.org/abs/1801.00690).
- [91] L. Fan, Y. Zhu, J. Zhu, Z. Liu, O. Zeng, A. Gupta, J. Creus-Costa, S. Savarese, L. Fei-Fei, SURREAL: open-source reinforcement learning framework and robot manipulation benchmark, in: 2nd Annual Conference on Robot Learning, CoRL 2018, Zürich, Switzerland, 29-31 October 2018, Proceedings, in: Proceedings of Machine Learning Research, vol. 87, PMLR, 2018, pp. 767–782.
- [92] B. Ellenberger, Pybullet gymperium, 2018–2019, <https://github.com/benelot/pybullet-gym>.
- [93] F. Xia, A. Zamir, Z.-Y. He, A. Sax, J. Malik, S. Savarese, Gibson env: Real-world perception for embodied agents, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 9068–9079.
- [94] B. Delhaisse, L.D. Roza, D.G. Caldwell, PyRoboLearn: A python framework for robot learning practitioners, in: L.P. Kaelbling, D. Kragic, K. Sugiura (Eds.), 3rd Annual Conference on Robot Learning, CoRL 2019, Osaka, Japan, October 30 - November 1, 2019, Proceedings, in: Proceedings of Machine Learning Research, vol. 100, PMLR, 2019, pp. 1348–1358.
- [95] S. Shah, D. Dey, C. Lovett, A. Kapoor, AirSim: High-fidelity visual and physical simulation for autonomous vehicles, in: M. Hutter, R. Siegwart (Eds.), Field and Service Robotics, Results of the 11th International Conference, FSR 2017, Zurich, Switzerland, 12-15 September 2017, in: Springer Proceedings in Advanced Robotics, vol. 5, Springer, 2017, pp. 621–635, http://dx.doi.org/10.1007/978-3-319-67361-5_40.
- [96] A. Dosovitskiy, G. Ros, F. Codevilla, A.M. López, V. Koltun, CARLA: an open urban driving simulator, in: 1st Annual Conference on Robot Learning, CoRL 2017, Mountain View, California, USA, November 13-15, 2017, Proceedings, in: Proceedings of Machine Learning Research, vol. 78, PMLR, 2017, pp. 1–16.
- [97] C.D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, O. Bachem, Brax - a differentiable physics engine for large scale rigid body simulation, in: J. Vanschoren, S. Yeung (Eds.), Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, Virtual, 2021, pp. 1–13.
- [98] Y. Hu, L. Anderson, T. Li, Q. Sun, N. Carr, J. Ragan-Kelley, F. Durand, Diff-Taichi: Differentiable programming for physical simulation, in: 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, OpenReview.net, 2020, pp. 234–456.
- [99] C. Zhou, L. Han, Y. Mao, G. Ye, Q. Lin, W. Ding, H. Liu, Z. Wang, Z. Zhang, JBDL: A JAX-based body dynamics algorithm library for robotics., 2021, ArXiv.
- [100] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J.L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, Orbit: A unified simulation framework for interactive robot learning environments, IEEE Robot. Autom. Lett. 8 (6) (2023) 3740–3747, <http://dx.doi.org/10.1109/LRA.2023.3270034>.
- [101] E. Todorov, T. Erez, Y. Tassa, Mujoco: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2012, pp. 5026–5033, <http://dx.doi.org/10.1109/IROS.2012.6386109>.
- [102] J. Boedecker, M. Asada, SimSpark-concepts and application in the RoboCup 3D soccer simulation league, Auton. Robots (2008) 174–181.
- [103] M. Hutter, C. Gehring, D. Jud, A. Lauber, D. Bellicoso, V. Tsounis, J. Hwangbo, K. Bodie, P. Fankhauser, M. Bloesch, R. Diethelm, S. Bachmann, A. Melzer, M.A. Höpflinger, ANYmal - a highly mobile and dynamic quadrupedal robot, in: 2016 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2016, pp. 38–44.
- [104] W.C. Martin, A. Wu, H. Geyer, Robust spring mass model running for a physical bipedal robot, in: 2015 IEEE International Conference on Robotics and Automation, ICRA, 2015, pp. 6307–6312.
- [105] A. Church, J. Lloyd, R. Hadsell, N.F. Lepora, Tactile sim-to-real policy transfer via real-to-sim image translation, in: A. Faust, D. Hsu, G. Neumann (Eds.), Conference on Robot Learning, 8-11 November 2021, London, UK, in: Proceedings of Machine Learning Research, vol. 164, PMLR, 2021, pp. 1645–1654.
- [106] Y. Lin, J. Lloyd, A. Church, N.F. Lepora, Tactile gym 2.0: Sim-to-real deep reinforcement learning for comparing low-cost high-resolution robot touch, IEEE Robot. Autom. Lett. 7 (2022) 10754–10761.
- [107] W. Yuan, S. Dong, E.H. Adelson, GelSight: High-resolution robot tactile sensors for estimating geometry and force, Sensors (Basel, Switzerland) 17 (2017).
- [108] J. Bradbury, R. Frostig, P. Hawkins, M.J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, Q. Zhang, JAX: composable transformations of Python+NumPy programs, 2018, URL <http://github.com/google/jax>.
- [109] S. Höfer, K. Bekris, A. Handa, J.C. Gamboa, M. Mozifian, F. Golemo, C. Atkeson, D. Fox, K. Goldberg, J. Leonard, C. Karen Liu, J. Peters, S. Song, P. Welinder, M. White, Sim2Real in robotics and automation: Applications and challenges, IEEE Trans. Autom. Sci. Eng. 18 (2) (2021) 398–400, <http://dx.doi.org/10.1109/TASE.2021.3064065>.
- [110] Y. Chebotar, A. Handa, V. Makoviyshuk, M. Macklin, J. Issac, N.D. Ratliff, D. Fox, Closing the sim-to-real loop: Adapting simulation randomization with real world experience, in: 2019 International Conference on Robotics and Automation, ICRA, 2018, pp. 8973–8979, URL <https://api.semanticscholar.org/CorpusID:53046511>.
- [111] E. Valassakis, Z. Ding, E. Johns, Crossing the gap: A deep dive into zero-shot sim-to-real transfer for dynamics, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2020, pp. 5372–5379, URL <https://api.semanticscholar.org/CorpusID:221140175>.
- [112] Y. Yao, L. Xiao, Z. An, W. Zhang, D. Luo, Sample efficient reinforcement learning via model-ensemble exploration and exploitation, in: IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021, IEEE, 2021, pp. 4202–4208, <http://dx.doi.org/10.1109/ICRA48506.2021.9561842>.
- [113] S. James, P. Wohlhart, M. Kalakrishnan, D. Kalashnikov, A. Irpan, J. Ibarz, S. Levine, R. Hadsell, K. Bousmalis, Sim-To-Real via Sim-To-Sim: Data-Efficient Robotic Grasping via Randomized-To-Canonical Adaptation Networks , in: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, IEEE Computer Society, Los Alamitos, CA, USA, 2019, pp. 12619–12629, <http://dx.doi.org/10.1109/CVPR.2019.01291>.
- [114] T.M. Moldovan, P. Abbeel, Safe exploration in Markov decision processes, 2012, ArXiv, [abs/1205.4810](https://arxiv.org/abs/1205.4810). URL <https://api.semanticscholar.org/CorpusID:9619086>.
- [115] T. Mannucci, E.-J. van Kampen, C. de Visser, Q. Chu, Safe exploration algorithms for reinforcement learning controllers, IEEE Trans. Neural Netw. Learn. Syst. 29 (2018) 1069–1081, URL <https://api.semanticscholar.org/CorpusID:3965928>.
- [116] A. Gupta, R. Mendonca, Y. Liu, P. Abbeel, S. Levine, Meta-reinforcement learning of structured exploration strategies, in: S. Bengio, H.M. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada, 2018, pp. 5307–5316.
- [117] A. Nair, B. McGrew, M. Andrychowicz, W. Zaremba, P. Abbeel, Overcoming exploration in reinforcement learning with demonstrations, in: 2018 IEEE International Conference on Robotics and Automation, ICRA, 2017, pp. 6292–6299.
- [118] K. Kristinsson, G.A. Dumont, System identification and control using genetic algorithms, IEEE Trans. Syst. Man Cybern. 22 (1992) 1033–1046.
- [119] R. Johansson, A. Robertsson, K. Nilsson, M. Verhaegen, State-space system identification of robot manipulator dynamics, Mechatronics 10 (2000) 403–418.
- [120] O. Akanyeti, U. Nehmzow, S.A. Billings, Robot training using system identification, Robot. Auton. Syst. 56 (2008) 1027–1041.
- [121] R.D. Al-Dabbagh, A. Kinsheel, S. Mekhilef, M.S.B. Baba, S. Shamsheerband, System identification and control of robot manipulator based on fuzzy adaptive differential evolution algorithm, Adv. Eng. Softw. 78 (2014) 60–66.

- [122] A. Allevato, E.S. Short, M.W. Pryor, A.L. Thomaz, TuneNet: One-shot residual tuning for system identification and sim-to-real robot task transfer, 2019, ArXiv, abs/1907.11200.
- [123] Y. Du, O. Watkins, T. Darrell, P. Abbeel, D. Pathak, Auto-tuned sim-to-real transfer, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2021, pp. 1290–1296.
- [124] R. Gao, Y.-Y. Chang, S. Mall, L. Fei-Fei, J. Wu, ObjectFolder: A dataset of objects with implicit visual, auditory, and tactile representations, 2021, ArXiv, abs/2109.07991.
- [125] R. Gao, Z. Si, Y.-Y. Chang, S. Clarke, J. Bohg, L. Fei-Fei, W. Yuan, J. Wu, ObjectFolder 2.0: A multisensory object dataset for Sim2Real transfer, in: 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2022, pp. 10588–10598.
- [126] A. Kumar, Z. Fu, D. Pathak, J. Malik, RMA: Rapid motor adaptation for legged robots, 2021, ArXiv, abs/2107.04034.
- [127] A. Kumar, Z. Li, J. Zeng, D. Pathak, K. Sreenath, J. Malik, Adapting rapid motor adaptation for bipedal robots, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2022, pp. 1161–1168.
- [128] H. Qi, A. Kumar, R. Calandra, Y. Ma, J. Malik, In-hand object rotation via rapid motor adaptation, in: K. Liu, D. Kulic, J. Ichnowski (Eds.), Conference on Robot Learning, CoRL 2022, 14–18 December 2022, Auckland, New Zealand, in: Proceedings of Machine Learning Research, vol. 205, PMLR, 2022, pp. 1722–1732.
- [129] D. Zhang, A. Loquercio, X. Wu, A. Kumar, J. Malik, M.W. Mueller, Learning a single near-hover position controller for vastly different quadcopters, in: 2023 IEEE International Conference on Robotics and Automation, ICRA, 2022, pp. 1263–1269.
- [130] M.S. Ruderman, F. Hoffmann, T. Bertram, Modeling and identification of elastic robot joints with hysteresis and backlash, IEEE Trans. Ind. Electron. 56 (2009) 3840–3847.
- [131] H. von Park, K. Sreenath, J.W. Hurst, J.W. Grizzle, Identification of a bipedal robot with a compliant drivetrain, IEEE Control. Syst. 31 (2011) 63–88.
- [132] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2017, pp. 23–30.
- [133] F. Muratore, C. Eilers, M. Gienger, J. Peters, Data-efficient domain randomization with Bayesian optimization, IEEE Robot. Autom. Lett. 6 (2020) 911–918.
- [134] X.B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, Sim-to-real transfer of robotic control with dynamics randomization, in: 2018 IEEE International Conference on Robotics and Automation, ICRA, 2017, pp. 1–8.
- [135] E. Valassakis, N. Di Palo, E. Johns, Coarse-to-fine for sim-to-real: Sub-millimetre precision across wide task spaces, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2021, pp. 5989–5996.
- [136] X. Ren, J. Luo, E. Solowjow, J.A. Ojeda, A. Gupta, A. Tamar, P. Abbeel, Domain randomization for active pose estimation, in: 2019 International Conference on Robotics and Automation, ICRA, 2019, pp. 7228–7234.
- [137] F. Muratore, F. Treede, M. Gienger, J. Peters, Domain randomization for simulation-based policy optimization with transferability assessment, in: 2nd Annual Conference on Robot Learning, CoRL 2018, Zürich, Switzerland, 29–31 October 2018, Proceedings, in: Proceedings of Machine Learning Research, vol. 87, PMLR, 2018, pp. 700–713, URL <http://proceedings.mlr.press/v87/muratore18a.html>.
- [138] F. Muratore, T. Gruner, F. Wiese, B. Belousov, M. Gienger, J. Peters, Neural posterior domain randomization, in: A. Faust, D. Hsu, G. Neumann (Eds.), Conference on Robot Learning, 8–11 November 2021, London, UK, in: Proceedings of Machine Learning Research, vol. 164, PMLR, 2021, pp. 1532–1542.
- [139] Y. Bengio, J. Louradour, R. Collobert, J. Weston, Curriculum learning, in: A.P. Danyluk, L. Bottou, M.L. Littman (Eds.), Proceedings of the 26th Annual International Conference on Machine Learning, ICML 2009, Montreal, Quebec, Canada, June 14–18, 2009, in: ACM International Conference Proceeding Series, vol. 382, ACM, 2009, pp. 41–48, <http://dx.doi.org/10.1145/1553374.1553380>.
- [140] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M.E. Taylor, P. Stone, Curriculum learning for reinforcement learning domains: A framework and survey, J. Mach. Learn. Res. 21 (2020) 181:1–181:50.
- [141] S. Narvekar, P. Stone, Learning curriculum policies for reinforcement learning, in: E. Elkind, M. Veloso, N. Agmon, M.E. Taylor (Eds.), Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS '19, Montreal, QC, Canada, May 13–17, 2019, International Foundation for Autonomous Agents and Multiagent Systems, 2019, pp. 25–33.
- [142] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, P. Abbeel, Reverse curriculum generation for reinforcement learning, in: 1st Annual Conference on Robot Learning, CoRL 2017, Mountain View, California, USA, November 13–15, 2017, Proceedings, in: Proceedings of Machine Learning Research, vol. 78, PMLR, 2017, pp. 482–495, URL <http://proceedings.mlr.press/v78/florensa17a.html>.
- [143] S.M. Nguyen, N. Duminy, A. Manoury, D. Duhaut, C. Buche, Robots learn increasingly complex tasks with intrinsic motivation and automatic curriculum learning, KI - Künstliche Intell. 35 (2021) 81–90.
- [144] S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, R. Fergus, Intrinsic motivation and automatic curricula via asymmetric self-play, in: 6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings, OpenReview.net, 2018, pp. 1–16.
- [145] Z. He, C. Gu, R. Xu, K. Wu, Automatic curriculum generation by hierarchical reinforcement learning, in: H. Yang, K. Pasupa, A.C. Leung, J.T. Kwok, J.H. Chan, I. King (Eds.), Neural Information Processing - 27th International Conference, ICONIP 2020, Bangkok, Thailand, November 23–27, 2020, Proceedings, Part II, in: Lecture Notes in Computer Science, vol. 12533, Springer, 2020, pp. 202–213.
- [146] Y. Shukla, C. Thierauf, R. Hosseini, G. Tatiya, J. Sinapov, Acute: Automatic curriculum transfer from simple to complex environments, in: P. Faliszewski, V. Mascardi, C. Pelachaud, M.E. Taylor (Eds.), 21st International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2022, Auckland, New Zealand, May 9–13, 2022, International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), 2022, pp. 1192–1200, <http://dx.doi.org/10.5555/3535850.3535983>.
- [147] T. Matisen, A. Oliver, T. Cohen, J. Schulman, Teacher–student curriculum learning, IEEE Trans. Neural Netw. Learn. Syst. 31 (2017) 3732–3740.
- [148] M.P. Deisenroth, C.E. Rasmussen, PILCO: a model-based and data-efficient approach to policy search, in: L. Getoor, T. Scheffer (Eds.), Proceedings of the 28th International Conference on Machine Learning, ICML 2011, Bellevue, Washington, USA, June 28 - July 2, 2011, Omni Press, 2011, pp. 465–472.
- [149] J. Truong, M. Rudolph, N. Yokoyama, S. Chernova, D. Batra, A. Rai, Rethinking Sim2Real: Lower fidelity simulation leads to higher Sim2Real transfer in navigation, in: K. Liu, D. Kulic, J. Ichnowski (Eds.), Conference on Robot Learning, CoRL 2022, 14–18 December 2022, Auckland, New Zealand, in: Proceedings of Machine Learning Research, vol. 205, PMLR, 2022, pp. 859–870.
- [150] X. Wang, Y. Chen, W. Zhu, A survey on curriculum learning, IEEE Trans. Pattern Anal. Mach. Intell. 44 (2021) 4555–4576.
- [151] Y. Zhou, H. Chen, Z. Pan, C. Yan, F. Lin, X. Wang, W. Zhu, Curml: A curriculum machine learning library, in: J. ao Magalhães, A.D. Bimbo, S. Satoh, N. Sebe, X. Alameda-Pineda, Q. Jin, V. Oria, L. Toni (Eds.), MM '22: The 30th ACM International Conference on Multimedia, Lisboa, Portugal, October 10 - 14, 2022, ACM, 2022, pp. 7359–7363, <http://dx.doi.org/10.1145/3503161.3548549>.
- [152] C. Cortes, M. Mohri, Domain adaptation and sample bias correction theory and algorithm for regression, Theoret. Comput. Sci. 519 (2014) 103–126, <http://dx.doi.org/10.1016/J.TCS.2013.09.027>.
- [153] N. Lu, T. Zhang, T. Fang, T. Teshima, M. Sugiyama, Rethinking importance weighting for transfer learning, in: Federated and Transfer Learning, in: Adaptation, Learning, and Optimization, vol. 27, Springer International Publishing AG, Switzerland, 2022, pp. 185–231.
- [154] A. Lazaric, M. Restelli, A. Bonarini, Transfer of samples in batch reinforcement learning, in: W.W. Cohen, A. McCallum, S.T. Roweis (Eds.), Machine Learning, Proceedings of the Twenty-Fifth International Conference (ICML 2008), Helsinki, Finland, June 5–9, 2008, in: ACM International Conference Proceeding Series, vol. 307, ACM, 2008, pp. 544–551, <http://dx.doi.org/10.1145/1390156.1390225>.
- [155] A. Tirinzoni, A. Sessa, M. Pirota, M. Restelli, Importance weighted transfer of samples in reinforcement learning, in: J.G. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10–15, 2018, in: Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 4943–4952.
- [156] D. Pomerleau, ALVINN: an autonomous land vehicle in a neural network, in: D.S. Touretzky (Ed.), Advances in Neural Information Processing Systems 1, [NIPS Conference, Denver, Colorado, USA, 1988], Morgan Kaufmann, 1988, pp. 305–313.
- [157] S. Ross, G.J. Gordon, D. Bagnell, A reduction of imitation learning and structured prediction to no-regret online learning, in: G.J. Gordon, D.B. Dunson, M. Dudík (Eds.), Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, AISTATS 2011, Fort Lauderdale, USA, April 11–13, 2011, in: JMLR Proceedings, vol. 15, JMLR.org, 2011, pp. 627–635.
- [158] A.Y. Ng, S. Russell, Algorithms for inverse reinforcement learning, in: P. Langley (Ed.), Proceedings of the Seventeenth International Conference on Machine Learning (ICML 2000), Stanford University, Stanford, CA, USA, June 29 - July 2, 2000, Morgan Kaufmann, 2000, pp. 663–670.
- [159] M. Yan, I. Prossio, S. Tyree, J. Kautz, Sim-to-real transfer of accurate grasping with eye-in-hand observations and continuous control, 2017, CoRR, abs/1712.03303. arXiv:1712.03303.
- [160] P.F. Christiano, Z. Shah, I. Mordatch, J. Schneider, T. Blackwell, J. Tobin, P. Abbeel, W. Zaremba, Transfer from simulation to real world through learning deep inverse dynamics model, 2016, CoRR, abs/1610.03518. arXiv:1610.03518.
- [161] R. Rahmatizadeh, P. Abolghasemi, A. Behal, L. Bölöni, From virtual demonstration to real-world manipulation using LSTM and MDN, in: S.A. McIlraith, K.Q. Weinberger (Eds.), Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th Innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances

- in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018, AAAI Press, 2018, pp. 6524–6531, <http://dx.doi.org/10.1609/AAAI.V32i1.12099>.
- [162] R. Kaushik, K. Arndt, V. Kyrki, SafeAPT: Safe simulation-to-real robot learning using diverse policies learned in simulation, *IEEE Robot. Autom. Lett.* 7 (3) (2022) 6838–6845, <http://dx.doi.org/10.1109/LRA.2022.3177294>.
- [163] H. Lai, W. Zhang, X. He, C. Yu, Z. Tian, Y. Yu, J. Wang, Sim-to-real transfer for quadrupedal locomotion via terrain transformer, in: *IEEE International Conference on Robotics and Automation, ICRA 2023*, London, UK, May 29 - June 2, 2023, IEEE, 2023, pp. 5141–5147, <http://dx.doi.org/10.1109/ICRA48891.2023.10160497>.
- [164] A.A. Rusu, M. Vecerik, T. Rothörl, N. Heess, R. Pascanu, R. Hadsell, Sim-to-real robot learning from pixels with progressive nets, in: *1st Annual Conference on Robot Learning, CoRL 2017*, Mountain View, California, USA, November 13-15, 2017, *Proceedings of Machine Learning Research*, vol. 78, PMLR, 2017, pp. 262–270.
- [165] K. Kang, S. Belkhal, G. Kahn, P. Abbeel, S. Levine, Generalization through simulation: Integrating simulated and real data into deep reinforcement learning for vision-based autonomous flight, in: *International Conference on Robotics and Automation, ICRA 2019*, Montreal, QC, Canada, May 20-24, 2019, IEEE, 2019, pp. 6008–6014, <http://dx.doi.org/10.1109/ICRA.2019.8793735>.
- [166] M. Wulfmeier, A. Abdolmaleki, R. Hafner, J.T. Springenberg, M. Neunert, T. Hertweck, T. Lampe, N. Siegel, N.M.O. Heess, M.A. Riedmiller, Regularized hierarchical policies for compositional transfer in robotics, 2019, *ArXiv*, [abs/1906.11228](https://api.semanticscholar.org/CorpusID:195657825). URL <https://api.semanticscholar.org/CorpusID:195657825>.
- [167] W. Yu, C.K. Liu, G. Turk, Policy transfer with strategy optimization, in: *7th International Conference on Learning Representations, ICLR 2019*, New Orleans, LA, USA, May 6-9, 2019, OpenReview.net, 2019, pp. 1–16, URL <https://openreview.net/forum?id=H1g6osRcFQ>.
- [168] A. Farchy, S. Barrett, P. MacAlpine, P. Stone, Humanoid robots learning to walk faster: from the real world to simulation and back, in: M.L. Gini, O. Shehory, T. Ito, C.M. Jonker (Eds.), *International Conference on Autonomous Agents and Multi-Agent Systems, AAMAS '13*, Saint Paul, MN, USA, May 6-10, 2013, IFAAMAS, 2013, pp. 39–46.
- [169] J.P. Hanna, P. Stone, Grounded action transformation for robot learning in simulation, in: S. Singh, S. Markovitch (Eds.), *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, February 4-9, 2017, San Francisco, California, USA, AAAI Press, 2017, pp. 3834–3840, <http://dx.doi.org/10.1609/AAAI.V31i1.11044>.
- [170] S. Desai, H. Karnan, J.P. Hanna, G. Warnell, P. Stone, Stochastic grounded action transformation for robot learning in simulation, in: *IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2020*, Las Vegas, NV, USA, October 24, 2020 - January 24, 2021, IEEE, 2020, pp. 6106–6111, <http://dx.doi.org/10.1109/IROS45743.2020.9340780>.
- [171] J.P. Hanna, S. Desai, H. Karnan, G.A. Warnell, P. Stone, Grounded action transformation for sim-to-real reinforcement learning, *Mach. Learn.* 110 (2021) 2469–2499.
- [172] H. Karnan, S. Desai, J.P. Hanna, G. Warnell, P. Stone, Reinforced grounded action transformation for sim-to-real transfer, in: *IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2020*, Las Vegas, NV, USA, October 24, 2020 - January 24, 2021, IEEE, 2020, pp. 4397–4402, <http://dx.doi.org/10.1109/IROS45743.2020.9341149>.
- [173] S. Ha, K. Yamane, Reducing hardware experiments for model learning and policy optimization, in: *IEEE International Conference on Robotics and Automation, ICRA 2015*, Seattle, WA, USA, 26-30 May, 2015, IEEE, 2015, pp. 2620–2626, <http://dx.doi.org/10.1109/ICRA.2015.7139552>.
- [174] F. Golemo, A.A. Taiga, A.C. Courville, P. Oudeyer, Sim-to-real transfer with neural-augmented robot simulation, in: *2nd Annual Conference on Robot Learning, CoRL 2018*, Zürich, Switzerland, 29-31 October 2018, *Proceedings of Machine Learning Research*, vol. 87, PMLR, 2018, pp. 817–828.
- [175] K. Ota, D.K. Jha, D. Romeres, J. van Baar, K.A. Smith, T. Semitsu, T. Oiki, A. Sullivan, D. Nikovski, J.B. Tenenbaum, Data-efficient learning for complex and real-time physical problem solving using augmented simulation, *IEEE Robot. Autom. Lett.* 6 (2021) 4241–4248.
- [176] Y. Liu, H. Xu, D. Liu, L. Wang, A digital twin-based sim-to-real transfer for deep reinforcement learning-enabled industrial robot grasping, *Robot. Comput. Integr. Manuf.* 78 (2022) 102365.
- [177] S.W. Abeyruwan, L. Graesser, D.B. D'Ambrosio, A. Singh, A. Shankar, A. Bewley, D. Jain, K.M. Choromanski, P.R. Sanketi, I-Sim2Real: Reinforcement learning of robotic policies in tight human-robot interaction loops, in: K. Liu, D. Kulic, J. Ichnowski (Eds.), *Conference on Robot Learning, CoRL 2022*, 14-18 December 2022, Auckland, New Zealand, in: *Proceedings of Machine Learning Research*, vol. 205, PMLR, 2022, pp. 212–224.
- [178] R. Laroche, M. Barlier, Transfer reinforcement learning with shared dynamics, in: S. Singh, S. Markovitch (Eds.), *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, February 4-9, 2017, San Francisco, California, USA, AAAI Press, 2017, pp. 2147–2153, <http://dx.doi.org/10.1609/AAAI.V31i1.10796>.
- [179] A. Barreto, W. Dabney, R. Munos, J.J. Hunt, T. Schaul, D. Silver, H. van Hasselt, Successor features for transfer in reinforcement learning, in: I. Guyon, U. von Luxburg, S. Bengio, H.M. Wallach, R. Fergus, S.V.N. Vishwanathan, R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017*, December 4-9, 2017, Long Beach, CA, USA, 2017, pp. 4055–4065.
- [180] Y. Chen, X. Li, S. Guo, X. Ng, M.H. Ang, Real2Sim or Sim2Real: Robotics visual insertion using deep reinforcement learning and Real2Sim policy adaptation, in: I. Petrovic, E. Menegatti, I. Markovic (Eds.), *Intelligent Autonomous Systems 17 - Proceedings of the 17th International Conference IAS-17*, Zagreb, Croatia, 13-16 June 2022, in: *Lecture Notes in Networks and Systems*, vol. 577, Springer, 2022, pp. 617–629, http://dx.doi.org/10.1007/978-3-031-22216-0_41.
- [181] G. Zhu, M. Zhang, H. Lee, C. Zhang, Bridging imagination and reality for model-based deep reinforcement learning, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020*, NeurIPS 2020, December 6-12, 2020, Virtual, 2020, p. 1.
- [182] B. Eysenbach, S. Chaudhari, S. Asawa, S. Levine, R. Salakhutdinov, Off-dynamics reinforcement learning: Training for transfer with domain classifiers, in: *9th International Conference on Learning Representations, ICLR 2021*, Virtual Event, Austria, May 3-7, 2021, OpenReview.net, 2021, p. 1.
- [183] M. Wulfmeier, I. Posner, P. Abbeel, Mutual alignment transfer learning, in: *1st Annual Conference on Robot Learning, CoRL 2017*, Mountain View, California, USA, November 13-15, 2017, *Proceedings of Machine Learning Research*, vol. 78, PMLR, 2017, pp. 281–290.
- [184] J. Vacaro, G. Marques, B. Oliveira, G. Paz, T.S. Paula, W. Staehler, D. Murphy, Sim-to-real in reinforcement learning for everyone, in: *Latin American Robotics Symposium, Brazilian Symposium on Robotics, and Workshop on Robotics in Education (WRE), LARS/SBR/WRE 2019*, Rio Grande, Brazil, October 23-25, 2019, IEEE, 2019, pp. 305–310, <http://dx.doi.org/10.1109/LARS-SBR-WRE48964.2019.00060>.
- [185] S.H. Chung, S.-H. Kong, S.J. Cho, I.M.A. Nahrendra, Segmented encoding for Sim2Real of RL-based end-to-end autonomous driving, in: *2022 IEEE Intelligent Vehicles Symposium, IV*, 2022, pp. 1290–1296.
- [186] S. Gamrian, Y. Goldberg, Transfer learning for related reinforcement learning tasks via image-to-image translation, in: K. Chaudhuri, R. Salakhutdinov (Eds.), *Proceedings of the 36th International Conference on Machine Learning, ICML 2019*, 9-15 June 2019, Long Beach, California, USA, in: *Proceedings of Machine Learning Research*, vol. 97, PMLR, 2019, pp. 2063–2072.
- [187] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kececy, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, S. Levine, V. Vanhoucke, Using simulation and domain adaptation to improve efficiency of deep robotic grasping, in: *2018 IEEE International Conference on Robotics and Automation, ICRA*, 2018, pp. 4243–4250.
- [188] J. Truong, S. Chernova, D. Batra, Bi-directional domain adaptation for Sim2Real transfer of embodied navigation agents, *IEEE Robot. Autom. Lett.* 6 (2020) 2634–2641.
- [189] J. Liang, V. Makovychuk, A. Handa, N. Chentanez, M. Macklin, D. Fox, GPU-accelerated robotic simulation for distributed reinforcement learning, in: *2nd Annual Conference on Robot Learning, CoRL 2018*, Zürich, Switzerland, 29-31 October 2018, *Proceedings of Machine Learning Research*, vol. 87, PMLR, 2018, pp. 270–282.
- [190] L. Da, J. Turnau, T.P. Kutralangam, A. Velasquez, P. Shakarian, H. Wei, A survey of sim-to-real methods in RL: progress, prospects and challenges with foundation models, 2025, <http://dx.doi.org/10.48550/ARXIV.2502.13187>, CoRR, [abs/2502.13187](https://arxiv.org/abs/2502.13187). arXiv:2502.13187.
- [191] J. Gao, M.Y. Michelis, A. Spielberg, R.K. Katschmann, Sim-to-real of soft robots with learned residual physics, *IEEE Robot. Autom. Lett.* 9 (10) (2024) 8523–8530, <http://dx.doi.org/10.1109/LRA.2024.3446287>.
- [192] D. Baek, Y. Sim, A. Purushottam, S. Gupta, J. Ramos, Real-to-sim adaptation via high-fidelity simulation to control a wheeled-humanoid robot with unknown dynamics, 2024, <http://dx.doi.org/10.48550/ARXIV.2403.10948>, CoRR, [abs/2403.10948](https://arxiv.org/abs/2403.10948). arXiv:2403.10948.
- [193] S. Salimpour, J.P. Queralta, D. Paez-Granados, J. Heikkonen, T. Westerlund, Sim-to-real transfer for mobile robots with reinforcement learning: from NVIDIA isaac sim to gazebo and real ROS 2 robots, 2025, <http://dx.doi.org/10.48550/ARXIV.2501.02902>, CoRR, [abs/2501.02902](https://arxiv.org/abs/2501.02902). arXiv:2501.02902.
- [194] X. Yang, C. Eppner, J. Tremblay, D. Fox, S. Birchfield, F. Ramos, Robot policy evaluation for sim-to-real transfer: A benchmarking perspective, 2025, <http://dx.doi.org/10.48550/ARXIV.2508.11117>, CoRR, [abs/2508.11117](https://arxiv.org/abs/2508.11117). arXiv:2508.11117.
- [195] J. Kober, J.A. Bagnell, J. Peters, Reinforcement learning in robotics: A survey, *Int. J. Robot. Res.* 32 (11) (2013) 1238–1274, <http://dx.doi.org/10.1177/0278364913495721>.
- [196] J. Abou-Chakra, L. Sun, K. Rana, B.B. May, K. Schmeckpeper, M.V. Minniti, L. Herlant, Real-is-sim: Bridging the sim-to-real gap with a dynamic digital twin for real-world robot policy evaluation, 2025, <http://dx.doi.org/10.48550/ARXIV.2504.03597>, CoRR, [abs/2504.03597](https://arxiv.org/abs/2504.03597). arXiv:2504.03597.

- [197] J.H. Park, S. Park, H.J. Kim, Model-based domain randomization of dynamics system with deep Bayesian locally linear embedding, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, 2021, pp. 4223–4229, <http://dx.doi.org/10.1109/ICRA48506.2021.9561942>.
- [198] Y. Du, O. Watkins, T. Darrell, P. Abbeel, D. Pathak, Auto-tuned sim-to-real transfer, in: 2021 IEEE International Conference on Robotics and Automation, ICRA, IEEE Press, 2021, pp. 1290–1296, <http://dx.doi.org/10.1109/ICRA48506.2021.9562091>.
- [199] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, S. Levine, Learning complex dexterous manipulation with deep reinforcement learning and demonstrations, in: H. Kress-Gazit, S.S. Srinivasa, T. Howard, N. Atanasov (Eds.), *Robotics: Science and Systems XIV*, Carnegie Mellon University, Pittsburgh, Pennsylvania, USA, June 26–30, 2018, 2018, p. 1, <http://dx.doi.org/10.15607/RSS.2018.XIV.049>.
- [200] D.J. Eckman, S.G. Henderson, S. Shashaani, Diagnostic tools for evaluating and comparing simulation-optimization algorithms, *INFORMS J. Comput.* 35 (2) (2023) 350–367, <http://dx.doi.org/10.1287/IJOC.2022.1261>.
- [201] X.B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, Sim-to-real transfer of robotic control with dynamics randomization, in: 2018 IEEE International Conference on Robotics and Automation, ICRA, IEEE Press, 2018, pp. 1–8, <http://dx.doi.org/10.1109/ICRA.2018.8460528>.
- [202] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, J. Peters, Robot learning from randomized simulations: A review, *Front. Robot. AI* 9 (2022) 799893, <http://dx.doi.org/10.3389/FROBT.2022.799893>.
- [203] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2017, Vancouver, BC, Canada, September 24–28, 2017, IEEE, 2017, pp. 23–30, <http://dx.doi.org/10.1109/IROS.2017.8202133>.
- [204] D.J. Rezende, S. Mohamed, Variational inference with normalizing flows, in: F.R. Bach, D.M. Blei (Eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6–11 July 2015*, in: *JMLR Workshop and Conference Proceedings*, vol. 37, JMLR.org, 2015, pp. 1530–1538.
- [205] A. Curtis, E. Li, M. Noseworthy, N. Gothoskar, S. Chitta, H. Li, L.P. Kaelbling, N. Carey, Flow-based domain randomization for learning and sequencing robotic skills, 2025, <http://dx.doi.org/10.48550/ARXIV.2502.01800>, CoRR, arXiv:2502.01800.
- [206] T. Huang, N. Sontakke, K.N. Kumar, I. Essa, S. Nikolaidis, D.W. Hong, S. Ha, BayRnTune: Adaptive Bayesian domain randomization via strategic fine-tuning, in: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2024, pp. 670–676, <http://dx.doi.org/10.1109/IROS58592.2024.10802145>.
- [207] A. Zook, F.-Y. Sun, J. Spjut, V. Blukis, S.T. Birchfield, J. Tremblay, GRS: Generating robotic simulation tasks from real-world images, in: 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, CVPRW, 2024, pp. 594–603.
- [208] G. Liu, Y. Deng, R. Zhao, H. Zhou, J. Chen, J. Chen, R. Xu, Y. Tai, K. Jia, DexScale: Automating data scaling for Sim2Real generalizable robot control, in: *Forty-Second International Conference on Machine Learning*, 2025, p. 1.
- [209] R. Wang, J. Zhang, J. Chen, Y. Xu, P. Li, T. Liu, H. Wang, DexGraspNet: A large-scale robotic dexterous grasp dataset for general objects based on simulation, in: 2023 IEEE International Conference on Robotics and Automation, ICRA, 2022, pp. 11359–11366.
- [210] J. Zhang, H. Liu, D. Li, X. Yu, H. Geng, Y. Ding, J. Chen, H. Wang, DexGraspNet 2.0: Learning generative dexterous grasping in large-scale synthetic cluttered scenes, in: *Conference on Robot Learning*, 2024, p. 1.
- [211] Z. Xie, Z. Liu, Z. Peng, W. Wu, B. Zhou, Vid2Sim: Realistic and interactive simulation from video for urban navigation, in: 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2025, pp. 1581–1591.
- [212] D.J. Eckman, S.G. Henderson, S. Shashaani, SimOpt: A testbed for simulation-optimization experiments, *INFORMS J. Comput.* 35 (2023) 495–508.
- [213] Z. Lu, Y. Wang, A differentiable dynamic modeling approach to integrated motion planning and actuator physical design for mobile manipulators, *J. Field Robot.* 42 (2024) 37–64.
- [214] N. Fey, G.B. Margolis, M. Peticco, P. Agrawal, Bridging the sim-to-real gap for athletic loco-manipulation, 2025, <http://dx.doi.org/10.48550/ARXIV.2502.10894>, CoRR, abs/2502.10894, arXiv:2502.10894.
- [215] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, Learning agile and dynamic motor skills for legged robots, *Sci. Robot.* 4 (2019).
- [216] B. van der Heijden, J. Kober, R. Babuka, L. Ferranti, REX: GPU-accelerated Sim2Real framework with delay and dynamics estimation, *Trans. Mach. Learn. Res.* 2025 (2025).
- [217] S. Bauer, M. Wüthrich, F. Widmaier, A. Buchholz, S. Stark, A. Goyal, T. Steinbrenner, J. Akpo, S. Joshi, V. Berenz, V. Agrawal, N. Funk, J.U.D. Jesus, J. Peters, J. Watson, C. Chen, K. Srinivasan, J. Zhang, J. Zhang, M.R. Walter, R. Madan, T. Yoneda, D. Yarats, A. Allshire, E.K. Gordon, T. Bhattacharjee, S.S. Srinivasa, A. Garg, T. Maeda, H. Sikichi, J. Wang, Q. Yao, S. Yang, R. McCarthy, F.R. Sanchez, Q. Wang, D.C. Bulens, K. McGuinness, N.E. O'Connor, S.J. Redmond, B. Schölkopf, Real robot challenge: A robotics competition in the cloud, in: D. Kiela, M. Ciccone, B. Caputo (Eds.), *NeurIPS 2021 Competitions and Demonstrations Track*, 6–14 December 2021, Online, in: *Proceedings of Machine Learning Research*, vol. 176, PMLR, 2021, pp. 190–204.
- [218] S. Yenamandra, A. Ramachandran, M. Khanna, K. Yadav, J. Vakil, A. Melnik, M. Büttner, L. Harz, L. Brown, G.C. Nandi, A.P. S. G.K. Yadav, R. Kala, R. Haschke, Y. Luo, J. Zhu, Y. Han, B. Lu, X. Gu, Q. Liu, Y. Zhao, Q. Ye, C. Dou, Y. Chua, V. Kuzma, V. Humenny, R. Partsey, J. Francis, D.S. Chaplot, G. Chhablani, A. Clegg, T. Gervet, V. Jain, R. Ramrakhy, A. Szot, A.S. Wang, T. Yang, A. Edsinger, C.C. Kemp, B. Shah, Z. Kira, D. Batra, R. Mottaghi, Y. Bisk, C. Paxton, Towards open-world mobile manipulation in homes: Lessons from the neurips 2023 HomeRobot open vocabulary mobile manipulation challenge, 2024, <http://dx.doi.org/10.48550/ARXIV.2407.06939>, CoRR, abs/2407.06939, arXiv:2407.06939.
- [219] P. De Roovere, S. Moonen, N. Michiels, F. Wyffels, Sim-to-real dataset of industrial metal objects, *Machines* 12 (2) (2024) <http://dx.doi.org/10.3390/machines12020099>, URL <https://www.mdpi.com/2075-1702/12/2/99>.
- [220] X. Xiao, Z. Xu, A. Datar, G. Warnell, P. Stone, J.J. Damanik, J. Jung, C.A. Deresa, T.D. Huy, C. Jinyu, C. Yichen, J.A. Cahyono, J. Wu, L. Mo, M. Lv, B. Lan, Q. Meng, W. Tao, L. Cheng, Autonomous ground navigation in highly constrained spaces: Lessons learned from the third BARN challenge at ICRA 2024 [competitions], *IEEE Robot. Autom. Mag.* 31 (3) (2024) 197–204, <http://dx.doi.org/10.1109/MRA.2024.3427891>.
- [221] M. Wüthrich, F. Widmaier, F. Grimmering, S. Joshi, V. Agrawal, B. Hammoud, M. Khadi, M. Bogdanovic, V. Berenz, J. Viereck, M. Naveau, L. Righetti, B. Schölkopf, S. Bauer, TriFinger: An open-source robot for learning dexterity, in: J. Kober, F. Ramos, C.J. Tomlin (Eds.), *4th Conference on Robot Learning, CoRL 2020*, 16–18 November 2020, Virtual Event / Cambridge, MA, USA, in: *Proceedings of Machine Learning Research*, vol. 155, PMLR, 2020, pp. 1871–1882.
- [222] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sabaghian, C. Jaehne, L. Hausperger, S. Haddadin, The franka emika robot: A reference platform for robotics research and education, *IEEE Robot. Autom. Mag.* 29 (2022) 46–64.
- [223] A. Fickinger, A. Bendahi, S. Russell, Provable sim-to-real transfer via offline domain randomization, 2025, <http://dx.doi.org/10.48550/ARXIV.2506.10133>, CoRR, abs/2506.10133, arXiv:2506.10133.
- [224] A. Wagenmaker, K. Huang, L. Ke, K.G. Jamieson, A. Gupta, Overcoming the sim-to-real gap: Leveraging simulation to learn to explore for real-world RL, in: A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J.M. Tomczak, C. Zhang (Eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024*, Vancouver, BC, Canada, December 10–15, 2024, 2024, p. 1.
- [225] A. Curtis, E. Li, M. Noseworthy, N. Gothoskar, S. Chitta, H. Li, L.P. Kaelbling, N. Carey, Flow-based domain randomization for learning and sequencing robotic skills, 2025, <http://dx.doi.org/10.48550/ARXIV.2502.01800>, CoRR, abs/2502.01800, arXiv:2502.01800.
- [226] S. Bauer, M. Wüthrich, F. Widmaier, A. Buchholz, S. Stark, A. Goyal, T. Steinbrenner, J. Akpo, S. Joshi, V. Berenz, V. Agrawal, N. Funk, J.U.D. Jesus, J. Peters, J. Watson, C. Chen, K. Srinivasan, J. Zhang, J. Zhang, M.R. Walter, R. Madan, T. Yoneda, D. Yarats, A. Allshire, E.K. Gordon, T. Bhattacharjee, S.S. Srinivasa, A. Garg, T. Maeda, H. Sikichi, J. Wang, Q. Yao, S. Yang, R. McCarthy, F.R. Sanchez, Q. Wang, D.C. Bulens, K. McGuinness, N.E. O'Connor, S.J. Redmond, B. Schölkopf, Real robot challenge: A robotics competition in the cloud, in: D. Kiela, M. Ciccone, B. Caputo (Eds.), *NeurIPS 2021 Competitions and Demonstrations Track*, 6–14 December 2021, Online, in: *Proceedings of Machine Learning Research*, vol. 176, PMLR, 2021, pp. 190–204.
- [227] S. Yenamandra, A. Ramachandran, K. Yadav, A. Wang, M. Khanna, T. Gervet, T.-Y. Yang, V. Jain, A.W. Clegg, J. Turner, Z. Kira, M. Savva, A. Chang, D.S. Chaplot, D. Batra, R. Mottaghi, Y. Bisk, C. Paxton, HomeRobot: Open-vocabulary mobile manipulation, 2024, arXiv:2306.11565.
- [228] X. Yang, C. Eppner, J. Tremblay, D. Fox, S. Birchfield, F. Ramos, Robot policy evaluation for sim-to-real transfer: A benchmarking perspective, 2025, Poster, RobotEvaluation@RSS 2025. Preprint available at <https://openreview.net/forum?id=WzNxyd14Sh>.
- [229] P. Chattopadhyay, J. Hoffman, R. Mottaghi, A. Kembhavi, RobustNav: Towards benchmarking robustness in embodied navigation, in: 2021 IEEE/CVF International Conference on Computer Vision, ICCV 2021, Montreal, QC, Canada, October 10–17, 2021, IEEE, 2021, pp. 15671–15680, <http://dx.doi.org/10.1109/ICCV48922.2021.01540>.
- [230] A. Bhardwaj, J. Rothfuss, B. Sukhija, Y. As, M. Hutter, S. Coros, A. Krause, Data-efficient task generalization via probabilistic model-based meta reinforcement learning, *IEEE Robot. Autom. Lett.* 9 (2023) 3918–3925.
- [231] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, S. Levine, Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning, in: L.P. Kaelbling, D. Kragic, K. Sugiura (Eds.), *3rd Annual Conference on Robot Learning, CoRL 2019*, Osaka, Japan, October 30–November 1, 2019, Proceedings, in: *Proceedings of Machine Learning Research*, vol. 100, PMLR, 2019, pp. 1094–1100, URL <http://proceedings.mlr.press/v100/yu20a.html>.

- [232] S. Park, O. Rybkin, S. Levine, METRA: scalable unsupervised RL with metric-aware abstraction, in: The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, OpenReview.net, 2024, p. 1.
- [233] C. Finn, P. Abbeel, S. Levine, Model-agnostic meta-learning for fast adaptation of deep networks, in: D. Precup, Y.W. Teh (Eds.), Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017, in: Proceedings of Machine Learning Research, vol. 70, PMLR, 2017, pp. 1126–1135.
- [234] K. Rakelly, A. Zhou, C. Finn, S. Levine, D. Quillen, Efficient off-policy meta-reinforcement learning via probabilistic context variables, in: K. Chaudhuri, R. Salakhutdinov (Eds.), Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA, in: Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 5331–5340.
- [235] L. Grillotti, L. Coiffard, O. Pang, M. Faldor, A. Cully, From tabula rasa to emergent abilities: Discovering robot skills via real-world unsupervised quality-diversity, 2025, ArXiv, [abs/2508.19172](https://arxiv.org/abs/2508.19172). URL <https://api.semanticscholar.org/CorpusID:280870024>.
- [236] A. Wagenmaker, K. Huang, L. Ke, B. Boots, K.G. Jamieson, A. Gupta, Overcoming the sim-to-real gap: Leveraging simulation to learn to explore for real-world RL, 2024, ArXiv, [abs/2410.20254](https://arxiv.org/abs/2410.20254).
- [237] R. Rodríguez-Aguilar, J.A. Marmolejo-Saucedo, U. Köse, Development of a digital twin driven by a deep learning model for fault diagnosis of electro-hydraulic actuators, *Mathematics* (2024).
- [238] N. Reginald, O. Al-Buraiki, T. Choopojcharoen, B. Fidan, E. Hashemi, Visual-inertial-wheel odometry with slip compensation and dynamic feature elimination, *Sensors* (Basel, Switzerland) 25 (2025).
- [239] Y. Wang, Z. Jiang, J. Chen, AMP in the wild: Learning robust, agile, natural legged locomotion skills, 2023, <http://dx.doi.org/10.48550/ARXIV.2304.10888>, CoRR, [abs/2304.10888](https://arxiv.org/abs/2304.10888). arXiv:2304.10888.
- [240] L. Campanaro, S. Gangapurwala, W. Merkt, I. Havoutis, Learning and deploying robust locomotion policies with minimal dynamics randomization, in: A. Abate, M. Cannon, K. Margellos, A. Papachristodoulou (Eds.), 6th Annual Learning for Dynamics & Control Conference, 15-17 July 2024, University of Oxford, Oxford, UK, in: Proceedings of Machine Learning Research, vol. 242, PMLR, 2024, pp. 578–590.
- [241] H. Dong, H. Fu, W. Xu, Z. Zhou, C. Chen, EASI: evolutionary adversarial simulator identification for sim-to-real transfer, in: A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J.M. Tomczak, C. Zhang (Eds.), Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024, 2024, p. 1.
- [242] N. Nilaksh, A. Ranjan, S. Agrawal, A. Jain, P. Jagtap, S.N.Y. Kolathaya, Barrier functions inspired reward shaping for reinforcement learning, in: 2024 IEEE International Conference on Robotics and Automation, ICRA, 2024, pp. 10807–10813.
- [243] Z. Li, C. Hu, Y. Wang, Y. Yang, S.E. Li, Safe reinforcement learning with dual robustness, *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (2023) 10876–10890.
- [244] J.-B. Bouvier, K. Nagpal, N. Mehr, POLICed RL: Learning closed-loop robot control policies with provable satisfaction of hard constraints, 2024, ArXiv, [abs/2403.13297](https://arxiv.org/abs/2403.13297).